



Blockchain Protocol Security Analysis Report

Customer: Push Chain

Date: 04/09/2026



We express our gratitude to the PushChain team for the collaborative engagement that enabled the execution of this Blockchain Protocol Security Assessment.

Push Chain is a shared-state Layer 1 that brings together Cosmos-style blockchain infrastructure with full Ethereum app compatibility, so developers can build in familiar tools while the chain coordinates activity across networks. Rather than connecting chains one bridge at a time, Push acts as a central hub where cross-chain actions are observed, agreed on, and executed—enabling universal applications that can reach users and assets on many chains from one place.

Document

Name	Blockchain Protocol Review and Security Analysis Report for PushChain
Audited By	Tanuj Soni, Alfredo Ortega
Approved By	Ece Örsel
Website	https://push.org/
Changelog	15/05/2026 - Preliminary Report
Changelog	05/06/2026 - Second Preliminary Report
Changelog	29/06/2026 - Third Preliminary Report
Changelog	04/09/2026 - Final Report
Platform	PushChain
Language	Golang
Tags	Cosmos SDK, EVM compatible, Cross-chain hub
Methodology	https://docs.hacken.io/methodologies/blockchain-protocols

Review Scope

Repository	https://github.com/pushchain/push-chain-node
Initial Commit	<code>e6b7c77</code>
Final Commit	<code>5f0341e</code>
Repository	https://github.com/pushchain/push-chain-evm/
Initial Commit	<code>9570d87</code>
Final Commit	<code>7ac130d</code>

Audit Summary

The system users should acknowledge all the risks summed up in the risks section of the report

118

Total Findings

98

Resolved

19

Accepted

1

Mitigated

Findings by Severity

Severity

Count

Critical	0
High	8
Medium	18
Low	71

Vulnerability	Severity	Status
F-2026-16597 - Arbitrary memory allocation at UniversalClient	High	Fixed
F-2026-16963 - [PUSHCHAIN REPORTED] Issue 4 — EVM TX resolver marks "Tx not found" as reverted (double-spend edge case)	High	Fixed
F-2026-18195 - [Dual Defense] isCEA Smart-Contract Inbound Lifecycle Gaps Strand Bridged Principal	High	Fixed
F-2026-18197 - [Dual Defense] Nested Message Dispatch Bypasses EVM Ante for MsgEthereumTx	High	Fixed
F-2026-18198 - [Dual Defense] Solana Inbound Event Forgery Enables Unbacked Synthetic Minting	High	Fixed
F-2026-18199 - [Dual Defense] TSS Setup Message Signs an Unverified Hash (Payload vs SigningHash Split)	High	Fixed
F-2026-18200 - [Dual Defense] Variable-Length Signer Addresses Enable Universal Executor Module Impersonation	High	Fixed
F-2026-18201 - [Dual Defense] Staking Precompile and Vesting Underflow StateDB Balance Enabling Native Mint and Drain	High	Fixed
F-2026-16738 - ExecuteInboundFundsAndPayload State Desync	Medium	Fixed
F-2026-16740 - Solana RPC Signature Order Misassumption	Medium	Fixed
F-2026-16960 - [PUSHCHAIN-REPORTED] Issue 1: RPC failover retries same endpoint under concurrent load	Medium	Fixed
F-2026-16961 - [PUSHCHAIN REPORTED] Issue 2 — Fund migration failure due to hardcoded GasLimit and missing L1GasFeeBuffer	Medium	Fixed
F-2026-16962 - [PUSHCHAIN-REPORTED] Issue 3 — Fund migration broadcast retry storm after peer already migrated funds	Medium	Fixed

Vulnerability	Severity	Status
F-2026-16966 - [PUSHCHAIN-REPORTED] Issue 7 — Expiry sweeper votes REVERT based on push chain state, not observed chain state and can result into false voting	Medium	Fixed
F-2026-17087 - [PUSHCHAIN-REPORTED] Issue 9 - Inbound-revert outbound ID collides when a single source tx contains multiple Inbounds	Medium	Fixed
F-2026-18186 - [Dual Defense] Gasless New-Account Ante Path Skips Signature Limits and Persists Failed-Message Accounts	Medium	Fixed
F-2026-18187 - [Dual Defense] Gov Precompile getTallyResult Persists SDK Tally Vote Deletions	Medium	Fixed
F-2026-18191 - [Dual Defense] Byzantine TSS Coordinator Can Freeze Outbound Withdrawals via Gap Nonce	Medium	Fixed
F-2026-18194 - [Dual Defense] Unbacked Gas PRC20 Mint on INBOUND_REVERT via applyGasRefund	Medium	Fixed
F-2026-18196 - [Dual Defense] Native SOL Misclassified as SPL Due to EVM-Only isNative Check	Medium	Fixed
F-2026-18825 - [Dual Defense] One Invalid Gateway Sibling Erases Valid Outbounds After Committed UEA Burn	Medium	Fixed
F-2026-18826 - [Dual Defense] Receipt RPC Error Treated as Not-Found Can Remint After Destination Already Paid	Medium	Fixed
F-2026-18827 - [Dual Defense] Normal EVM Outbound Resolution Uses Current TSS Address After Key Rotation	Medium	Fixed
F-2026-18829 - [Dual Defense] UEA_SVM Hardcodes Ed25519 Verifier 0x...00ca After Runtime Moved to 0xEC...01	Medium	Fixed
F-2026-16998 - Hardcoded EOA as owner of every system-contract ProxyAdmin	Medium	Accepted
F-2026-18192 - [Dual Defense] Inactive Joiner Can Vote on Stale Inbound Ballot and Mint Unbacked PRC20	Medium	Accepted
F-2026-16039 - Inbound ballot identity derived from full serialized Inbound	Low	Fixed
F-2026-16599 - DoS via Slowloris attack on API server at UniversalClient	Low	Fixed
F-2026-16632 - Outbound ballot identity is coupled to full OutboundObservation encoding	Low	Fixed
F-2026-16642 - Pending inbound index can outlive terminal ballot state	Low	Fixed
F-2026-16874 - Coordinator retains prior validator set when refresh RPC fails	Low	Fixed
F-2026-16875 - Event confirmers advance confirmation depth without execution success	Low	Fixed
F-2026-16939 - Unbounded frame allocation in TSS libp2p readFramed	Low	Fixed
F-2026-16944 - Event Skipping via Pagination Failure in Solana Event Listener	Low	Fixed
F-2026-16964 - [PUSHCHAIN REPORTED] Issue 5 — SVM TX resolver lacks a way to differentiate invalid signature vs transient simulation failure	Low	Fixed
F-2026-16965 - [PUSHCHAIN REPORTED] Issue 6 — Reverted txs can get stuck due to architecture (failure visibility limited to signer set)	Low	Fixed

Vulnerability	Severity	Status
F-2026-16967 - [PUSHCHAIN-REPORTED] Issue 8 - MsgExecutePayload rejects when precomputed UEA is pre-funded but not deployed	Low	Fixed
F-2026-16991 - Stale active universal validators inflate ballot quorum and can deadlock finalization	Low	Fixed
F-2026-16992 - Removing an active universal validator bypasses the ongoing-TSS guard used elsewhere	Low	Fixed
F-2026-16993 - Votes are accepted on pending ballots without checking block height expiry	Low	Fixed
F-2026-17022 - Case-sensitive token keys versus case-insensitive PRC20 lookup and EIP-55 observer addresses	Low	Fixed
F-2026-17025 - isContractDeployed treats any account with a non-empty code hash as a deployed contract	Low	Fixed
F-2026-17038 - Genesis export/import omits pending TSS event index	Low	Fixed
F-2026-17041 - Fund migration ballot key uses raw txHash string	Low	Fixed
F-2026-17043 - usigverifier Ed25519 precompile verifies signature over ASCII hex, not the raw message	Low	Fixed
F-2026-17736 - Derived call can commit state even when commit=false	Low	Fixed
F-2026-17738 - Reverted derived execution still emits events and mutates block bloom	Low	Fixed
F-2026-17740 - KV indexer path misses derived transactions	Low	Fixed
F-2026-17745 - Derived transactions share a constant txIndex value	Low	Fixed
F-2026-17752 - Derived RPC transaction uses GasUsed as gas field	Low	Fixed
F-2026-17754 - TraceTransaction mixes Cosmos and Ethereum index domains	Low	Fixed
F-2026-17775 - Inconsistent log reporting for failed derived transactions across JSON-RPC endpoints	Low	Fixed
F-2026-18135 - [Dual Defense] Nested Stateful Precompile Double-Applies Balance Events (Upstream ASA-2026-002)	Low	Fixed
F-2026-18139 - [Dual Defense] Cross-RPC Height Skew Can Underflow Confirmation Counts and Prematurely Finalize Inbounds	Low	Fixed
F-2026-18140 - [Dual Defense] Underpriced Ed25519 Raw-Message Precompile Gas	Low	Fixed
F-2026-18142 - [Dual Defense] Fund Migration Signing Hash Uses Live Balance and Is Raceable by Dust Inflows	Low	Fixed
F-2026-18144 - [Dual Defense] Unchecked Cumulative GasWanted Can Fail FinalizeBlock Under Unbounded Block Gas	Low	Fixed
F-2026-18145 - [Dual Defense] Missing OP-Stack L1 Data-Fee Accounting Under-Reports Outbound Cost	Low	Fixed
F-2026-18146 - [Dual Defense] Oversized Gateway Payload Blinds Shipped Outbound Poll via gRPC Size Limit	Low	Fixed
F-2026-18147 - [Dual Defense] RecomputeBallotQuorum Does Not Finalize When Preserved Votes Already Meet Threshold	Low	Fixed

Vulnerability	Severity	Status
F-2026-18177 - [Dual Defense] User-Selected Rescue PRC20 Can Reinterpret a Stuck Inbound as Another Vault Asset	Low	Fixed
F-2026-18182 - [Dual Defense] Unbounded UEA Payload Gas Can Stall Synchronous EVM Execution	Low	Fixed
F-2026-18184 - [Dual Defense] Empty SVM CEA Recipient Encodes to Invalid Address and Strands Outbound	Low	Fixed
F-2026-18188 - [Dual Defense] IBC Timeout Callback Invokes CallEVM on Outer Context Instead of Cached Context	Low	Fixed
F-2026-18189 - [Dual Defense] Manual Module EVM Nonce Desync on Reverted Inbound Execution	Low	Fixed
F-2026-18190 - [Dual Defense] Obsolete TSS Keyshares Survive Quorum Change Without Deletion or Retirement	Low	Fixed
F-2026-18784 - [Dual Defense] Uppercase Bech32 Fee Payer Bypasses DIRECT_AUX Fee-Payer Guard	Low	Fixed
F-2026-18785 - [Dual Defense] RevertMultiStore EventManager / writeCache Desync	Low	Fixed
F-2026-18786 - [Dual Defense] Unknown Selector Still Runs CommitWithCacheCtx Before ABI Fail	Low	Fixed
F-2026-18793 - [Dual Defense] RecomputeBallotQuorum Applies 2/3+1 Even to TSS Ballots	Low	Fixed
F-2026-18795 - [Dual Defense] Unbounded Active-Ballot Expiry Scan on Create + Disabled 100M Expiry	Low	Fixed
F-2026-18797 - [Dual Defense] Universal Client Chain Registry Lifecycle: Stale-Remove Deadlock and Unclosed DB Handles	Low	Fixed
F-2026-18798 - [Dual Defense] UExecutor ValidateBasic Parses Unbounded Decimals Before Ante	Low	Fixed
F-2026-18799 - [Dual Defense] SIGN_FUND_MIGRATE Selects Current Sign Threshold, Not TssKeyHistory Shareholders	Low	Fixed
F-2026-18800 - [Dual Defense] Short SVM send_funds Defaults TxType=0 While Claiming Funds → Mapped to GAS	Low	Fixed
F-2026-18802 - [Dual Defense] EVM FilterLogs Fixed 9000-Block Chunks Stall Cursor With No Adaptive Shrink	Low	Fixed
F-2026-18803 - [Dual Defense] Gasless MsgVoteChainMeta Can Inflate Unregistered ChainMetas Keys	Low	Fixed
F-2026-18804 - [Dual Defense] EVM ABI readDynamicBytes Uint64 Overflow Panics Can Kill puniversald	Low	Fixed
F-2026-18815 - [Dual Defense] SVM Client Omits Relayer-Paid Recipient ATA Rent From GasFeeUsed	Low	Fixed
F-2026-18816 - [Dual Defense] Empty authz.MsgExec Vacuously Gasless Can Stuff Declared Block Gas	Low	Fixed
F-2026-18817 - [Dual Defense] SVM Log Truncation + Resolver No Success-Vote Starves Oldest-1000 Pending Queue	Low	Fixed

Vulnerability	Severity	Status
F-2026-18818 - [Dual Defense] CallEVMWithData Ignores gasCap — Callback Gas Sandbox Broken	Low	Fixed
F-2026-18819 - [Dual Defense] IBC Refund Conversion Swallows EVM Error After Escrowing Coins Into erc20 Module	Low	Fixed
F-2026-18821 - [Dual Defense] Public InboundKeys Query Runs Unbounded Quadratic Solana Base58 Decode	Low	Fixed
F-2026-18822 - [Dual Defense] Native ERC20 Convert Skips Approval Log Check — Unbacked Mint on Malicious Pair	Low	Fixed
F-2026-18823 - [Dual Defense] buildRevertOutbound Fail-Open Leaves Unsignable INBOUND_REVERT and Blocks Rescue	Low	Fixed
F-2026-18824 - [Dual Defense] Failed DerivedEVMCallWithData / CallEVMWithData Skip Cosmos ConsumeGas	Low	Fixed
F-2026-16658 - Threshold signing ballots require all process participants	Low	Accepted
F-2026-16659 - Event-driven outbound observations set success unconditionally	Low	Accepted
F-2026-16868 - Fixed gas limit and fee for all vote transactions	Low	Accepted
F-2026-16876 - Observers bind to registry-supplied gateway addresses without on-chain contract integrity checks	Low	Accepted
F-2026-16940 - Non-idempotent ordering between failure vote submission and local persistence in the expiry sweeper	Low	Accepted
F-2026-18141 - [Dual Defense] Uninitialized Genesis UEA Factory Allows First Caller to Claim Ownership	Low	Accepted
F-2026-18796 - [Dual Defense] Outbound Ballot Key Hashes Client GasFeeUsed / ErrorMessage Strings	Low	Accepted
F-2026-18801 - [Dual Defense] REJECTED Inbounds Land in ExpiredInbounds With No Shipped Refund Hatch	Low	Accepted
F-2026-18820 - [Dual Defense] GAS Auto-Swap minPCOut = 95% of Live Spot Quote Is Self-Referential Slippage	Low	Accepted
F-2026-17758 - Missing upstream security patch train after fork point	Low	Mitigated
F-2026-16598 - Unsafe Status Modification Order in MarkBallotExpired() and MarkBallotFinalized()	Info	Fixed
F-2026-16616 - Chain Metadata Oracle Using First Vote Instead of Median Aggregation	Info	Fixed
F-2026-16648 - Default registry administrator in DefaultParams	Info	Fixed
F-2026-16867 - Logging and standard output disclose sensitive material	Info	Fixed
F-2026-16877 - Chains.Start returns successfully when the Push native chain client is not attached	Info	Fixed
F-2026-16994 - Nil Network on MsgUpdateUniversalValidator panics in validation and in the handler	Info	Fixed
F-2026-16996 - Unwired v2 migration overwrites every universal validator network identity with placeholders	Info	Fixed

Vulnerability	Severity	Status
F-2026-17024 - Nil nested configs on add/update messages cause panics in validation and logging	Info	Fixed
F-2026-17035 - Unpaginated query walks and full-collection scan in GetTokenConfigByPRC20	Info	Fixed
F-2026-18133 - [Dual Defense] Same-Block Validator Jailing Can Snapshot an Oversized Ballot Quorum	Info	Fixed
F-2026-18136 - [Dual Defense] Unauthenticated Peers Can Exhaust TSS Stream Quota and Stall Outbound Signing	Info	Fixed
F-2026-18148 - [Dual Defense] Removed Universal Validator Retains ChainMeta Vote Authority	Info	Fixed
F-2026-18789 - [Dual Defense] First Fund Migration id=0 Is Unvotable	Info	Fixed
F-2026-16614 - Centralized registry and Universal Validator roster	Info	Accepted
F-2026-16619 - ExecutePayload and binding of UniversalAccountId.owner to the signer	Info	Accepted
F-2026-16655 - Fund migration votes pass txHash without normalization	Info	Accepted
F-2026-17040 - TSS event status updates scan entire event store	Info	Accepted
F-2026-18137 - [Dual Defense] Unbounded Concurrent Debug Tracing Can Cause RPC Node Out-of-Memory	Info	Accepted
F-2026-18787 - [Dual Defense] IBC v2 dest_callback Memo Without Callbacks Middleware	Info	Accepted
F-2026-18788 - [Dual Defense] Unauthenticated TraceCall Timeout Stops Tracer Only	Info	Accepted
F-2026-18792 - [Dual Defense] SVM revert_universal_tx / funds_rescued → Success: true Matches Instruction Completion	Info	Accepted

Documentation quality

- The root `readme.md` provides a concise product overview, environment prerequisites, local network startup guidance, and a repository layout. It does not articulate functional requirements, explicit security assumptions, or normative guarantees for cross-chain execution paths.
- Documentation for the custom modules (`x/uexecutor`, `x/uregistry`, `x/uvalidator`, `x/utss`) is largely introductory. It does not systematically document message lifecycles, failure and recovery behavior, voting or quorum semantics, or administrative control boundaries to a level typically expected for independent protocol review.
- A consolidated technical specification covering trust model, finality and confirmation policy, upgrade authority, and state-transition invariants—is not present as a standalone deliverable. Protocol behavior is inferred primarily from source code and protobuf definitions the latter describe data shapes but do not substitute for a full protocol specification. `chain_metadata.json` includes placeholder or non-specific external references, which is consistent with metadata that has not been finalized for production publication.

Code quality

- The implementation builds on established Cosmos SDK and Cosmos EVM components in a conventional manner. Push-specific EVM system contracts are integrated using widely used proxy patterns, with contract bytecode embedded in the registry module rather than distributed as a separately versioned contract package.
- Some module scaffolding remains (for example unwired or placeholder migrations and example chain metadata). If activated or merged without additional review, such artifacts can increase operational and maintenance risk.
- The universal modules and Universal Client introduce substantial custom logic. Cross-cutting concerns including hash normalization, ballot or observation identity, and oracle-related updates—are not uniformly enforced across all components, which can complicate reasoning about end-to-end behavior.
- The repository provides a workable development setup: `Makefile`, Docker-oriented tooling, local and testnet-oriented scripts, and stated prerequisites (Go 1.23+, Docker, `jq`).

Architecture quality

- **Scalability:** The network is implemented as a single CometBFT-based L1 without sharding. Capacity is bounded by block limits, EVM execution cost, and off-chain operator throughput. Certain on-chain bookkeeping paths rely on linear scans over growing state, which may become a scaling consideration as history accumulates.
- **Decentralization:** Consensus follows a standard validator model. Cross-chain operation additionally depends on a distinct universal-validator role, registry-defined routing and asset policy, and EVM-layer upgrade control for system contracts. Together, these introduce administrative and operational trust assumptions beyond the base staking layer.
- **Interoperability:** The design targets interoperability via IBC, EVM interfaces, CosmWasm, and hub-style connectivity to external EVM and Solana networks through Universal Client. Cross-chain settlement is attestation-based and configuration-dependent rather than light-client-verified on a per-event basis.
- **Resilience:** End-to-end resilience of cross-chain flows is not fully evidenced by the reviewed materials. Operational availability depends on Universal Client processes, external RPC reliability,

and alignment between staking-derived validator state and universal-validator registry state. EVM system-contract upgrade authority represents a concentrated dependency in the overall architecture.

Table of Contents

System Overview	17
Custom Modules	17
EVM Precompiles	17
Risks	18
Findings	19
Vulnerability Details	19
F-2026-16597 - Arbitrary Memory Allocation At UniversalClient - High	19
F-2026-16963 - [PUSHCHAIN REPORTED] Issue 4 — EVM TX Resolver Marks "Tx Not Found" As Reverted (Double-Spend Edge Case) - High	21
F-2026-18195 - [Dual Defense] IsCEA Smart-Contract Inbound Lifecycle Gaps Strand Bridged Principal - High	22
F-2026-18197 - [Dual Defense] Nested Message Dispatch Bypasses EVM Ante For MsgEthereum Tx - High	25
F-2026-18198 - [Dual Defense] Solana Inbound Event Forgery Enables Unbacked Synthetic Minting - High	28
F-2026-18199 - [Dual Defense] TSS Setup Message Signs An Unverified Hash (Payload Vs SigningHash Split) - High	30
F-2026-18200 - [Dual Defense] Variable-Length Signer Addresses Enable Universal Executor Module Impersonation - High	32
F-2026-18201 - [Dual Defense] Staking Precompile And Vesting Underflow StateDB Balance Enabling Native Mint And Drain - High	34
F-2026-16738 - ExecuteInboundFundsAndPayload State Desync - Medium	36
F-2026-16740 - Solana RPC Signature Order Misassumption - Medium	38
2. Detailed Explanation	38
Technical Analysis	38
Impact	38
F-2026-16960 - [PUSHCHAIN-REPORTED] Issue 1: RPC Failover Retries Same Endpoint Under Concurrent Load - Medium	40
Why It Matters	6
F-2026-16961 - [PUSHCHAIN REPORTED] Issue 2 — Fund Migration Failure Due To Hardcoded GasLimit And Missing L1GasFeeBuffer - Medium	41
Why It Matters	6
F-2026-16962 - [PUSHCHAIN-REPORTED] Issue 3 — Fund Migration Broadcast Retry Storm After Peer Already Migrated Funds - Medium	43
Evidence / Permalinks (Audit-Main)	43
F-2026-16966 - [PUSHCHAIN-REPORTED] Issue 7 — Expiry Sweeper Votes REVERT Based On Push Chain State, Not Observed Chain State And Can Result Into False Voting - Medium	44
Why It Matters	6
Evidence / Permalinks (Audit-Main)	44
F-2026-16998 - Hardcoded EOA As Owner Of Every System-Contract ProxyAdmin - Medium	46

F-2026-17087 - [PUSHCHAIN-REPORTED] Issue 9 - Inbound-Revert Outbound ID Collides When A Single Source Tx Contains Multiple Inbounds - Medium	48
F-2026-18186 - [Dual Defense] Gasless New-Account Ante Path Skips Signature Limits And Persists Failed-Message Accounts - Medium	50
F-2026-18187 - [Dual Defense] Gov Precompile GetTallyResult Persists SDK Tally Vote Deletions - Medium	52
F-2026-18191 - [Dual Defense] Byzantine TSS Coordinator Can Freeze Outbound Withdrawals Via Gap Nonce - Medium	54
F-2026-18192 - [Dual Defense] Inactive Joiner Can Vote On Stale Inbound Ballot And Mint Unbacked PRC20 - Medium	56
F-2026-18194 - [Dual Defense] Unbacked Gas PRC20 Mint On INBOUND_REVERT Via ApplyGasRefund - Medium	58
F-2026-18196 - [Dual Defense] Native SOL Misclassified As SPL Due To EVM-Only IsNative Check - Medium	60
F-2026-18825 - [Dual Defense] One Invalid Gateway Sibling Erases Valid Outbounds After Committed UEA Burn - Medium	6
F-2026-18826 - [Dual Defense] Receipt RPC Error Treated As Not-Found Can Remint After Destination Already Paid - Medium	64
F-2026-18827 - [Dual Defense] Normal EVM Outbound Resolution Uses Current TSS Address After Key Rotation - Medium	66
F-2026-18829 - [Dual Defense] UEA_SVM Hardcodes Ed25519 Verifier 0x...00ca After Runtime Moved To 0xEC...01 - Medium	68
F-2026-16039 - Inbound Ballot Identity Derived From Full Serialized Inbound - Low	70
F-2026-16599 - DoS Via Slowloris Attack On API Server At UniversalClient - Low	6
F-2026-16632 - Outbound Ballot Identity Is Coupled To Full OutboundObservation Encoding - Low	74
F-2026-16642 - Pending Inbound Index Can Outlive Terminal Ballot State - Low	76
F-2026-16658 - Threshold Signing Ballots Require All Process Participants - Low	78
F-2026-16659 - Event-Driven Outbound Observations Set Success Unconditionally - Low	80
F-2026-16868 - Fixed Gas Limit And Fee For All Vote Transactions - Low	81
F-2026-16874 - Coordinator Retains Prior Validator Set When Refresh RPC Fails - Low	83
F-2026-16875 - Event Confirmers Advance Confirmation Depth Without Execution Success - Low	85
F-2026-16876 - Observers Bind To Registry-Supplied Gateway Addresses Without On-Chain Contract Integrity Checks - Low	87
F-2026-16939 - Unbounded Frame Allocation In TSS Libp2p ReadFramed - Low	89
F-2026-16940 - Non-Idempotent Ordering Between Failure Vote Submission And Local Persistence In The Expiry Sweeper - Low	91
F-2026-16944 - Event Skipping Via Pagination Failure In Solana Event Listener - Low	94
F-2026-16964 - [PUSHCHAIN REPORTED] Issue 5 — SVM TX Resolver Lacks A Way To Differentiate Invalid Signature Vs Transient Simulation Failure - Low	96
Description	96
F-2026-16965 - [PUSHCHAIN REPORTED] Issue 6 — Reverted Txns Can Get Stuck Due To Architecture (Failure Visibility Limited To Signer Set) - Low	97

F-2026-16967 - [PUSHCHAIN-REPORTED] Issue 8 - MsgExecutePayload Rejects When Precompiled UEA Is Pre-Funded But Not Deployed - Low	99
Background -Current Deployment Model	99
Description	99
F-2026-16991 - Stale Active Universal Validators Inflate Ballot Quorum And Can Deadlock Finalization - Low	101
F-2026-16992 - Removing An Active Universal Validator Bypasses The Ongoing-TSS Guard Used Elsewhere - Low	104
F-2026-16993 - Votes Are Accepted On Pending Ballots Without Checking Block Height Expiry - Low	106
F-2026-17022 - Case-Sensitive Token Keys Versus Case-Insensitive PRC20 Lookup And EIP-55 Observer Addresses - Low	109
F-2026-17025 - IsContractDeployed Treats Any Account With A Non-Empty Code Hash As A Deployed Contract - Low	112
F-2026-17038 - Genesis Export/Import Omits Pending TSS Event Index - Low	114
F-2026-17041 - Fund Migration Ballot Key Uses Raw TxHash String - Low	116
F-2026-17043 - Usigverifier Ed25519 Precompile Verifies Signature Over ASCII Hex, Not The Raw Message - Low	118
F-2026-17736 - Derived Call Can Commit State Even When Commit=False - Low	120
F-2026-17738 - Reverted Derived Execution Still Emits Events And Mutates Block Bloom - Low	121
F-2026-17740 - KV Indexer Path Misses Derived Transactions - Low	123
F-2026-17745 - Derived Transactions Share A Constant TxIndex Value - Low	125
F-2026-17752 - Derived RPC Transaction Uses GasUsed As Gas Field - Low	127
F-2026-17754 - TraceTransaction Mixes Cosmos And Ethereum Index Domains - Low	129
F-2026-17758 - Missing Upstream Security Patch Train After Fork Point - Low	131
F-2026-17775 - Inconsistent Log Reporting For Failed Derived Transactions Across JSON-RPC Endpoints - Low	134
F-2026-18135 - [Dual Defense] Nested Stateful Precompile Double-Applies Balance Events (Upstream ASA-2026-002) - Low	136
F-2026-18139 - [Dual Defense] Cross-RPC Height Skew Can Underflow Confirmation Counts And Prematurely Finalize Inbounds - Low	138
F-2026-18140 - [Dual Defense] Underpriced Ed25519 Raw-Message Precompile Gas - Low	140
F-2026-18141 - [Dual Defense] Uninitialized Genesis UEA Factory Allows First Caller To Claim Ownership - Low	142
F-2026-18142 - [Dual Defense] Fund Migration Signing Hash Uses Live Balance And Is Raceable By Dust Inflows - Low	144
F-2026-18144 - [Dual Defense] Unchecked Cumulative GasWanted Can Fail FinalizeBlock Under Unbounded Block Gas - Low	146
F-2026-18145 - [Dual Defense] Missing OP-Stack L1 Data-Fee Accounting Under-Reports Outbound Cost - Low	148
F-2026-18146 - [Dual Defense] Oversized Gateway Payload Blinds Shipped Outbound Poll Via G RPC Size Limit - Low	150
F-2026-18147 - [Dual Defense] RecomputeBallotQuorum Does Not Finalize When Preserved Votes Already Meet Threshold - Low	152

F-2026-18177 - [Dual Defense] User-Selected Rescue PRC20 Can Reinterpret A Stuck Inbound As Another Vault Asset - Low	154
F-2026-18182 - [Dual Defense] Unbounded UEA Payload Gas Can Stall Synchronous EVM Execution - Low	156
F-2026-18184 - [Dual Defense] Empty SVM CEA Recipient Encodes To Invalid Address And Strands Outbound - Low	158
F-2026-18188 - [Dual Defense] IBC Timeout Callback Invokes CallEVM On Outer Context Instead Of Cached Context - Low	160
F-2026-18189 - [Dual Defense] Manual Module EVM Nonce Desync On Reverted Inbound Execution - Low	162
F-2026-18190 - [Dual Defense] Obsolete TSS Keyshares Survive Quorum Change Without Deletion Or Retirement - Low	164
F-2026-18784 - [Dual Defense] Uppercase Bech32 Fee Payer Bypasses DIRECT_AUX Fee-Payer Guard - Low	166
F-2026-18785 - [Dual Defense] RevertMultiStore EventManager / WriteCache Desync - Low	168
F-2026-18786 - [Dual Defense] Unknown Selector Still Runs CommitWithCacheCtx Before ABI Failure - Low	170
F-2026-18793 - [Dual Defense] RecomputeBallotQuorum Applies 2/3+1 Even To TSS Ballots - Low	172
F-2026-18795 - [Dual Defense] Unbounded Active-Ballot Expiry Scan On Create + Disabled 100M Expiry - Low	174
F-2026-18796 - [Dual Defense] Outbound Ballot Key Hashes Client GasFeeUsed / ErrorMessage Strings - Low	176
F-2026-18797 - [Dual Defense] Universal Client Chain Registry Lifecycle: Stale-Remove Deadlock And Unclosed DB Handles - Low	178
F-2026-18798 - [Dual Defense] UExecutor ValidateBasic Parses Unbounded Decimals Before Ante - Low	180
F-2026-18799 - [Dual Defense] SIGN_FUND_MIGRATE Selects Current Sign Threshold, Not TssKeyHistory Shareholders - Low	182
F-2026-18800 - [Dual Defense] Short SVM Send_funds Defaults TxType=0 While Claiming Funds → Mapped To GAS - Low	184
F-2026-18801 - [Dual Defense] REJECTED Inbounds Land In ExpiredInbounds With No Shipped Refund Hatch - Low	186
F-2026-18802 - [Dual Defense] EVM FilterLogs Fixed 9000-Block Chunks Stall Cursor With No Adaptive Shrink - Low	188
F-2026-18803 - [Dual Defense] Gasless MsgVoteChainMeta Can Inflate Unregistered ChainMetadata Keys - Low	190
F-2026-18804 - [Dual Defense] EVM ABI ReadDynamicBytes Uint64 Overflow Panics Can Kill Punniversald - Low	192
F-2026-18815 - [Dual Defense] SVM Client Omits Relayer-Paid Recipient ATA Rent From GasFeeUsed - Low	194
F-2026-18816 - [Dual Defense] Empty Authz.MsgExec Vacuously Gasless Can Stuff Declared Block Gas - Low	196
F-2026-18817 - [Dual Defense] SVM Log Truncation + Resolver No Success-Vote Starves Oldest -1000 Pending Queue - Low	198

Impact	198
F-2026-18818 - [Dual Defense] CallEVMWithData Ignores GasCap — Callback Gas Sandbox Broken - Low	200
F-2026-18819 - [Dual Defense] IBC Refund Conversion Swallows EVM Error After Escrowing Coins Into Erc20 Module - Low	202
F-2026-18820 - [Dual Defense] GAS Auto-Swap MinPCOut = 95% Of Live Spot Quote Is Self-Referential Slippage - Low	204
F-2026-18821 - [Dual Defense] Public InboundKeys Query Runs Unbounded Quadratic Solana Base58 Decode - Low	206
F-2026-18822 - [Dual Defense] Native ERC20 Convert Skips Approval Log Check — Unbacked Mint On Malicious Pair - Low	208
F-2026-18823 - [Dual Defense] BuildRevertOutbound Fail-Open Leaves Unsignable INBOUND_REVERT And Blocks Rescue - Low	210
F-2026-18824 - [Dual Defense] Failed DerivedEVMCallWithData / CallEVMWithData Skip Cosmos ConsumeGas - Low	212
F-2026-16598 - Unsafe Status Modification Order In MarkBallotExpired() And MarkBallotFinalized() - Info	214
F-2026-16614 - Centralized Registry And Universal Validator Roster - Info	215
F-2026-16616 - Chain Metadata Oracle Using First Vote Instead Of Median Aggregation - Info	216
F-2026-16619 - ExecutePayload And Binding Of UniversalAccountId.Owner To The Signer - Info	217
F-2026-16648 - Default Registry Administrator In DefaultParams - Info	218
F-2026-16655 - Fund Migration Votes Pass TxHash Without Normalization - Info	219
F-2026-16867 - Logging And Standard Output Disclose Sensitive Material - Info	221
F-2026-16877 - Chains.Start Returns Successfully When The Push Native Chain Client Is Not Attached - Info	226
F-2026-16994 - Nil Network On MsgUpdateUniversalValidator Panics In Validation And In The Handler - Info	225
F-2026-16996 - Unwired V2 Migration Overwrites Every Universal Validator Network Identity With Placeholders - Info	227
F-2026-17024 - Nil Nested Configs On Add/Update Messages Cause Panics In Validation And Logging - Info	229
F-2026-17035 - Unpaginated Query Walks And Full-Collection Scan In GetTokenConfigByPRC20 - Info	231
F-2026-17040 - TSS Event Status Updates Scan Entire Event Store - Info	234
F-2026-18133 - [Dual Defense] Same-Block Validator Jailing Can Snapshot An Oversized Ballot Quorum - Info	236
F-2026-18136 - [Dual Defense] Unauthenticated Peers Can Exhaust TSS Stream Quota And Stall Outbound Signing - Info	238
F-2026-18137 - [Dual Defense] Unbounded Concurrent Debug Tracing Can Cause RPC Node Out-Of-Memory - Info	240
F-2026-18148 - [Dual Defense] Removed Universal Validator Retains ChainMeta Vote Authority - Info	242
F-2026-18787 - [Dual Defense] IBC V2 Dest_callback Memo Without Callbacks Middleware - Info	244

F-2026-18788 - [Dual Defense] Unauthenticated TraceCall Timeout Stops Tracer Only - Info	246
F-2026-18789 - [Dual Defense] First Fund Migration Id=0 Is Unvotable - Info	248
F-2026-18792 - [Dual Defense] SVM Revert_universal_tx / Funds_rescued → Success: True Matches Instruction Completion - Info	250
Observation Details	252
Disclaimers	253
Hacken Disclaimer	253
Technical Disclaimer	253
Appendix 1. Severity Definitions	254
Appendix 2. Scope	255
Appendix 3. Additional Valuables	256
Frameworks And Methodologies	256

System Overview

Push Chain is a Cosmos SDK Layer 1 on CometBFT with native EVM and CosmWasm support. The node binary is `pchaind` the native token is `upc`. Alongside standard Cosmos, EVM, IBC, and token-factory modules, the chain adds four custom modules for universal cross-chain apps: registry, validator voting, on-chain execution, and threshold-signing coordination.

External chains connect through a hub model: operators run `puniversald` to observe events, validators vote them into consensus, Push executes the workflow (including EVM), and signed transactions complete actions on destination chains. Selected user and validator transactions can be processed without gas fees when they use only approved message types.

Custom Modules

- **Universal Client** (`universalClient`)

Off-chain service run by universal validators alongside `pchaind`. Watches external chains (EVM and Solana), submits vote transactions to Push, and collectively signs outbound transactions via threshold cryptography so no single operator holds full signing authority. Persists the chain-specific state locally and coordinates TSS sessions between validators.

- **Universal Registry** (`x/uregistry`)

Defines which external chains, tokens, and gateways Push recognizes. Administrators manage this configuration; execution modules reject operations on unsupported or disabled routes.

- **Universal Validator** (`x/uvalidator`)

Maintains the universal validator set and runs ballot voting on cross-chain observations. When enough validators agree, results are passed to the executor and TSS modules.

- **Universal Executor** (`x/uexecutor`)

Runs universal workflows on Push: processing inbounds from other chains, tracking transaction status, executing user payloads, and handling outbounds. Integrates with the EVM layer for contracts and account logic.

- **Universal TSS** (`x/utss`)

Coordinates threshold signing and fund migration when validator keys rotate. On-chain votes align with off-chain signing performed by Universal Client operators.

EVM Precompiles

Push extends the standard EVM precompile set so Solidity can call into Cosmos: bech32, p256, staking, distribution, IBC transfer, bank, gov, slashing, and evidence.

Push also adds `usigverifier`, which lets contracts verify signatures from supported external chains (e.g. Ed25519).

Risks

- Core cross-chain contracts can be upgraded outside normal Cosmos governance; misuse could change system behavior and affect bridged assets.
- Settlement still relies on validator agreement, with limited automatic protection against incorrect votes.
- Key-management steps that need full participant agreement can stall if someone does not take part.
- Events may be recorded before they are fully settled elsewhere, which can lead to later mismatches with source chains.
- Accounts are tied to each source chain and owner, not automatically to one identity across all chains.

Findings

Vulnerability Details

[F-2026-16597](#) - Arbitrary memory allocation at UniversalClient - High

Description:

The `'decodePayload'` function in the Solana (SVM) transaction builder at `universalclient/chains/svm/tx_builder.go` is vulnerable to a memory exhaustion-based denial-of-service (DoS) attack. The function reads `'accountsCount'` (`uint32`) and `'ixDataLen'` (`uint32`) directly from an untrusted payload byte slice and immediately uses these values to allocate slices via `'make()'` without any upper-bound validation. A maliciously crafted payload with extremely large values can force the Go runtime to attempt allocating gigabytes of memory, triggering an "out of memory" panic that crashes the validator process.

In this code:

```
func decodePayload(payload []byte) ([]GatewayAccountMeta, []byte, uint8, [32]byte, error) {
    var targetProgram [32]byte
    // Minimum payload: accounts_count(4) + ix_data_len(4) + instruction_id(1) + target_program(32) = 41
    if len(payload) < 41 {
        return nil, nil, 0, targetProgram, fmt.Errorf("payload too short: %d bytes (minimum 41)", len(payload))
    }
    offset := 0
    accountsCount := binary.BigEndian.Uint32(payload[offset : offset+4])
    offset += 4

    accounts := make([]GatewayAccountMeta, accountsCount)
```

We can see how the amount of accounts (`accountsCount`) is never validated for either maximum or minimum values.

This could cause an arbitrary memory allocation that will ultimately cause a Out-of-Memory crash in the client.

Also, no payload validation is done before parsing of the fields.

Status:

Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 4/5

Severity: High

Recommendations

Resolution: This issue has been addressed and resolved in commit [17581bf](#).

[F-2026-16963](#) - [PUSHCHAIN REPORTED] Issue 4 — EVM TX resolver marks “Tx not found” as reverted (double-spend edge case) - High

Description: If a tx is not found on chain after broadcast, it is marked as reverted. This is unsafe if the signed tx was merely dropped from mempool due to gas spikes; a malicious UV could rebroadcast later when gas is lower (if nonce not yet consumed).

Status: Fixed

Classification

Impact Rate: 5/5

Likelihood Rate: 2/5

Severity: High

Recommendations

Remediation: Retry broadcast and only revert once nonce is consumed

Resolution: This issue has been addressed and resolved in commit [78a44a4](#).

[F-2026-18195](#) - [Dual Defense] isCEA Smart-Contract Inbound Lifecycle Gaps Strand Bridged Principal - High

Description:

On isCEA `FUNDS_AND_PAYLOAD` and `GAS_AND_PAYLOAD` inbound paths with a non-UEA contract recipient, UExecutor deposits PRC20 to the recipient, records a SUCCESS PCTx on the outer SDK context, then invokes `executeUniversalTx` via module `DerivedEVMCall` inside a separate `CacheContext` shared with fee deduction. Two independent defects on this smart-contract branch break the invariant that a finalized inbound must either complete destination delivery or return funds through a shipped recovery path.

Deposit commits before callback acceptance. The deposit and its SUCCESS PCTx are written outside the callback cache. Inline handler comments state that the deposit remains committed regardless of callback outcome. If `CallExecuteUniversalTx` or fee deduction fails, the cache is discarded and a FAILED PCTx is recorded, but the PRC20 credit remains. isCEA paths never schedule `INBOUND_REVERT`. Public CEA rescue requires the first PCTx to be FAILED (deposit never arrived on Push / funds still locked on source). A SUCCESS deposit followed by a FAILED callback therefore matches neither automated revert nor rescue. Admin `RevertStuckInbound` requires an expired ballot with no existing UTX and does not apply after finalization with a SUCCESS deposit.

Successful callback omits outbound attach. On smart-contract SUCCESS, after `CallExecuteUniversalTx`, `DeductGasFeesFromReceipt`, and `writeCache()`, the handler records the callback PCTx and returns without `AttachOutboundsToExistingUniversalTx(contractReceipt)`. Nested `UniversalGatewayPC` burns and `UniversalTxOutbound` logs therefore never become `OutboundTx` or `PendingOutbounds` entries. The UEA branch for the same handlers performs that attach. Derived EVM calls skip `PostTxProcessing`, so the EVM hook does not compensate. Because deposit and callback both succeed, CEA rescue remains ineligible and outbound fail remind cannot run.

Loss is scoped to the inbound sender who selected the CEA recipient, or to the party whose contract performed a nested gateway burn after receiving its own deposit. Funds remain as PRC20 at the contract address or as burned Push supply without a pending outbound; this is not third-party vault extraction. SUCCESS PCTx records and `ethereum_tx` attributes may still support upgrade or administrative reconstruction.

Impact

- PRC20 deposit commits before mandatory `executeUniversalTx` acceptance and survives callback or fee failure.

- isCEA paths omit `INBOUND_REVERT` when the callback fails after a successful deposit.
- CEA rescue eligibility requires `PcTx[0].Status == "FAILED"`, so SUCCESS deposit plus FAILED callback blocks rescue.
- Nested gateway burns on a successful smart-contract callback never enter `PendingOutbounds`; destination release and automated remind do not run.
- Admin stuck-ballot revert is inapplicable once the UTX is finalized with a SUCCESS deposit.
- EOA deposit-only and UEA attach paths are unaffected; scope is the isCEA smart-contract branch.

Status:

Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 3/5

Severity: High

Recommendations

Remediation:

1. Wrap deposit, `executeUniversalTx`, and fees in a single atomic cache, or reverse the deposit (burn or unmint) on callback or fee failure.
2. On smart-contract callback or fee failure after a successful deposit, schedule an isCEA-compatible return path (revert outbound, reverse mint, or source return).
3. On isCEA smart-contract SUCCESS, call `AttachOutboundsToExistingUniversalTx(sdkCtx, contractReceipt, utx)` mirroring the UEA branch. If attach fails, do not leave burned state without an outbound: revert the cache, schedule remind, or mark the UTX for administrative intervention.
4. Widen CEA rescue eligibility to cover `PcTx[0] == SUCCESS` with a later callback PCTx `FAILED`, and distinguish source-locked versus Push-minted accounting.
5. Optional defense in depth: extract outbounds from derived-call receipts, or reject runtime state where gateway burn logs appear without attach.
6. Regression coverage:
 - Reverting `executeUniversalTx` after a positive deposit does not leave stranded PRC20 without a recoverable outbound or PCTx state.
 - CEA smart-contract callback that invokes `UniversalGatewayPC` outbound produces non-empty `OutboundTx` and populated `PendingOutbounds`.

7. Administrative or upgrade tooling to rebuild pending outbounds from historical `PcTx.TxHash` for already stranded UTX identifiers.

Resolution:

The issue has been addressed in commit [3deb6d7](#).

[F-2026-18197](#) - [Dual Defense] Nested Message Dispatch Bypasses

EVM Ante for MsgEthereumTx - High

Description:

Push Chain enforces Ethereum signature verification (`VerifySender`), account nonce advancement, and Ethereum gas fee and block checks in the EVM ante handler. Top-level `RejectMessagesDecorator` rejects bare `MsgEthereumTx`, and `AuthzLimiterDecorator` rejects nested AuthZ payloads that authorize `MsgEthereumTx`. Those controls apply only to message sets inspected during ante; they do not cover every SDK module that later unpacks and dispatches embedded messages through `MsgServiceRouter`.

Two enabled dispatch paths reintroduce the same incomplete-mitigation class:

CosmWasm `stargate`. Contracts with the `stargate` capability emit SDK messages that the wasm keeper routes into `MsgServiceRouter` without traversing `app/ante`. `AuthzLimiterDecorator` does not inspect Stargate payloads produced during `MsgExecuteContract` execution. Direct `From` equal to a 32-byte contract address fails because `HexAddress` assumes a 20-byte buffer. The operative path uses an AuthZ grant ladder (`MsgGrant` of `MsgGrant`, then grant of `MsgEthereumTx`, then `MsgExec(MsgEthereumTx)`) with `From` set to a 20-byte Cosmos account while the raw Ethereum transaction remains victim-signed. AuthZ treats the attacker as the Cosmos signer; `ApplyTransaction` recovers the victim from ECDSA and moves victim funds.

x/group proposal execution. Group `doExecuteMsgs` unpacks proposal messages, runs `ensureMsgAuthZ` (all signers equal the group policy account), and invokes handlers through `MsgServiceRouter` without ante. `AuthzLimiterDecorator` walks only `authz.MsgExec` / `authz.MsgGrant` and does not unpack `group.MsgSubmitProposal` or `group.MsgExec`. A single-member group policy can embed `MsgEthereumTx` with `From` set to the 20-byte group policy account while the Ethereum payload is victim-signed. Group policy accounts are ordinary `AccAddresses`; CosmWasm upload rights are not required for this vector. The same path also admits unbounded Ethereum `GasLimit` values that ante would have constrained, degrading `PrepareProposal` once such a transaction is gossiped.

In both cases, `Keeper.EthereumTx` forwards to `ApplyTransaction` → `core.TransactionToMessage` (signature recovery) and does not invoke `VerifySender`. `ApplyMessageWithConfig` assumes intrinsic gas and block checks were performed in ante. Exploitation for fund movement requires an observable victim-signed Ethereum transaction (mempool or RPC); it is not silent drain of idle accounts.

Impact

- Fund drain by replaying a victim-signed `MsgEthereumTx` without nonce or `VerifySender` enforcement on nested dispatch paths.
- Nonce and fee checks that exist only in ante are skipped when `CosmWasm` stargate or `x/group` execute embedded messages.
- `PrepareProposal` availability degradation via unbounded Ethereum `GasLimit` on the group bypass path.
- Replay remains possible while the signed transfer remains executable against remaining balance; this is not unbounded native mint.
- `CosmWasm` vector requires deploy or instantiate rights plus AuthZ setup; group vector requires only group policy control and a victim-signed Ethereum transaction.

Status:

Fixed

Classification

Impact Rate: 5/5

Likelihood Rate: 3/5

Severity: High

Recommendations

Remediation:

1. Introduce a shared nested-message denylist (or guarded `MessageRouter`) used by `CosmWasm`, `x/group`, and any other unpack-and-dispatch module. Reject `MsgEthereumTx` and AuthZ nests that authorize it, with a nesting depth cap.
2. Custom WASM messenger or `MessageHandlerDecorator` that rejects `MsgEthereumTx` and recursively nested AuthZ Exec/Grant of that type. Disable `stargate` or allowlist only safe type URLs if stargate is not required.
3. Extend ante (or the shared walker) to unpack `group.MsgSubmitProposal` / `group.MsgExec` and reject disabled types before execution.
4. In `Keeper.EthereumTx`: require `VerifySender` before `ApplyTransaction`; enforce account nonce against sequence; bind Ethereum gas consumption to a Cosmos or block budget even when ante was skipped; never trust `msg.From` independently of recovery for non-matching AuthZ or group signers.
5. Regression coverage:
 - WASM contract cannot `MsgExec` / `MsgGrant` `MsgEthereumTx`; replay of the same signed Ethereum transaction via WASM fails nonce or sender checks.
 - Single-member group `EXEC_TRY` with embedded `MsgEthereumTx` fails `CheckTx` or `DeliverTx`.

- Excessive `GasLimit` infinite-loop proposals cannot hang `PrepareProposal`.
- Victim-signed transfer cannot execute via group with `From` equal to the policy account when the recovered signer differs.

Resolution:

The issue has been addressed in commit [8e28eed](#) and commit [cf39387](#).

[F-2026-18198](#) - [Dual Defense] Solana Inbound Event Forgery

Enables Unbacked Synthetic Minting - High

Description:

The Solana observer selects transactions merely because they reference the configured gateway address, then treats every `Program data:` log as a possible gateway event based only on its public 8-byte discriminator. The observer does not establish that the gateway program was the executing program when the log was emitted.

Solana's `getSignaturesForAddress` returns transactions that reference the supplied address in `accountKeys`. It does not require that the address was the invoked program, was writable, signed the transaction, or emitted any log. An attacker can include the gateway public key as an unused read-only account in an instruction to an attacker-controlled program.

Event discriminators identify an encoding schema; they are not secrets or authenticators. Any Solana program can call `sol_log_data` with bytes beginning with the same discriminator. An attacker-controlled program can emit a correctly encoded `send_funds` event containing a registered asset, arbitrary amount, and attacker recipient. All honest Universal Validators observe the same successful transaction and deterministically vote for the same forged inbound. After quorum, Push Chain mints PRC20 tokens without a corresponding source-chain deposit.

Impact

An unauthenticated attacker can:

- Forge a successful Solana transaction that references the gateway without invoking it.
- Cause all honest validators to attest to a nonexistent deposit.
- Mint unbacked registered PRC20 assets to an attacker-controlled Push address.
- Repeat with new Solana transaction signatures and log indexes.
- Attempt to sell, swap, or redeem unbacked assets, transferring deficit to liquidity providers, vault reserves, or other users.

Unauthorized minting corrupts synthetic supply. The attack requires no compromised validator and no gateway-program privilege. Realized attacker profit depends on swap/outbound liquidity and may be interrupted by operational controls.

Status:

Fixed

Classification

Impact Rate: 5/5

Likelihood Rate: 3/5

Severity: High

Recommendations

Remediation:

1. Parse `Program <id> invoke [depth]`, success, and failure log entries as a stack.
2. Accept `Program data:` only while the configured gateway program is the active invocation frame.
3. Correctly handle nested CPI invocation depth and malformed or unbalanced logs.
4. Prefer a structurally attributable event mechanism such as gateway CPI data tied to verified inner instructions.
5. Decode the transaction and verify the relevant top-level or inner instruction program ID against the configured gateway.
6. Cross-check claimed token, amount, sender, and vault using SPL token pre/post balances and gateway-controlled accounts.
7. Add negative tests in which an attacker program emits a valid gateway discriminator while merely referencing the gateway account.
8. Pause Solana inbound processing until fixed, then review historical inbounds for forged events.

Resolution: The issue has been addressed in commit [3bddedc](#).

[F-2026-18199](#) - [Dual Defense] TSS Setup Message Signs an Unverified Hash (Payload vs SigningHash Split) - High

Description:

Followers independently recompute the outbound (or fund-migration) hash and compare it only to `Message.UnsignedSigningReq.SigningHash`. They then pass coordinator-controlled `Message.Payload` into `DklsSignSessionFromSetup` as `setupData`. In DKLS23, the hash that is actually signed lives inside that setup blob; the follower's `messageHash` is used only in a post-hoc `ecdsa.Verify` inside `GetResult`, after signing-round shares have already been exchanged. Nothing in `verifyOutboundSigningRequest` or `verifyFundMigrationSigningRequest` reads `Payload` or calls `DklsDecodeMessage`.

`coordinator/types.go` `Message` carries `Payload` (DKLS setup bytes consumed by `DklsSignSessionFromSetup`) and `UnsignedSigningReq` (whose `SigningHash` followers verify), with no MAC or signature binding them. `createSignSetup` / `createFundMigrationSignSetup` normally align both; a malicious coordinator can substitute `Payload` after verification.

`verifyOutboundSigningRequest` rebuilds the hash via `GetOutboundSigningRequest(ctx, &outboundData, req.Nonce)` and compares only to `req.SigningHash`; `Payload` is unused. For `EventTypeSignOutbound` / `SignFundMigrate`, `createSession` invokes `dkls.NewSignSession(msg.Payload, ..., msg.UnsignedSigningReq.SigningHash, ...)`. `dkls/sign.go` creates the session with `DklsSignSessionFromSetup(setupData, ...)`; `messageHash` is stored only for `GetResult`'s after-the-fact verify.

Honest followers who verify `H_legit` but run `setup(H_attacker)` fail `GetResult` when the produced signature does not verify under `H_legit`, but shares for `H_attacker` have already been emitted. The malicious coordinator, running a local session whose `messageHash` matches the embedded hash, can finish and obtain a valid TSS signature over `H_attacker`.

Impact

Breaks the intended verify-then-sign-the-same-bytes invariant. A malicious coordinator can present `H_legit` for verification while embedding `H_attacker` in `Payload`, eliciting honest co-sign shares over `H_attacker`. Enables a TSS signature over an attacker-chosen 32-byte hash (e.g. a vault call digest) when DKLS participants complete the round. External vaults that authorize solely via TSS role will accept that signature for matching calldata.

The attacker must be a bonded Universal Validator elected as coordinator with a genuine CONFIRMED sign event in peers' stores. Honest nodes fail local post-hoc verify. Draining requires the adversary to assemble and

submit the external-chain transaction matching `H_attacker`; stock `handleSigningComplete` paths are built around the local outbound/migration event and `signingReq`. Coordinator role rotates by block window; not every block.

Status:

Fixed

Classification

Impact Rate: 5/5

Likelihood Rate: 3/5

Severity: High

Recommendations

Remediation:

1. Do not trust `msg.Payload` for sign sessions. After verifying `UnsignedSigningReq`, locally call `DklsSignSetupMsgNew(keyID, nil, verifiedSigningHash, participants)` and use that setup in `NewSignSession`.

Alternatively:

1. `DklsDecodeMessage(msg.Payload)` and require equality with `req.SigningHash` before ACK/BEGIN; or
2. Cryptographically bind Payload to `UnsignedSigningReq` under the coordinator's identity.

Add regression tests: mismatched Payload hash vs verified SigningHash must refuse session creation and produce no shares.

Resolution:

The issue has been addressed in commit [6a1403e](#).

[F-2026-18200](#) - [Dual Defense] Variable-Length Signer Addresses

Enable Universal Executor Module Impersonation - High

Description:

The Universal Executor Account (UEA) execution path accepts a Cosmos signer address that is not cryptographically bound to the transaction public key on the gasless new-account ante path. A transaction can declare a crafted 21-byte Bech32 signer while signing with an unrelated key pair. Cosmos SDK's default address verifier accepts non-empty addresses up to 255 bytes, and the application does not enforce a 20-byte length constraint.

When the declared signer is converted to an EVM caller address, `common.BytesToAddress` retains only the rightmost 20 bytes. An attacker can construct a 21-byte address whose final 20 bytes equal the `uexecutor` module address (`sha256("uexecutor")[:20]`). The UEA contract treats calls from that module address as trusted and skips owner-signature verification.

The attack requires no privileges, no victim interaction, and no Cosmos-layer fee. The attacker supplies a victim `UniversalAccountId` and arbitrary payload fields. Gasless fee exemption, missing public-key-to-signer binding in `AccountInitDecorator`, and address truncation in `GetAddressPair` compose into a complete impersonation chain.

Impact

An unauthenticated remote attacker can:

Target any deployed or prefunded UEA identified by a valid `UniversalAccountId`.

Execute arbitrary calls as that UEA without the owner's signature.

Transfer the UEA's native funds or invoke token contracts to transfer approved or held assets.

Charge execution gas to the victim UEA after execution.

Loss is bounded per victim UEA. The attacker must know a funded target; empty or dust UEAs are not economically viable because post-execution fees are charged to the UEA. Push Chain block production and synthetic supply are unaffected.

Status:

Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 4/5

Severity:**High****Recommendations****Remediation:**

1. In `AccountInitDecorator`, reject every signature whose public-key-derived address does not exactly equal the corresponding declared signer before verifying or creating an account.
2. Before every Cosmos-to-EVM conversion, require exactly `common.AddressLength` (20) bytes. `GetAddressPair` must return an error instead of allowing `common.BytesToAddress` to truncate.
3. Remove or tightly redesign the contract's unconditional module-caller signature bypass where possible.
4. Add an end-to-end regression test using `0x01 || moduleAddress` and an unrelated signing key; the transaction must fail in ante handling and must never invoke the UEA.

Resolution:

The issue has been addressed in commit [6a1403e](#).

[F-2026-18201](#) - [Dual Defense] Staking Precompile and Vesting Underflow StateDB Balance Enabling Native Mint and Drain - High

Description:

EVM `StateDB` balance tracks spendable bank coins, while Cosmos staking allows vesting accounts to delegate locked coins. After a staking-precompile delegate, `BalanceHandler.AfterBalanceChange` subtracts the full `coin_spent` amount from the cached EVM balance. `stateObject.SubBalance` uses unchecked `uint256` subtraction, so when delegated amount `D` exceeds spendable balance `S`, the result wraps. On commit, `Keeper.SetBalance` reconciles Cosmos bank to the inflated EVM view, minting or burning spendable native balance as required to match StateDB.

Queries and bank wrappers expose spendable balance into the EVM view. Vesting accounts may still delegate locked coins through Cosmos staking, so the staking precompile can emit `coin_spent` for an amount larger than the account's current StateDB (spendable) balance. Ante blocks `MsgCreateVestingAccount` inside AuthZ only; direct vesting-account creation to arbitrary addresses remains available.

The same underflow supports two impact variants:

Self-directed mint. After wrap, transfer an attacker-chosen finite amount to a fresh address and discard residual wrap (for example via EIP-6780 same-transaction sink or `SELFDESTRUCT`) so commit does not materialize the full wrapped value. Net native supply increases by the extracted amount after subtracting seed spendable balance and gas.

Cross-account drain. From a wrapped helper balance, transferring `$2^{256} - B$` to a victim with balance `B` zeros the victim in uint256 arithmetic and leaves `$B - 1$` on the helper. Commit burns the victim's spendable balance and credits the extracted amount to the attacker-controlled account. The victim need not sign the wrap transfer after the helper is wrapped.

Attack choreography for both variants is permissionless once funded: create a delayed vesting account at a predicted `CREATE` address, fund locked plus small spendable balance, deploy contract code onto that account, staking-precompile delegate with `$D > S$` , then perform the wrap transfer. Setup is multi-step and chain-visible.

Impact

- Unauthorized inflation of the native denom to attacker-controlled addresses.
- Permissionless theft of an attacker-chosen victim's spendable native balance once the helper account is wrapped.
- Loss bounded by victim spendable balance at commit time, or by attacker-chosen finite mint per cycle, plus attacker setup and gas

costs.

- Repeatable with new predicted addresses; no validator key or admin role required.
- Each cycle requires funding a vesting account (locked coins plus spendable) plus gas. Locked coins remain locked or delegated; they are not free capital.
- Native supply deltas and hybrid vesting-plus-code accounts are operationally detectable; emergency halt or upgrade can interrupt extraction.

Status:

Fixed

Classification

Impact Rate: 5/5

Likelihood Rate: 3/5

Severity: High

Recommendations

Remediation:

1. In `stateObject.SubBalance` and/or `StateDB.SubBalance`, revert or error on insufficient balance (`Cmp` / `SubOverflow`).
2. In `BalanceHandler.AfterBalanceChange`, never subtract more than the current StateDB balance; preferably sync from spendable bank truth or clamp with error.
3. Reject or specially handle precompile balance events for vesting accounts where spent locked coins exceed spendable EVM balance.
4. Harden or disable permissionless `MsgCreateVestingAccount` to arbitrary addresses if the product does not require it.
5. Regression coverage:
 - Vesting plus staking-precompile delegate with `D > S` must revert, not mint.
 - Victim spendable balance must remain unchanged across an attempted wrap-transfer extraction.

Resolution:

The issue has been addressed in commit [a220b4f](#) and commit [757157d](#).

[F-2026-16738](#) - ExecuteInboundFundsAndPayload State Desync - Medium

Description:

The vulnerability resides in the `ExecuteInboundFundsAndPayload` function within the `x/uexecutor` module. The execution flow performs an external smart contract call (`CallExecuteUniversalTx`) which modifies the EVM state, followed by a local state update (`DeductGasFeesFromReceipt`). If the gas fee deduction fails, the function catches the error, marks the Universal Transaction (UTX) status as `FAILED`, and returns `nil` to the Cosmos SDK runtime.

As function returns `nil`, Cosmos SDK doesn't revert it. There is no rollback, compensation, or state reconciliation mechanism. This creates a critical divergence between the Cosmos SDK module state and the EVM state, leading to unrecoverable accounting discrepancies and potential fund misallocation.

This behavior violates the principle of atomicity in transaction processing. By returning `nil` instead of propagating the error, the Cosmos SDK commits the transaction, thereby finalizing the state changes made by `CallExecuteUniversalTx` on the EVM side. However, the Cosmos module state records the transaction as failed. This creates a critical divergence:

1. **Irreversible State Change:** The EVM execution (e.g., token transfers, contract interactions) is permanently committed on the target chain.
2. **Accounting Mismatch:** The source chain believes the transaction failed, potentially leaving funds stranded or creating double-spend vulnerabilities if the user retries the transaction.

This flaw can be exploited by any user submitting a Universal Transaction where the EVM execution succeeds but the subsequent gas deduction fails (e.g., due to insufficient balance in the fee account or parsing errors). The result is a loss of funds or system integrity, as the executed payload has no corresponding valid receipt or refund path.

Status:

Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 1/5

Severity: Medium

Recommendations

Remediation:

To mitigate this vulnerability, implement one of the following strategies to ensure atomicity and state consistency:

Propagate Errors for Automatic Revert: Modify

`ExecuteInboundFundsAndPayload` to return the error from `DeductGasFeesFromReceipt` directly to the Cosmos SDK runtime. This will cause the entire transaction, including the EVM state changes, to be reverted, ensuring that no state is committed unless all steps succeed.

Implement Pre-Validation: Move the gas fee deduction logic or balance checks before the `CallExecuteUniversalTx` execution. Ensure that sufficient funds and valid conditions are verified prior to initiating the irreversible EVM call.

Resolution:

This issue has been addressed and resolved in commit [b890c7f](#).

F-2026-16740 - Solana RPC Signature Order Misassumption - Medium

Description:

The `processSlotRange` function incorrectly terminates prematurely and skip critical inbound events, potentially stalling cross-chain operations.

2. Detailed Explanation

The vulnerability is located in the `processSlotRange` function within `universalClient/chains/svm/event_listener.go`. The function is responsible for scanning a specific range of slots (`fromSlot` to `toSlot`) to identify and process transactions associated with a gateway address.

Technical Analysis

The implementation iterates through the results of the `GetSignaturesForAddress` RPC call using the following logic:

```
for _, sig := range signatures {
    if sig.Slot < fromSlot {
        continue
    }
    if sig.Slot > toSlot {
        break
    }
    // ... process transaction ...
}
```

The flaw lies in the use of the `break` statement when `sig.Slot > toSlot`. According to the official Solana RPC specification, the `getSignaturesForAddress` method returns signatures in **reverse chronological order** (newest first/descending slot order), but even this ordering is not guaranteed in all implementations (<https://github.com/solana-labs/solana/issues/22456>)

Because the list is sorted from highest slot to lowest slot, the first element encountered is the most recent. If the RPC returns any signature with a slot number greater than `toSlot` (which is obtained using `GetLatestSlot()` but could be outdated if new signatures arrive between calls), the `break` condition is triggered immediately. This terminates the loop and ignores all subsequent signatures in the slice, even those that fall perfectly within the `[fromSlot, toSlot]` range.

Impact

This bug creates a critical failure in the event detection pipeline. Since the system updates the `lastProcessedSlot` in the database after calling `processSlotRange`, any signatures skipped due to the premature `break` are effectively marked as processed and will never be revisited.

The consequences include:

- **Stalled Cross-Chain Operations:** Inbound events from the Solana chain to the Push Chain will be missed.
- **Loss of State Synchronicity:** The validator's internal state will diverge from the actual on-chain state.
- **Operational Failure:** User requests to move assets or trigger actions will remain undetected, leading to a complete breakdown of the bridge/gateway functionality.

Status:

Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 4/5

Severity: Medium

Recommendations

Remediation: Pre-order the signatures so the algorithm do not depends on the Solana ordering.

Resolution: This issue has been addressed and resolved in commit [35bd9b6](#).

[F-2026-16960](#) - [PUSHCHAIN-REPORTED] Issue 1: RPC failover retries same endpoint under concurrent load - Medium

Description:

Both EVM and SVM clients implement endpoint failover via `executeWithFailover` using a shared atomic counter `rc.index`. Because the counter is incremented *inside* the retry loop, concurrent callers can advance the same counter between retries. This can cause a single caller to retry the same failing endpoint and never try the alternate endpoint.

Why it matters

Under load, failover can silently degrade into “repeat the bad endpoint,” increasing the probability of request failure and cascading component failures (listener/confirmers/oracle/signer, etc.).

Evidence / permalinks (audit-main)

- EVM: https://github.com/pushchain/push-chain-node/blob/audit-main/universalClient/chains/evm/rpc_client.go#L107
- SVM: https://github.com/pushchain/push-chain-node/blob/audit-main/universalClient/chains/svm/rpc_client.go#L122

Status:

Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 3/5

Severity: Medium

Recommendations

Remediation:

Snapshot the counter once before the retry loop, then index by `startIndex + attempt` so each caller deterministically rotates through all endpoints.

```
startIndex := atomic.AddUint64(&rc.index, 1) - 1
for attempt := 0; attempt < maxAttempts; attempt++ {
    client := clients[(startIndex + uint64(attempt)) % uint64(len(clients))]
    // ... try client, break on success ...
}
```

Resolution:

This issue has been addressed and resolved in commit [4706e90](#).

[F-2026-16961](#) - [PUSHCHAIN REPORTED] Issue 2 — Fund migration failure due to hardcoded GasLimit and missing L1GasFeeBuffer - Medium

Description:

`InitiateFundMigration` records gas parameters used by UV to broadcast the migration transfer.

- **Gas price** is dynamic (from chain meta oracle).
- **Gas limit** is hardcoded to `21000` via `nativeTransferGasLimit`.
- There is **no L1 data fee buffer** concept for OP-Stack chains (Base/Optimism), where fees include L2 execution + L1 calldata posting costs.

Why it matters

`21000` is not universal across EVM networks (e.g., some L2s require higher gas for native transfer).

OP-Stack requires budgeting for L1 data fee; without it, tx may be rejected or the relayer subsidizes unexpectedly.

Evidence / permalinks (audit-main)

- Constant: https://github.com/pushchain/push-chain-node/blob/audit-main/x/utss/keeper/msg_initiate_fund_migration.go#L11
- Use site: https://github.com/pushchain/push-chain-node/blob/audit-main/x/utss/keeper/msg_initiate_fund_migration.go#L88

Status:

Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Severity: Medium

Recommendations

Remediation:

Move `gas_limit` and `l1_gas_fee_buffer` into an admin-modifiable param in `UniversalCore` contract on push chain, and include both in the migration record/event.

- **Core:** fetch gas params from universalCore for that chain in `MsgInitiateFundMigration` and emit those out in the `FundMigration` event that UV can use.
- **UV:** honor `L1GasFeeBuffer` when computing total fee allowance for OP-Stack.

Resolution:

This issue has been addressed and resolved in commit [cc7e027](#).

[F-2026-16962](#) - [PUSHCHAIN-REPORTED] Issue 3 — Fund migration broadcast retry storm after peer already migrated funds - Medium

Description: All UV nodes race to broadcast the signed fund migration tx. The first to land drains the old TSS address to zero. Others then attempt to assemble a tx by fetching current balance and fail with "insufficient balance for gas," then retry every tick.

Example log:

```
WRN failed to assemble fund migration tx, will retry next tick
error="insufficient balance for gas: balance=0 gasCost=316875657000
llGasFee=0"
chain=eip155:11155111
component=txbroadcaster
event_id=e57ab88f58e2f154c9f6fd6210b23c4d0db26bec2a9593a9ce134dbb70
b316bd
```

Evidence / permalinks (audit-main)

- Retry log site: <https://github.com/pushchain/push-chain-node/blob/audit-main/universalClient/tss/txbroadcaster/evm.go#L121>
- Function: <https://github.com/pushchain/push-chain-node/blob/audit-main/universalClient/tss/txbroadcaster/evm.go#L73>

Status: Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 2/5

Severity: Medium

Recommendations

Remediation: Use the **balance/value assumptions from signing time** (or signed payload) rather than re-fetching balance at assembly time, and/or add a "already migrated" detection path to stop retries.

Resolution: This issue has been addressed and resolved in commit [58ed01b](#).

[F-2026-16966](#) - [PUSHCHAIN-REPORTED] Issue 7 — Expiry sweeper votes REVERT based on push chain state, not observed chain state and can result into false voting - Medium

Description:

Each UV node runs an expirysweeper that:

- Finds CONFIRMED events whose expiry block has passed.
- Votes failure (`voteOutboundFailure` / `voteFundMigrationFailure`) on Push Chain.
- Marks events REVERTED locally.

Why it matters

- Reset/lagging nodes keep stale local state until timer fires; then they cast redundant votes.
- Non-signing committee nodes can vote failure even if the tx succeeded on-chain but inbound observation is temporarily broken → split votes / false failures.

Evidence / permalinks (audit-main)

- Sweeper entry: <https://github.com/pushchain/push-chain-node/blob/audit-main/universalClient/tss/expirysweeper/sweeper.go#L80>
- Outbound revert vote: <https://github.com/pushchain/push-chain-node/blob/audit-main/universalClient/tss/expirysweeper/sweeper.go#L125>
- Fund migration revert vote: <https://github.com/pushchain/push-chain-node/blob/audit-main/universalClient/tss/expirysweeper/sweeper.go#L161>

Status:

Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Severity: Medium

Recommendations

Remediation:

Remove expirysweeper from the voting path and make settlement observation-based for outbound and fund migration events.

Resolution:

This issue has been addressed and resolved in commit [65b529a](#).

[F-2026-16998](#) - Hardcoded EOA as owner of every system-contract

ProxyAdmin - Medium

Description:

Push EVM system contracts deployed from the universal registry sit behind transparent proxies whose `ProxyAdmin` contracts follow an OpenZeppelin-style `Ownable` pattern. At genesis, the owner of each `ProxyAdmin` is set from a single hardcoded externally owned account constant. That account can invoke `upgradeAndCall` on every such proxy without going through Cosmos governance. Module messages such as `MsgUpdateParams` only govern registry metadata (parameters, chain and token configuration) they do not constrain who may upgrade the EVM execution surface. The trust model therefore splits chain configuration authority from upgrade authority for cross-chain messaging contracts, with the latter resting on one EOA and normal EVM transactions.

File: `push-chain-node/x/uregistry/types/constants.go` ·

```
const PROXY_ADMIN_OWNER_ADDRESS_HEX = "0xa96CaA79eb2312DbEb0B8E93c1Ce84C98b67bF11"
```

File: `push-chain-node/x/uregistry/keeper/genesis.go` ·

```
func deployProxyAdminContract(ctx context.Context, evmKeeper types.EVMKeeper, proxyAdminContractAddress string, runtimeBytecode []byte) {
    sdkCtx := sdk.UnwrapSDKContext(ctx)
    proxyAdminAddress := common.HexToAddress(proxyAdminContractAddress)
    owner := common.HexToAddress(types.PROXY_ADMIN_OWNER_ADDRESS_HEX)
    // ...
    evmKeeper.SetState(sdkCtx, proxyAdminAddress, common.Hash{}, common.LeftPadBytes(owner.Bytes(), 32))
}
```

Storage slot zero is written with the owner address (standard `Ownable.owner` layout for the embedded proxy-admin bytecode), granting that address upgrade rights for the associated proxies.

File: `push-chain-node/x/uregistry/types/constants.go` ·

```
var SYSTEM_CONTRACTS = map[string]ContractAddresses{
    "UNIVERSAL_CORE": {
        Address: "0x0000000000000000000000000000000000000000000000000000000000000000",
        ProxyAdmin: "0xf200000000000000000000000000000000000000000000000000000000000000",
        Implementation: "0xf100000000000000000000000000000000000000000000000000000000000000",
    },
    // ... UNIVERSAL_BATCH_CALL, UNIVERSAL_GATEWAY_PC, RESERVED_0..2
}
```

File: `push-chain-node/x/uregistry/keeper/genesis.go` ·

```
func deploySystemContracts(ctx context.Context, evmKeeper types.EVMKeeper, systemContracts map[string]types.ContractAddresses) {
    // ...
}
```

```

for _, name := range names {
    contract := systemContracts[name]
    // ...
    deployProxyAdminContract(ctx, evmKeeper, contract.ProxyAdmin, bytecodes.ADMIN_RUNTIME)
    deployImplementationContract(ctx, evmKeeper, contract.Implementation, bytecodes.IMPL_RUNTIME)
    deployProxyContract(ctx, evmKeeper, contract.Address, contract.ProxyAdmin, contract.Implementation, bytecodes.PROXY_RUNTIME)
}
}

```

The same constant is referenced from `x/uexecutor` for related system-contract deployment, so upgrade authority is unified across the universal execution surface at the EVM layer.

Compromise, loss, or misuse of the owner key allows arbitrary replacement of implementations for core and reserved system contracts. An attacker could redirect vault balances, alter gateway or batch-call behavior, or pre-empt reserved proxy slots. Cosmos governance cannot revoke or rotate this authority on its own; recovery depends on the EOA performing `transferOwnership` or equivalent EVM actions. This centralization exceeds the blast radius of Cosmos-level registry admin parameters documented elsewhere, because it affects live contract code rather than configuration records alone.

Status:

Accepted

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Severity: Medium

Recommendations

Remediation:

- Initialize `ProxyAdmin` ownership with a multisig or other contract-governed account rather than a single EOA before production launch.
- Tie upgrades to an on-chain path that reflects governance decisions (for example an adapter contract callable only after a passed proposal is verified).
- Add a timelock on upgrade actions so operators and the community can react to unexpected changes.
- Publish the production ownership model, rotation procedure, and post-genesis transfer plan as launch criteria.
- Monitor that every deployed `ProxyAdmin` owner matches the intended governance-controlled set after each upgrade window.

[F-2026-17087](#) - [PUSHCHAIN-REPORTED] Issue 9 - Inbound-revert outbound ID collides when a single source tx contains multiple Inbounds - Medium

Description:

When the chain decides an inbound cannot be executed (validation failure post-quorum), it constructs a synthetic `INBOUND_REVERT` outbound to refund the user on the source chain. That outbound ID is generated by `GetOutboundRevertId`, and the same ID is later used as the subTxId on the source-chain gateway / vault calls that perform the revert.

The current implementation derives the ID from only (sourceChain, txHash, "REVERT"):

```
// x/uexecutor/types/keys.go
func GetOutboundRevertId(sourceChain string, inboundTxHash string)
string {
    data := fmt.Sprintf("%s:%s:REVERT", sourceChain, inboundTxHash)
    hash := sha256.Sum256([]byte(data))
    return hex.EncodeToString(hash[:])
}
```

If a single source-chain transaction emits multiple bridge events (different log_index values) and more than one of them needs to be reverted, every revert outbound for that tx maps to the same ID. The two inbound observations are correctly separated by log_index at every other point in the system (UTX key, ballot, pending-inbound set), but the revert ID is not.

Status:

Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Severity: Medium

Recommendations

Remediation:

Add `log_index` to the digest input so the revert ID is unique per inbound event. Concretely:

```
// x/uexecutor/types/keys.go
func GetOutboundRevertId(sourceChain, inboundTxHash, logIndex string) string {
    data := fmt.Sprintf("%s:%s:%s:REVERT", sourceChain, inboundTxHash, logIndex)
    hash := sha256.Sum256([]byte(data))
}
```

```
return hex.EncodeToString(hash[:])
}
```

And update the single caller to pass `inbound.LogIndex`:

```
// x/uexecutor/keeper/build_revert_outbound.go
```

```
Id: types.GetOutboundRevertId(inbound.SourceChain, inbound.TxHash,
inbound.LogIndex)
```

This brings the revert key in line with the inbound UTX key (GetInboundUniversalTxKey already uses `source_chain:tx_hash:log_index`) and the rescue key (GetRescueFundsOutboundId already uses `pcCaip:txHash:logIndex:RESCUE`).

Resolution:

This issue has been addressed and resolved in commit [e46574f](#).

[F-2026-18186](#) - [Dual Defense] Gasless New-Account Ante Path Skips Signature Limits and Persists Failed-Message Accounts - Medium

Description:

User-facing messages including `MsgExecutePayload`, `MsgMigrateUEA`, and validator vote messages are allowlisted as gasless, bypassing Cosmos fee deduction and minimum-fee enforcement. For a gasless transaction whose signer has no existing account, `AccountInitDecorator` verifies the signature, creates a new account at sequence `1`, writes the account through `AccountKeeper`, and returns without invoking the remainder of the ante chain.

That early return has two linked consequences on the same code path:

Skipped signature-policy enforcement. Recursive `TxSigLimit` validation and signature-verification gas metering never run. The custom verifier checks only outer `len(sigs) == len(signers)`, then calls full `VerifySignature`, including recursive multisig verification. An attacker can construct a correctly derived `LegacyAminoPubKey` multisig with thousands of duplicate child keys, copy one valid DIRECT signature into every selected slot, and force validators to perform quadratic marshal-and-verify work while outer gas and fees remain zero. SDK `ValidateSigCountDecorator` would otherwise reject pubkeys whose `CountSubKeys` exceeds `TxSigLimit` (default `7`).

Ante account writes survive failed messages. Cosmos SDK commits the ante cache before message execution. When subsequent message handling fails (for example, an unbonded signer submitting gasless `MsgVoteInbound`), the account created during ante handling remains committed. Fresh keypairs repeat the path indefinitely. Current code contains no per-origin gasless-account quota, proof-of-work requirement, or deposit bond for first-time gasless signers.

Gasless allowlisting is intentional product behavior. Skipping `TxSigLimit` and signature-gas consumption while performing full recursive verification, and committing free accounts before message authorization succeeds, are not documented security properties of that design.

Impact

- Zero-fee CPU and finality degradation via unmetered recursive signature verification on the new-account gasless path.
- Permanent accumulation of otherwise unused account records when message execution fails after successful ante initialization.
- Increased database size, snapshot and state-sync footprint, backup cost, and query overhead, rate-bounded by mempool, block-size, and transaction admission limits.
- Repeatable with fresh keys; exact replay of an identical signature set is blocked once the account exists.

- Not demonstrated as permanent consensus halt, FinalizeBlock panic, fund theft, or consensus divergence.

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 3/5

Severity: Medium

Recommendations

Remediation:

1. Never return from `AccountInitDecorator` before equivalent `ValidateSigCount` and `SigGasConsume` checks (call `next` after account creation, or inline those checks before verification completes).
2. In `verifySignatureForNewAccount`: enforce `CountSubKeys(pubKey) <= TxSigLimit` before `VerifySignature`; consume signature-verification gas for each selected child key.
3. Prefer rejecting gasless messages from nonexistent or unbonded signers before expensive verification.
4. Defer new-account creation until after successful message authorization and execution for permissionless user-facing gasless messages, or require a minimum fee, tip, deposit, or refundable bond for first-time account creation.
5. Separate validator-only gasless messages from public relay messages and apply different admission policies.
6. Add mempool quotas and per-origin rate limits for first-time gasless signers; monitor abnormal new-account creation rates.
7. Regression coverage:
 - Multisig with at most `TxSigLimit` children is accepted and metered on the new-account gasless path.
 - Multisig exceeding `TxSigLimit` is rejected before account creation completes.
 - Oversized equal-byte multisig no longer outruns a `TxSigLimit`-compliant control by orders of magnitude.
 - Failed gasless message execution does not leave a free account when policy requires post-authorization creation.

Resolution:

The issue has been addressed in commits [6a1403e](#) and [c2a8e3f](#).

[F-2026-18187](#) - [Dual Defense] Gov Precompile getTallyResult Persists SDK Tally Vote Deletions - Medium

Description:

The gov precompile's `getTallyResult` is ABI-documented and classified as a view/query via `IsTransaction`, but for proposals in the voting period it forwards to the Cosmos SDK `TallyResult` query, which invokes `Keeper.Tally`. That keeper method deletes vote records as it walks them. On ephemeral gRPC query contexts, those mutations are discarded. Through the EVM precompile, the same call runs on StateDB's cache context; a successful delivering EVM transaction commits via `writeCache()`, persisting vote removals.

Any EVM user with gas can wipe votes on a voting-period proposal and repeat as new votes appear. No proposer, voter, or deposit privileges are required. The precompile at `0x...0805` is enabled in Push testnet genesis `active_static_precompiles`.

gRPC query safety does not transfer to precompile execution: the same SDK function has different commit semantics when invoked through a delivering EVM transaction.

Impact

- Permissionless governance vote suppression and tally integrity break on a shipped precompile.
- Continuous wipe can prevent vote accumulation; voters can re-submit votes after deletion, but an attacker can race repeated calls during the voting window.
- No direct mint, custody drain, or consensus panic.
- Indirect impact on fund safety or upgrade outcomes depends on a live contentious proposal and voter non-response.
- Not a cosmetic query bug while the gov precompile remains active.

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 3/5

Severity: Medium

Recommendations

Remediation:

1. Do not call `Keeper.Tally` from the precompile. Compute a read-only tally (walk votes without `Remove`), or call Tally on a throwaway `CacheContext` that is never written via `writeCache`.
2. For voting-period proposals, return empty or partial info without invoking mutating Tally (match deposit-period empty behavior, or read stored aggregates if available).
3. Treat any precompile path that can mutate native stores as `IsTransaction == true`, or reject mutating keeper calls when `readOnly`.
4. Regression: after a delivering transaction calling `getTallyResult(id)` on a voting proposal with N votes, vote count remains N; staticcall and CALL both non-destructive.

Resolution:

The issue has been addressed in commit [3358532](#).

[F-2026-18191](#) - [Dual Defense] Byzantine TSS Coordinator Can Freeze Outbound Withdrawals via Gap Nonce - Medium

Description:

Followers accept any coordinator `req.Nonce >= finalizedNonce` without an upper bound or contiguity check, then recompute `SigningHash` with that nonce so a gap nonce is self-consistent. Fund-flow fields still come from the on-chain outbound event, so this is a freeze, not a redirect or theft. A far-future nonce never mines; resolver and broadcaster never see `NonceConsumed`, so they rewind and retry without casting failure; hub remint in `handleFailedOutbound` never runs. Fund-migration verify uses the same lower-bound-only pattern.

In `verifyOutboundSigningRequest` (`sessionmanager.go`), the implementation rejects only `req.Nonce < finalizedNonce` (when nonce lookup succeeds). The coordinator nonce is treated as the source of truth.

`GetOutboundSigningRequest(..., req.Nonce)` makes the hash check pass for any still-unused nonce, including large gaps. The same pattern exists in `verifyFundMigrationSigningRequest`.

Participants rebuild from `OutboundCreatedEvent` with the coordinator nonce. Altering fund-flow fields fails the hash match. The corruption surface is the nonce only, producing an undeliverable signed transaction rather than an attacker payout.

`txflow.CheckNonce` returns `NonceConsumed` if and only if `signedNonce < finalizedNonce`; otherwise `NonceAvailable`. The resolver, on not-found with `NonceAvailable`, rewinds to SIGNED. Broadcaster errors similarly avoid treating gap nonces as dead. `NonceConsumed` (and thus failure vote leading to hub remint) would require the chain's next nonce to surpass the gap, which is impractical for `finalized+2^32`.

Burn already happened at gateway withdraw. Remint runs from failed-outbound finalize. No failure observation means no remint.

`RevertStuckInbound` does not cover outbounds. The expiry sweeper targets CONFIRMED → REVERT patterns, not this SIGNED gap-nonce loop.

Impact

A malicious coordinator can freeze specific EVM outbounds and fund migrations when honest co-signers approve a gap nonce. PRC20 remains burned until remint; funds are not redirected to the attacker. Honest outbounds under honest coordinators continue unaffected because contiguous nonces are not impacted.

The attack requires an active validator in the TSS participant set who becomes coordinator for the epoch. It is not available to unprivileged EOAs. Only outbounds or migrations signed under the malicious

coordinator's setup are affected; other withdrawals in other epochs proceed. Automatic remind is absent in this build; upgrade or admin remind remains possible. Far-future nonces, endless rebroadcast, and coordinator identity by height are monitorable.

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Severity: Medium

Recommendations

Remediation:

1. In verify: require `req.Nonce == expectedNextNonce` (or `<= expected+ε` with strict policy), not merely `>= finalized`.
2. Reject nonces above a small absolute gap (e.g. `finalized` or `pending+1` only).
3. Resolver: if signed nonce exceeds finalized by more than K and the transaction never lands for T, vote failure or mark dead.
4. Add hub `RevertStuckOutbound` (or generalize admin revert) for PENDING outbounds with no success/failure observation.
5. Allow honest re-coordinate of stuck SIGNED events with a fresh contiguous nonce after timeout.
6. Regression: coordinator proposes `finalized+1000` → followers refuse; if somehow signed, sweeper reminds.

Resolution: The issue has been addressed in commit [49de107](#).

[F-2026-18192](#) - [Dual Defense] Inactive Joiner Can Vote on Stale Inbound Ballot and Mint Unbacked PRC20 - Medium

Description:

Inbound ballots snapshot eligible voters and voting threshold at creation time. A `PENDING_JOIN` onboarding candidate is included in that snapshot. If onboarding TSS fails and an administrator moves the candidate to `INACTIVE`, existing ballots are not updated. The inactive validator remains in the universal validator set, can stay bonded, passes `MsgVoteInbound` admission checks, and `AddVote` still accepts votes from snapshotted members who have not yet voted.

Two colluding `ACTIVE` universal validators plus a removed failed joiner casting a first vote on a stale 3-of-4 snapshot can finalize a ballot while the live set requires 3-of-3. The execute path then calls `depositPRC20Token` and mints PRC20 without a corresponding source deposit.

Exploitation requires a bonded universal validator that was admin-onboarded as `PENDING_JOIN`, two colluding `ACTIVE` validators, a ballot opened while the candidate was still join-eligible, and admin removal without invoking `RecomputeBallotQuorum`. The `RecomputeBallotQuorum` admin API implements the correct repair when invoked manually but is not hooked to status transitions. `DefaultExpiryAfterBlocks = 100_000_000` makes height-based expiry ineffective as a practical control.

Impact

- Unbacked PRC20 mint when a stale ballot finalizes under a sub-live quorum.
- Supply integrity violation: mint proceeds without a verified source deposit.
- Ballot authorization persists for voters no longer eligible under current lifecycle rules.
- Requires collusion among universal validators and an admin removal window without ballot recomputation; not permissionless mint by arbitrary outsiders.

Status:

Accepted

Classification

Impact Rate: 4/5

Likelihood Rate: 2/5

Severity: Medium

Recommendations

Remediation:

1. On any universal validator status transition out of the eligible set (`PENDING_JOIN` / `ACTIVE` → `INACTIVE` / `PENDING_LEAVE`), automatically recompute all pending ballots or invalidate votes from the departing voter.
2. In `AddVote` / `VoteOnBallot` , require the voter to be in the current `GetEligibleVoters()` set, not only the snapshot (defense in depth).
3. Align `MsgVoteInbound` admission with lifecycle eligibility rules used for ballot creation.
4. Hook removal and status-change handlers to ballot maintenance; document mandatory `RecomputeBallotQuorum` until automatic hooks ship.
5. Add regression coverage: after `PENDING_JOIN` is snapshotted then moved to `INACTIVE` , the first vote must fail or recompute must run; two `ACTIVE` validators alone must not be able to mint with an inactive helper.

[F-2026-18194](#) - [Dual Defense] Unbacked Gas PRC20 Mint on INBOUND_REVERT via applyGasRefund - Medium

Description:

User PC outbounds carry `GasFee` after a prepaid burn path. `INBOUND_REVERT` outbounds are protocol-built via `buildRevertOutbound`, which fills `GasFee`, `GasToken`, `GasPrice`, and `GasLimit` from `getOutboundTxGasAndFees` only, with no `swapAndBurnGas` or escrow. On outbound finalize, `handleSuccessfulOutbound` and `handleFailedOutbound` still call `applyGasRefund` regardless of transaction type. If `GasFee > GasFeeUsed`, `CallUniversalCoreRefundUnusedGas` → `PRC20.deposit` → `_mint` credits the revert recipient with gas PRC20 (or swapped native) that was never burned for that outbound.

`build_revert_outbound.go` sets gas fields from `GetGasFeeInfoForRevertOutbound` (UniversalCore quote with baseLimit) without a burn or escrow call. `outbound.go` runs `applyGasRefund` on success and failure paths without excluding `TxType_INBOUND_REVERT`. `applyGasRefund` mints the difference via `refundUnusedGas` (swap-first, then no-swap `deposit`) when `GasFee > GasFeeUsed`.

By contrast, `create_outbound.go` takes `GasFee` from gateway events tied to the prepaid user outbound gas path, where refund returns unused prepaid balance. The revert path never prepaid.

Impact

When `GasFee > GasFeeUsed`, finalize produces unbacked gas PRC20 (or swapped PC) on the revert path. No prior burn exists for that outbound's `GasFee`. This is unauthorized `_mint` of gas PRC20 and a supply-integrity failure.

The path requires a failed non-CEA inbound and the full revert outbound lifecycle (UV ballot, TSS/source execution, outbound finalize); it is not a zero-cost empty-account mint. Per-event mint is capped by oracle `GasFee` and reduced by reported `GasFeeUsed`. CEA failures intentionally do not create `INBOUND_REVERT`. Source principal return on revert is intended product behavior; the defect is unpaid gas refund minting.

Minted gas PRC20 must be swapped or sold into finite liquidity. Harm is synthetic inflation and unfair credit of gas tokens, not immediate emptying of source-chain vaults via this path alone.

Status:

Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 3/5

Severity: Medium

Recommendations

Remediation:

1. Skip `applyGasRefund` for `TxType_INBOUND_REVERT` (and any outbound without prepaid burn proof).
2. Alternatively, escrow or burn gas PRC20 when building revert if protocol wants user-paid revert gas, then refund unused against that escrow only.
3. Ensure outbound observations never leave `GasFeeUsed="0"` when a real fee was paid.
4. Regression: INBOUND_REVERT finalize with `GasFee > GasFeeUsed` must not call `refundUnusedGas` or increase gas PRC20 supply

Resolution: The issue has been addressed in commit [8ad6728](#).

[F-2026-18196](#) - [Dual Defense] Native SOL Misclassified as SPL Due to EVM-Only isNative Check - Medium

Description:

The SVM outbound builder decides native versus SPL with an EVM-only zero-address check. Shipped native SOL and pSOL configuration stores the Solana native marker `11111111111111111111111111111111` (SystemProgram). That string fails `isNative`, so withdrawals, reverts, and rescues are built as SPL transfers with mint equal to SystemProgram, including `CreateAssociatedTokenAccountIdempotent`, which reverts because SystemProgram is not an SPL mint.

There is no production path that uses the native SOL account layout for the configured asset address. Failed outbounds remint pSOL via `handleFailedOutbound`, so users retain Push-side balance but cannot redeem to SOL while the builder is broken. Vault SOL that backed deposits remains on Solana.

Impact

- Every native SOL withdrawal, revert, and rescue built with shipped SystemProgram configuration fails pre-broadcast.
- Vault SOL cannot leave via the native path until client or registry fix.
- Failed outbound attempts remint pSOL; per-transaction user wipe is limited but redeemability is broken.
- Scope is native SOL and pSOL asset class only; real SPL mints with valid mint pubkeys are unaffected.
- Rescue and revert builders share the same `isNative` defect; failed native-SOL deposit returns are also blocked.
- Universal Client fix recognizing SystemProgram restores native layout without Push consensus upgrade.

Status:

Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 3/5

Severity: Medium

Recommendations

Remediation:

1. Centralize `isNativeAsset(addr string) bool` recognizing at least empty string, EVM zero hex, `11111111111111111111111111111111`, and `SystemProgramID.String()`.
2. Use the helper in all three builder sites plus rescue and revert account builders.
3. Fix tests to use the shipped SystemProgram address; assert native account layout and no ATA create.
4. Align registry `token_type` with product meaning; document the canonical native marker.
5. Disable pSOL outbound in registry until UV clients are upgraded.
6. Inventory vault SOL versus pSOL supply; remind kept balances consistent

Resolution:

The issue has been addressed in commit [49f8e7c](#).

[F-2026-18825](#) - [Dual Defense] One Invalid Gateway Sibling Erases Valid Outbounds After Committed UEA Burn - Medium

Description:

`BuildOutboundsFromReceipt` accumulates `UniversalGatewayPC` outbound events into a local slice, then on any single registry/chain check failure (`GetTokenConfigByPRC20`, `IsChainOutboundEnabled`, decode error) returns `nil, err`, discarding every outbound already built in that loop.

On inbound UEA paths, `ExecutePayloadV2` commits EVM state (including PRC20 burn) via `writeCache()` before the caller attaches outbounds.

`execute_inbound_gas_and_payload.go` and `execute_inbound_funds_and_payload.go` treat `AttachOutboundsToExistingUniversalTx` failure as non-fatal: they store the string in `UniversalTx.RevertError` (write-only; no recovery consumer), record the payload `PcTx` as `SUCCESS`, and return success. Burned principal therefore has no `OutboundTx` / `PendingOutbounds` row and no shipped signer/TSS/remint lifecycle.

The poison sibling must be a genuine `UniversalGatewayPC` log in the same UEA receipt as the victim outbound, which requires the victim-signed payload to call code that invokes the gateway with a token/chain that fails node-side lookup. Direct `MsgExecutePayload` and `EVMHooks.PostTxProcessing` propagate `CreateUniversalTxFromReceiptIfOutbound` errors and do not use the swallow-`RevertError` pattern, so those surfaces roll back rather than commit-then-drop.

Impact

- all-or-nothing `return nil, err` in `BuildOutboundsFromReceipt` after partial append.
- inbound UEA handlers commit payload via `ExecutePayloadV2` then swallow attach errors into unread `RevertError` while marking payload `SUCCESS`.
- with no pending outbound, automated fail-remint / TSS signing cannot run for the dropped sibling.
- Attack shape requires victim-composed UEA multicall that includes a callee able to emit a second canonical Gateway outbound (e.g. unregistered PRC20 / disabled chain) in the same receipt.
- Not protocol vault extraction or unbacked mint; loss is the victim's own burned PRC20 for that outbound.
- Receipt / `PcTx.TxHash` may still support upgrade or admin reconstruction; shipped automated recovery is closed.

Status:

Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Severity: Medium

Recommendations

Remediation:

1. In `BuildOutboundsFromReceipt`: skip invalid siblings (log + continue) or collect valid ones and attach them; never discard a fully built valid outbound because another log fails registry checks.
2. On inbound UEA paths: if attach fails after committed burns, do not mark clean SUCCESS without pending rows—return error before outer commit, remind, or attach partial valid outbounds and quarantine invalids.
3. Better: extend the `ExecutePayloadV2` cache to include attach, or attach inside the cache before `writeCache()`.
4. Wire `RevertError` (or a dedicated status) to an automated recovery / admin path.
5. Regression: multical with one valid supported outbound + one unregistered-token gateway event must leave the valid outbound in `PendingOutbounds`.

Resolution:

The issue has been addressed in commit [9b05ec7](#).

[F-2026-18826](#) - [Dual Defense] Receipt RPC Error Treated as Not-Found Can Remint After Destination Already Paid - Medium

Description:

`TxBUILDER.VerifyBroadcastedTx` calls `eth_getTransactionReceipt` and, on **any** error, returns `found=false, err=nil`:

```
receipt, err := tb.rpcClient.GetTransactionReceipt(ctx, hash)
if err != nil {
    return false, 0, 0, 0, nil
}
```

The resolver's documented contract (`evm.go` header) is that a `VerifyBroadcastedTx` error should keep the event `BROADCASTED` and retry. Because errors are collapsed to "not found," that retry path never runs (`vErr` is always nil).

When the destination `Vault.finalizeUniversalTx` has already succeeded, the TSS nonce is `NonceConsumed`. The resolver then takes:

```
found=false + NonceConsumed → voteOutboundFailure(... "tx not executed on destination chain")
```

Outbound ballots key on `Success` (`GetOutboundBallotKey`), so failure votes accumulate on a separate ballot from success. If the failure ballot finalizes first, `handleFailedOutbound` remints PRC20 via `CallPRC20Deposit` even though the destination vault already paid amount `A` — duplicate settlement for that outbound.

An attacker cannot force honest UV receipt RPCs to fail; the path needs correlated receipt/RPC failures (or equivalent) across enough UVs to finalize the failure ballot while the dest tx already succeeded, racing any success observations. Per-outbound double credit is High-class fund inconsistency, not automatic all-TVL drain.

Impact

- any receipt RPC error → `found=false, err=nil` (contradicts resolver "error → retry" comments).
- `found=false` + `NonceConsumed` → failure vote with empty tx hash / "not executed."
- failure finalize remints via `handleFailedOutbound` → `CallPRC20Deposit`.
- success and failure use distinct ballot keys (`Success` in digest).
- Double settlement for one outbound when failure quorum wins after dest pay.
- Needs UV-side receipt RPC failure at scale / quorum; race with success voters who observe vault/gateway **events** (event_processor path).

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 2/5

Severity: Medium

Recommendations

Remediation:

1. Map `ethereum.NotFound` (and nil receipt) to `found=false, err=nil`; propagate timeouts, rate limits, and other RPC errors as `err != nil`.
2. Keep resolver retry on verification errors (already coded for `vErr`).
3. Optional: require multi-RPC agreement before `NonceConsumed` failure finalize; never fail-vote with empty hash if any healthy RPC returns a success receipt.
4. Regression: simulated receipt timeout must not emit failure vote when nonce is consumed and a later receipt shows status=1.

Resolution: The issue has been addressed in commit [f176bfa](#).

[F-2026-18827](#) - [Dual Defense] Normal EVM Outbound Resolution Uses Current TSS Address After Key Rotation - Medium

Description:

Fund-migration events persist `OldTssPubkey` and `ReadFundMigrationSigner` / migration broadcast check nonces against that event-bound old EVM address. Normal outbound `SigningData` stores only `signature`, `signing_hash`, and `nonce`—no key id, pubkey, or signer. On tx-not-found, `resolveOutboundEVM` → `outboundSigner` calls `getTSSAddress(ctx)` (mutable current TSS) and `CheckNonce(currentAddr, signedNonce)`.

After a normal KEYGEN rotation $K1 \rightarrow K2$, $K1$ and $K2$ are independent EOA nonce domains. If a $K1$ -signed outbound remains `BROADCASTED`, its hash is not visible, and $K2$'s finalized nonce has advanced past the $K1$ -signed `N`, the resolver emits `NonceConsumed` and `voteOutboundFailure` ("tx not executed") even though $K1$'s nonce `N` may still be free. `handleFailedOutbound` can remind amount `A` on Push while the $K1$ -signed destination tx remains valid and can still mine later → double settlement for that outbound.

Code comments in `txresolver/evm.go` explicitly document this asymmetry ("current TSS for outbound, OLD TSS for fund migration"). Fund migration initiation requires outbound disabled + no pending outbounds; KEYGEN does not. While outbound is disabled, the resolver skips resolution—but once outbound is enabled under $K2$ with leftover $K1$ `BROADCASTED` rows, the wrong-signer nonce check applies.

Impact

- outbound `SigningData` has no signer/key epoch; not-found path uses current TSS for `CheckNonce`.
- fund migration binds `OldTssPubkey` and checks that address.
- false `NonceConsumed` → failure vote → possible `handleFailedOutbound` remind while $K1$ tx can still execute.
- Requires TSS key rotation + unresolved old-key outbounds with outbound resolution active under the new key.
- Per-outbound double credit; success observation can still race (event path).

Status:

Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 3/5

Severity:

Medium

Recommendations**Remediation:**

1. Persist signing TSS pubkey / key id / derived `from` on outbound `SigningData` at sign time (mirror fund migration).
2. Resolver and broadcaster nonce checks must use that event-bound address, never `getTSSAddress()` for historical outbounds.
3. KEYGEN/rotation: require outbound disabled and no pending/SIGNED/BROADCASTED old-key rows (or refuse current-key flip until drained).
4. Never failure-vote `NonceConsumed` unless the nonce domain matches the recovered signer of the signed payload.
5. Regression: sign under K1 → rotate to K2 → advance K2 nonce → not-found K1 outbound must not vote failure while K1 nonce is still available.

Resolution:

The issue has been addressed in commit [bb73afb](#).

[F-2026-18829](#) - [Dual Defense] UEA_SVM Hardcodes Ed25519 Verifier
0x...00ca After Runtime Moved to 0xEC...01 - Medium

Description: `UEA_SVM` (core contracts and embedded `UEA_SVM_BYTECODE`) hardcodes:

```
address public constant VERIFIER_PRECOMPILE =  
0x00000000000000000000000000000000000000ca;
```

`_verifySignatureSVM` `staticcall` s that address for `verifyEd25519`. The shipped node registers `usigverifier` only at `0xEC0000...0001` (`precompiles/usigverifier`, `app/app.go`). Dual registration of legacy `0x...00ca` for bytecode compat was removed in the audit-fixes merge; docs/`x/uexecutor/README.md` still describe SVM verify via `0x00...00ca`.

Owner-authorized `executeUniversalTx` (and any path that is not `msg.sender == UNIVERSAL_EXECUTOR_MODULE`) therefore cannot complete Ed25519 checks against a live verifier. Embedded bytecode contains `PUSH1 0xca / STATICCALL` (not `0xEC...01`).

Node hotfix restores spend without waiting on every owner: re-registering the same usigverifier impl at `0x...00ca` (the prior dual-compat design) unblocks existing bytecode immediately. Contract recompile to `0xEC...01` + factory/migration is the durable fix. Balances remain on the UEA; EVM-origin UEAs unaffected. Inbound FUNDS/GAS can still credit accounts; broken path is signature-gated owner (and non-UEM) execution.

Impact

- `UEA_SVM` / embedded bytecode target `0x...00ca`; runtime registers only `0xEC...01`.
- owner `MsgExecutePayload` relies on that precompile (`evmFrom != UEM` → sig required).
- Residual High: SVM-owned UEA funds are unspendable via legitimate Ed25519 authorization until `0x...00ca` is restored or implementations migrate to V2.
- One-shot precompile re-bind or UEA impl upgrade recovers access; no shown third-party drain.

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 3/5

Severity:

Medium

Recommendations**Remediation:**

1. Short-term: register the usigverifier implementation again at `0x...00ca` (and keep `0xEC...01`) until all SVM UEA impls are migrated.
2. Durable: ship `UEA_SVM` with `VERIFIER_PRECOMPILE = 0xEC...01`; update factory default impl; migrate existing proxies via `UEAMigration` / UEM-sponsored path once callable.
3. Align README and address book; add an integration test that deploys embedded `UEA_SVM_BYTECODE`, funds it, and succeeds `executeUniversalTx` with a real Ed25519 owner sig on the pin's precompile set.
4. Prefer constructor/immutable verifier address over a buried constant for future moves.

Resolution:

The issue has been addressed in commit `cc0a13a` (node precompile set in `c63e0af`).

[F-2026-16039](#) - Inbound ballot identity derived from full serialized Inbound - Low

Description:

Inbound voting derives its ballot identifier from the complete protobuf encoding of the `Inbound` message, whereas the on-chain Universal Transaction identifier uses only a deterministic digest of `source_chain`, `tx_hash`, and `log_index`. Universal Validators must therefore marshal identical `Inbound` payloads for the same logical bridge event. Divergence in optional fields, decoding, amounts, or metadata yields distinct ballot identifiers, fragments votes across ballots, and can prevent quorum from forming even when operators agree off-chain.

The implementation exposes the ballot key as `hex.EncodeToString` of `Inbound.Marshal()`, not as a separate hash of those bytes; the liveness concern is unchanged: voting identity is coupled to the full serialized message

The Universal Transaction key hashes a minimal string tuple; the inbound ballot key uses the marshaled message body.

```
func GetInboundUniversalTxKey(inbound Inbound) string {
    data := fmt.Sprintf("%s:%s:%s", inbound.SourceChain, inbound.TxHash,
        inbound.LogIndex)
    hash := sha256.Sum256([]byte(data))
    return hex.EncodeToString(hash[:]) // hash[:] converts [32]byte → []byte
}

func GetInboundBallotKey(inbound Inbound) (string, error) {
    bz, err := inbound.Marshal()
    if err != nil {
        return "", err
    }
    return hex.EncodeToString(bz), nil
}
```

The inbound ballot path calls `GetInboundBallotKey` before `VoteOnBallot`. The keeper records that the ballot uses the inbound payload as observed, so differing field data produces distinct votes by construction.

```
ballotKey, err := types.GetInboundBallotKey(inbound)
if err != nil {
    return false, false, err
}
```

End users may see funds appear stalled at the gateway while explorers still show activity, because votes never consolidate onto one ballot identifier. Operators incur longer incidents spent diffing raw payloads and client versions instead of isolating business-logic faults. At the protocol level, bridge finalization depends on strict software homogeneity across the validator set, not merely on an honest majority agreeing on the same chain event.

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation: Define a minimal, versioned canonical observation identifier used only for voting, and derive the ballot key from that digest while retaining the full **Inbound** for execution after quorum. Publish a single reference encoder with conformance tests so every Universal Validator release emits identical vote keys for the same on-chain event. Document explicitly for integrators that identical transaction hashes do not imply identical inbound ballots under the current scheme.

Resolution: This issue has been addressed and resolved in commit [93de525](#).

[F-2026-16599](#) - DoS via Slowloris attack on API server at UniversalClient - Low

Description:

The function `NewServer()` at `universalClient/api/server.go` initializes an `http.Server` without specifying any timeout configurations (such as `ReadTimeout`, `WriteTimeout`, `IdleTimeout`, or `ReadHeaderTimeout`). By using the default `http.Server` settings, the server is vulnerable to Slowloris attacks and other Denial of Service (DoS) vectors where an attacker can keep connections open indefinitely by sending data very slowly or not sending the request header at all, eventually exhausting the server's connection pool and making it unresponsive to legitimate traffic.(R)

The `http.Server` struct is created with only two fields populated:

No timeout fields are set, making all timeout values default to zero.

```
func (s *Server) Start() error {
    // ...
    ln, err := net.Listen("tcp", s.server.Addr)
    // ...
    go func() {
        err := s.server.Serve(ln)
        // ...
    }()
    return nil
}
```

The server binds to `0.0.0.0:<port>` (all interfaces) and serves indefinitely without timeout protections.

The Slowloris attack is defined as:

- Attacker sends: `GET /health HTTP/1.1\r\nHost: target:8080\r\nX-slow: a\r\n`
- Attacker sends one byte every 30 seconds to keep the connection open
- Attacker send as many connections as configured by the operative system, blocking the server from opening any valid connections.

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

On Linux, the default max amount of connection is a very high number (>1000000) thus making this attack slow and unlikely from outside a local network, but still possible.

Resolution:

This issue has been addressed and resolved in commit [6f961a6](#).

[F-2026-16632](#) - Outbound ballot identity is coupled to full OutboundObservation encoding - Low

Description:

The outbound voting key is derived from `utxId`, `outboundIndex`, and the full serialized `OutboundObservation`. This design binds voting identity to complete observation encoding rather than to a narrowly defined canonical representation of the underlying outbound fact.

As a result, validators can reach the same business conclusion while still deriving different ballot keys if their observation construction differs in non-essential fields. In that condition, votes fragment across multiple ballot identities and quorum formation may be delayed or prevented.

This failure mode is materially aligned with [F-2026-16039](#) on inbound observation identity, with the same core risk expressed in the outbound voting path.

The key derivation routine serializes the entire observation payload and includes it in the hash preimage used for ballot identity.

```
func GetOutboundBallotKey(
    utxId string,
    outboundIndex string,
    observedTx OutboundObservation,
) (string, error) {

    bz, err := observedTx.Marshal()
    if err != nil {
        return "", err
    }

    data := append([]byte(utxId+"-"+outboundIndex+":"), bz...)
    hash := sha256.Sum256(data)

    return hex.EncodeToString(hash[:]), nil
}
```

Because ballot identity depends on full observation materialization, any implementation variance in observation composition can alter the resulting key even when validators are observing the same outbound event.

Status:

Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

Adopt a canonical voting digest that includes only consensus-critical outbound attributes and use this digest exclusively for ballot key derivation. Persist the full `OutboundObservation` only as supporting evidence after quorum is finalized.

To reduce recurrence risk:

- Publish a single normative specification for outbound observation construction.
- Maintain conformance vectors and deterministic test fixtures across validator implementations.

Resolution:

This issue has been addressed and resolved in commit [93de525](#).

[F-2026-16642](#) - Pending inbound index can outlive terminal ballot state - Low

Description:

`VoteInbound` adds an inbound to `PendingInbounds` before recording a ballot vote. Cleanup of `PendingInbounds` is tied to successful post-finalization paths that create a `UniversalTx`. If a ballot reaches a terminal non-pending status (for example, `EXPIRED`) without traversing those cleanup paths, the pending entry can remain indefinitely. At that point, further voting is rejected because ballot voting only accepts `PENDING` ballots.

Result: the ballot is terminal, but the inbound may still appear pending in executor state.

The inbound is inserted into `PendingInbounds` before ballot resolution and that cached state is then committed. Cleanup is performed later on post-finalization execution paths, and `RemovePendingInbound` is called from this file only. If a terminal state is reached outside that cleanup path, the pending state can remain while the ballot is already terminal.

The lock condition is explicit in the ballot keeper:

File: `x/uexecutor/keeper/msg_vote_inbound.go`

```
if ballot.Status != types.BallotStatus_BALLOT_STATUS_PENDING {
    k.Logger().Warn("ballot is not in pending state, cannot vote",
        "ballot_id", id,
        "ballot_status", ballot.Status.String(),
        "voter", voter,
    )
    return ballot, false, false, fmt.Errorf("ballot %s is already %s",
        id, ballot.Status.String())
}
```

The keying model also differs by design: pending index entries use `GetInboundUniversalTxKey(inbound)` (hash of `source_chain:tx_hash:log_index`), while ballots use `GetInboundBallotKey(inbound)` (marshaled full inbound payload), as defined in `x/uexecutor/types/keys.go`. This is valid behavior, but it makes recovery more complex once those two state views diverge.

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

Add explicit reconciliation hooks so that any inbound ballot transition to a terminal state that does not create a `UniversalTx` clear `PendingInbounds` entry. Define and document canonical inbound lifecycle transitions (`PENDING` → `PASSED` / `REJECTED` / `EXPIRED`) with required secondary-index side effects, ship expiry tuning, and reconciliation in the same release train, and add integration tests that assert terminal-ballot and pending-index consistency invariants. Until automated reconciliation is in place, document an interim operational recovery runbook.

Resolution:

This issue has been addressed and resolved in commit [cbb9cf48](#).

[F-2026-16658](#) - Threshold signing ballots require all process participants - Low

Description:

For `VoteOnTssBallot`, `votesNeeded` equals the number of participants in the active threshold signing process, requiring unanimous participation from the signing cohort. In contrast, inbound and outbound bridge-style ballots require a two-thirds majority of the eligible Universal Validator set. As a result, a single unavailable or faulty participant in the signing cohort can block progress, introducing stricter liveness requirements compared to chain-wide bridge voting.

File: `x/utss/keeper/voting.go` ·

```
// votesNeeded = number of participants in the tss process
// 100% quorum needed
votesNeeded := int64(len(existing.Participants))
if votesNeeded <= 0 {
    return false, false, fmt.Errorf("no participants in process %d", processId)
}
// ...
_, isFinalized, isNew, err = k.ValidatorKeeper.VoteOnBallot(
    ctx,
    ballotKey,
    validatorTypes.BallotObservationType_BALLOT_OBSERVATION_TYPE_TSS_KEY,
    universalValidator.String(),
    validatorTypes.VoteResult_VOTE_RESULT_SUCCESS,
    existing.Participants,
    int64(votesNeeded),
    expiryAfterBlocks,
)
```

Operations teams must maintain high availability for every participant in an active signing process. Protocol liveness for key rotation or generation can stall on one faulty node.

Status:

Accepted

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

After `CurrentTssProcess.ExpiryHeight`, `VoteOnTssBallot` returns an error and accepts no further votes; ballot expiry is driven by `VoteOnBallot` /

`CreateBallot` in `x/uvalidator` using the remaining blocks until that height. Recovery is starting a new process through `MsgInitiateTssKeyProcess`, which the module gates on `Params.Admin` (see `x/utss/keeper/msg_server.go`), not an arbitrary governance vote unless admin is configured that way.

Where the off-chain ceremony is configured as t-of-n (signing succeeds with t cooperating participants among n), consider setting the on-chain ballot threshold to t instead of n (`len(existing.Participants)`), so finalization aligns with the MPC layer's liveness assumptions rather than unanimity. t must be represented on-chain for the active process (stored at initiation or derived in one consistent place), `VoteOnTssBallot` must use it for `votesNeeded`, and a cryptographic review must confirm that threshold is safe for attesting the pubkey/key-id and that eligible voters still match the ceremony.

[F-2026-16659](#) - Event-driven outbound observations set success unconditionally - Low

Description:

`buildOutboundObservation` always sets `Success` to true and leaves `ErrorMsg` empty when constructing an `OutboundObservation` from event-driven processing. If a failed destination transaction were still processed through this path, validators could vote success incorrectly unless every deployment routes failures exclusively through alternate builders that set `Success` to false.

File: `push-chain-node/universalClient/chains/common/event_processor.go` .

```
observation := &uexecutortypes.OutboundObservation{
    Success: true,
    BlockHeight: event.BlockHeight,
    TxHash: txHashHex,
    ErrorMsg: "",
    GasFeeUsed: gasFeeUsed,
}
```

Incorrect finalization risk arises if failure events ever reach this builder. Users could observe on-chain state that disagrees with destination-chain reality. Operators bear reputational and incident-handling cost when votes disagree with receipts.

Status:

Accepted

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

Derive `Success` and `ErrorMsg` from receipt status, structured event fields, or verified remote procedure call data rather than constants. Guard `processOutboundEvent` so ambiguous events do not vote until success is provable. Add regression tests for reverted destination transactions and map every code path that emits `MsgVoteOutbound` for mutual exclusivity between success and failure handling.

F-2026-16868 - Fixed gas limit and fee for all vote transactions -

Low

Description:

All vote operations inbound, outbound, chain metadata, TSS key process, and fund migration are submitted through the shared `vote` function, which supplies a single hard-coded gas limit, a single fee string, and fixed timeouts to `signAndBroadcastAuthZTx`. The configuration schema does not expose parameters to adjust gas, fee amount, fee denomination, or confirmation timing per deployment. If the chain's minimum fee, native denomination, or required execution gas diverges from these constants, every Universal Validator process built from this code path can fail to include or confirm vote transactions under the same conditions, which concentrates liveness and quorum risk at the application layer rather than in operator-controlled settings.

Package-level constants define gas, fee, and timing for the vote path.

`/universalClient/pushsigner/vote.go`

```
const (  
    defaultGasLimit = uint64(5000000000)  
    defaultFeeAmount = "5000000000000000upc"  
    defaultVoteTimeout = 30 * time.Second  
    txPollInterval = 500 * time.Millisecond  
    txConfirmTimeout = 15 * time.Second  
)
```

The `vote` function parses `defaultFeeAmount`, applies `defaultGasLimit` to the AuthZ transaction, and enforces `defaultVoteTimeout` for the broadcast and confirmation path; the specialized helpers `voteInbound`, `voteChainMeta`, `voteOutbound`, `voteFundMigration`, and `voteTssKeyProcess` in the same file all delegate to this implementation.

A sustained mismatch between the hard-coded fee or gas and live chain parameters can produce systematic transaction rejection or non-confirmation, which impairs voting participation for the role. Constants that substantially exceed minima can also increase fee expenditure without configuration to tune cost. Remediation for a given network currently requires a code change and release rather than configuration or runtime adjustment.

Status:

Accepted

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity:

Low

Recommendations**Remediation:**

The product should allow operators to set vote gas, fee coins (amount and denomination), and relevant timeouts in `pushuv_config.json` (or equivalent), with validation against node-reported minimum gas prices and denoms where applicable. Optional integration with transaction simulation or dynamic estimation can further reduce the risk of under- or over-payment across environments.

[F-2026-16874](#) - Coordinator retains prior validator set when refresh RPC fails - Low

Description:

The coordinator caches the universal validator set in `allValidators`. After a successful query the cache is replaced. After a failed query the function returns without modifying the cache, so the previous slice remains in memory. Coordinator selection (`IsPeerCoordinator` maps peer ID through the cached roster and compares with `coordinatorAddressForBlock`), peer and party ID resolution, and eligible-participant selection all read this cache. During membership or coordinator-period transitions on chain, repeated gRPC failures can leave the process using an outdated roster until a later refresh succeeds (including through the periodic ticker in `pollLoop`).

The refresh routine replaces the cache only after a successful RPC response; there is no invalidation branch on error.

`/universalClient/tss/coordinator/coordinator.go`

```
// updateValidators fetches and caches all validators.
func (c *Coordinator) updateValidators(ctx context.Context) {
    allValidators, err := c.pushCore.GetAllUniversalValidators(ctx)
    if err != nil {
        c.logger.Warn().Err(err).Msg("failed to update validators cache")
        return
    }

    c.mu.Lock()
    c.allValidators = allValidators
    c.mu.Unlock()

    c.logger.Debug().Int("count", len(allValidators)).Msg("updated validators cache")
}
```

The poll loop invokes `updateValidators` at startup and on each tick before event processing; if refresh fails on a tick, the last successful snapshot remains.

`/universalClient/tss/coordinator/coordinator.go`

```
// pollLoop polls the database for pending events and processes the m.
func (c *Coordinator) pollLoop(ctx context.Context) {
    ticker := time.NewTicker(c.pollInterval)
    defer ticker.Stop()

    // Update validators immediately on start
    c.updateValidators(ctx)

    for {
        select {
        case <-ctx.Done():
            return
        case <-c.stopCh:
            return
        case <-ticker.C:
            // Update validators at each polling interval
        }
    }
}
```

```
c.updateValidators(ctx)
if err := c.processConfirmedEvents(ctx); err != nil {
```

Downstream logic reads `c.allValidators` under read locks; for example `IsPeerCoordinator` resolves `peerID` to `CoreValidatorAddress` from the cache and tests coordinator assignment for the current block.

Stale membership can cause incorrect coordinator identification for incoming setup messages, incorrect participant sets for new threshold rounds, or routing inconsistency relative to on-chain state until the cache reconciles. Residual risk depends on outage duration relative to validator set changes and on reliance on the next successful poll for recovery.

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation: Invalidate or clear the cache after a failed refresh (with defined behavior when the cache is empty), enforce a maximum staleness interval after which coordinator-driven actions halt, or combine periodic refresh with backoff and alerting on repeated query failure.

Resolution: This issue has been addressed and resolved in commit [7d748bdf](#).

[F-2026-16875](#) - Event confirmers advance confirmation depth without execution success - Low

Description:

On EVM chains, after a receipt is loaded, confirmation logic compares block depth to the required confirmation threshold it does not require `receipt.Status` to indicate success. On Solana, after a transaction is loaded, confirmation compares slot depth only it does not require `Meta.Err` to be absent. Events that reached the pending queue with a stored block height or slot may therefore be marked `CONFIRMED` even when the underlying transaction failed execution, and downstream voting proceeds from that record.

```
/universalClient/chains/evm/event_confirmer.go
```

```
requiredConfirmations := ec.getRequiredConfirmations(event.ConfirmationType)
confirmations := latestBlock - receipt.BlockNumber.Uint64() + 1

if confirmations >= requiredConfirmations {
```

There is no branch on `receipt.Status` before the status transition.

```
/universalClient/chains/svm/event_confirmer.go
```

```
if confirmations >= requiredConfirmations {
    rowsAffected, err := ec.chainStore.UpdateEventStatus(event.EventID,
        store.StatusPending, store.StatusConfirmed)
```

There is no check of `tx.Meta.Err` prior to confirmation.

Confirmed and subsequently voted paths may include failed destination executions unless upstream indexing excludes those events from the store.

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

Require a successful execution outcome before promoting to **CONFIRMED** , or introduce an explicit failure path for reverted or failed transactions.

Resolution:

This issue has been addressed and resolved in commit [ed82cbeb](#).

[F-2026-16876](#) - Observers bind to registry-supplied gateway addresses without on-chain contract integrity checks - Low

Description:

The EVM client uses `GatewayAddress` from that configuration, resolves the vault through `VAULT()` at that address, and subscribes to events using registry-supplied interface metadata. The Universal Client does not compare deployed bytecode, implementation addresses for proxies, EIP-1967 implementation slots, or code hashes to approved release artifacts.

Reliance on observing the intended gateway deployment therefore assumes correct registry state and correct RPC behavior at initialization. A defective or malicious registry change, or inappropriate reliance on an RPC endpoint during bootstrap, can direct deployments toward an unintended contract surface without an independent client-side integrity check. On Solana, the gateway program identifier is likewise taken from configuration or registry state without program-hash verification in this client.

The vault address is derived only from calls to the configured gateway address.

```
/universalClient/chains/evm/client.go
```

```
// Fetch vault address from gateway contract - required for event listening and tx building
fetchCtx, fetchCancel := context.WithTimeout(c.ctx, 15*time.Second)
vaultAddr, err := FetchVaultAddress(fetchCtx, c.rpcClient, ethcommon.HexToAddress(c.registryConfig.GatewayAddress))
fetchCancel()
if err != nil {
    return fmt.Errorf("failed to fetch vault address from gateway: %w", err)
}
c.logger.Info().Str("vault_address", vaultAddr.Hex()).Msg("vault address fetched from gateway")
```

No subsequent step validates deployed code against an expected hash or implementation reference.

Status:

Accepted

Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

If registry or bootstrap RPC context is wrong, validators may collectively observe and vote with respect to an unintended contract until the configuration is corrected, concentrating exposure on a single configuration path.

[F-2026-16939](#) - Unbounded frame allocation in TSS libp2p

readFramed - Low

Description:

The TSS networking layer uses a length-prefixed framing scheme on libp2p streams. The receiver reads a 32-bit length field and immediately allocates a byte slice of that size prior to reading the payload. No upper bound is enforced on the length value. A remote peer can therefore cause large, attacker-chosen heap allocations (up to approximately 4 GiB per frame, the maximum value of `uint32`) on each processed stream. This constitutes a resource-exhaustion vector against node availability. Only principals who can establish libp2p streams to the TSS listener can trigger the behavior, and deployment-specific firewalling may further restrict reachability.

`/universalClient/tss/networking/libp2p/network.go` — (`readFramed`)

```
func readFramed(r io.Reader) ([]byte, error) {
    br := bufio.NewReader(r)
    var length uint32
    if err := binary.Read(br, binary.BigEndian, &length); err != nil {
        return nil, err
    }
    buf := make([]byte, length)
    if _, err := io.ReadFull(br, buf); err != nil {
        return nil, err
    }
    return buf, nil
}
```

`/universalClient/tss/networking/libp2p/network.go` — (`handleStream`)

```
func (n *Network) handleStream(stream network.Stream) {
    defer stream.Close()

    if deadline := time.Now().Add(n.cfg.IOTimeout); true {
        _ = stream.SetReadDeadline(deadline)
    }

    data, err := readFramed(stream)
    if err != nil {
        n.logger.Warn().Err(err).Msg("read failed")
        return
    }

    n.handlerMu.RLock()
    handler := n.handler
    n.handlerMu.RUnlock()
    if handler == nil {
        return
    }

    // Call handler in a goroutine to avoid blocking
    go handler(stream.Conn().RemotePeer().String(), data)
}
```

The libp2p host is constructed without an application-level `ConnectionGater` or analogous inbound allowlist in this module; reachability to the listener is

therefore primarily determined by network deployment rather than an explicit peer whitelist at stream open time.

`readFramed` interprets the first four octets of an inbound stream as a big-endian `uint32` frame length `L`, executes `make([]byte, L)`, and then reads exactly `L` bytes into that buffer. There is no validation that `L` is within an expected maximum for legitimate DKLS/TSS messages. The `IOTimeout` configuration bounds wall-clock read duration but does not cap allocation size: once the length prefix is consumed, the allocation has already occurred.

Status: Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

1. Define a **maximum frame size** consistent with the largest legitimate TSS/DKLS message for this deployment.
2. **Reject** inbound lengths exceeding that maximum **before** allocation (return error, close stream, log at appropriate severity).
3. Optionally use `io.LimitReader` or equivalent so reads cannot exceed the cap even if the peer lies about length (combined with rejecting oversize length prefixes).
4. Apply the **same cap** to `writeFramed` for symmetry and defense in depth.

Resolution: This issue has been addressed and resolved in commit [9524142b](#).

[F-2026-16940](#) - Non-idempotent ordering between failure vote submission and local persistence in the expiry sweeper - Low

Description:

The expiry sweeper processes expired, confirmed signing events by submitting a chain-side failure observation, then updating the local SQLite store to mark the event reverted. The implementation performs the on-chain vote before the `eventStore.Update` that records `StatusReverted` (and `vote_tx_hash` when applicable).

If the vote transaction is accepted by the network but the subsequent `eventStore.Update` operation fails, the local record does not reflect completion. The event therefore remains in a state returned by `GetExpiredConfirmedEvents` on subsequent sweeps. The sweep loop records the error and continues with the next event, which allows the same logical operation to be attempted again for the same `EventID`—a second failure vote or repeated observation unless the protocol module treats duplicate observations as strictly idempotent.

Outbound path

`voteOutboundFailureAndMarkReverted` invokes `pushSigner.VoteOutbound` prior to `eventStore.Update`:

```
// voteOutboundFailureAndMarkReverted submits a failure vote for an
// outbound event and marks it REVERTED.
func (s *Sweeper) voteOutboundFailureAndMarkReverted(ctx context.Co
ntext, event *store.Event, errorMsg string) error {
    var data uexecutortypes.OutboundCreatedEvent
    if err := json.Unmarshal(event.EventData, &data); err != nil {
        return fmt.Errorf("failed to parse outbound event data for event %s
        : %w", event.EventID, err)
    }

    fields := map[string]any{"status": store.StatusReverted}

    if s.pushSigner == nil {
        s.logger.Warn().Str("event_id", event.EventID).Msg("pushSigner not
        configured, skipping failure vote")
    } else {
        observation := &uexecutortypes.OutboundObservation{
            Success: false,
            TxHash: "",
            ErrorMsg: errorMsg,
            GasFeeUsed: "0",
        }
        voteTxHash, err := s.pushSigner.VoteOutbound(ctx, data.TxID, data.U
        niversalTxID, observation)
        if err != nil {
            return fmt.Errorf("failed to vote failure for event %s: %w", event.
            EventID, err)
        }
        fields["vote_tx_hash"] = voteTxHash
    }

    if err := s.eventStore.Update(event.EventID, fields); err != nil {
        return fmt.Errorf("failed to mark event %s as reverted: %w", event.
        EventID, err)
    }
    s.logger.Info().
        Str("event_id", event.EventID).
        Str("tx_id", data.TxID).
```

```

    Str("error_msg", errorMsg).
    Msg("voted outbound failure and marked REVERTED")
    return nil
}

```

Fund migration path

`voteFundMigrationFailureAndMarkReverted` applies the same ordering:

```

// voteFundMigrationFailureAndMarkReverted submits a failure vote f
// or a fund migration event and marks it REVERTED.
func (s *Sweeper) voteFundMigrationFailureAndMarkReverted(ctx conte
xt.Context, event *store.Event, errorMsg string) error {
    var data utstypes.FundMigrationInitiatedEventData
    if err := json.Unmarshal(event.EventData, &data); err != nil {
        return fmt.Errorf("failed to parse fund migration event data for ev
ent %s: %w", event.EventID, err)
    }

    fields := map[string]any{"status": store.StatusReverted}

    if s.pushSigner == nil {
        s.logger.Warn().Str("event_id", event.EventID).Msg("pushSigner not
configured, skipping failure vote")
    } else {
        voteTxHash, err := s.pushSigner.VoteFundMigration(ctx, data.Migrati
onID, "", false)
        if err != nil {
            return fmt.Errorf("failed to vote fund migration failure for event
%s: %w", event.EventID, err)
        }
        fields["vote_tx_hash"] = voteTxHash
    }

    if err := s.eventStore.Update(event.EventID, fields); err != nil {
        return fmt.Errorf("failed to mark event %s as reverted: %w", event.
EventID, err)
    }
    s.logger.Info().
        Str("event_id", event.EventID).
        Uint64("migration_id", data.MigrationID).
        Str("error_msg", errorMsg).
        Msg("voted fund migration failure and marked REVERTED")
    return nil
}

```

Control flow on persistence failure

Per-event failures in the sweep iteration do not halt the sweep; processing advances via `continue`, which permits re-selection of the same event until local state converges:

```

for _, event := range events {
    if event.Type == store.EventTypeSignOutbound {
        if err := s.voteOutboundFailureAndMarkReverted(ctx, &event, "event
expired before TSS could start"); err != nil {
            s.logger.Error().Err(err).Str("event_id", event.EventID).Msg("faile
d to sweep expired SIGN_OUTBOUND event")
            continue
        }
    } else if event.Type == store.EventTypeSignFundMigrate {
        if err := s.voteFundMigrationFailureAndMarkReverted(ctx, &event, "e
vent expired before TSS could start"); err != nil {
            s.logger.Error().Err(err).Str("event_id", event.EventID).Msg("faile
d to sweep expired SIGN_FUND_MIGRATE event")
            continue
        }
    } else {
        if err := s.eventStore.Update(event.EventID, map[string]any{"status
": store.StatusReverted}); err != nil {
            s.logger.Error().Err(err).Str("event_id", event.EventID).Msg("faile
d to revert expired key event")
            continue
        }
    }
}

```

```
}  
}  
swept++  
}
```

Where the client does not suppress duplicate submissions before broadcast, repeated failure votes incur additional transaction fees and consume account sequence. The substantive effect on ledger state and operator workload depends on how `x/uexecutor` and `x/utss` treat duplicate failure observations: they may be treated as idempotent no-ops, rejected at execution with recurring errors, or reflected in higher log volume. Any consequence for validator or operator standing is defined by those modules and should be validated against the live protocol rules.

Status:

Accepted

Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

The ordering hazard is addressed when local persistence and chain submission share a single, replay-safe lifecycle: failure processing remains eligible for resweep only until durable evidence exists that the vote will not be duplicated (for example, an intermediate status, persisted `vote_tx_hash`, or vote intent recorded atomically with exclusion from the expired-confirmed query set). Retry policy after broadcast ought to gate on inclusion or on-chain reconciliation rather than immediate resubmission. A preparatory local flag—where permitted by module semantics—may separate “vote pending” from `StatusReverted` until both submission and storage succeed; recovery logic must then treat an accepted vote as terminal for that event regardless of subsequent database errors.

[F-2026-16944](#) - Event Skipping via Pagination Failure in Solana Event Listener - Low

Description:

The `processSlotRange` function in the SVM event listener fails to implement pagination when fetching transaction signatures, leading to permanent data loss and missed cross-chain events during periods of high network activity.

The function located in `./push-chain-node-clean/universalClient/chains/svm/event_listener.go` is responsible for discovering and processing inbound events from the Solana gateway address within a specific slot range.

The implementation relies on the `GetSignaturesForAddress` method, which wraps the Solana RPC call `getSignaturesForAddress`. Per the Solana RPC specification, this endpoint is paginated and returns a maximum of 1,000 signatures per request. To retrieve the full history of transactions, a client must iteratively call the endpoint using the `before` parameter, passing the signature of the oldest transaction from the previous batch.

The current implementation performs a single call to `GetSignaturesForAddress` without any pagination logic:

```
signatures, err := el.rpcClient.GetSignaturesForAddress(ctx, gatewayAddr)
// ... filtering logic follows ...
```

Because the code does not loop to fetch subsequent pages, it only retrieves the 1,000 most recent signatures. If more than 1,000 transactions are sent to the gateway address between polling intervals, any transactions exceeding this limit, even those falling within the target `fromSlot` and `toSlot` range, will be ignored.

Status:

Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

Update the `processSlotRange` function to implement a pagination loop.

Note: Implement reasonable limits in the number of events to prevent DoS via unbounded event processing.

Resolution:

This issue has been addressed and resolved in commit [35bd9b67](#).

[F-2026-16964](#) - [PUSHCHAIN REPORTED] Issue 5 — SVM TX resolver lacks a way to differentiate invalid signature vs transient simulation failure - Low

Description:

Description

Currently tx may be marked reverted if CPI reverts, but this can be transient (market conditions, account state changes). Without a signature validity check, false reverts can lead to replay/double-spend style issues.

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

Add `deadline` to the TSS-signed SVM payload and enforce `Clock::unix_timestamp <= deadline` in gateway TSS verification alongside existing signature and `ExecutedSubTx` replay checks. Keep PDA-based execution detection for the multi-relayer race. Align Push outbound failure/revert voting with the deadline window. Coordinate the SVM program, UV client, and TSS rollout document deadline source and duration.

Resolution:

This issue has been addressed and resolved in commit [fcb0f786](#).

[F-2026-16965](#) - [PUSHCHAIN REPORTED] Issue 6 — Reverted txs can get stuck due to architecture (failure visibility limited to signer set) - Low

Description:

Successful outbounds are observed by every UV and voted accordingly. Failed txs (revert/other failure) are primarily visible to the signing set (2/3). If a signing node crashes, it might never mark the tx as reverted, leaving funds stuck as pending.

There is also an expiry sweeper that marks tx reverted after N time; but that can cause multiple signatures, double spend risks, and operational confusion.

Status:

Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 1/5

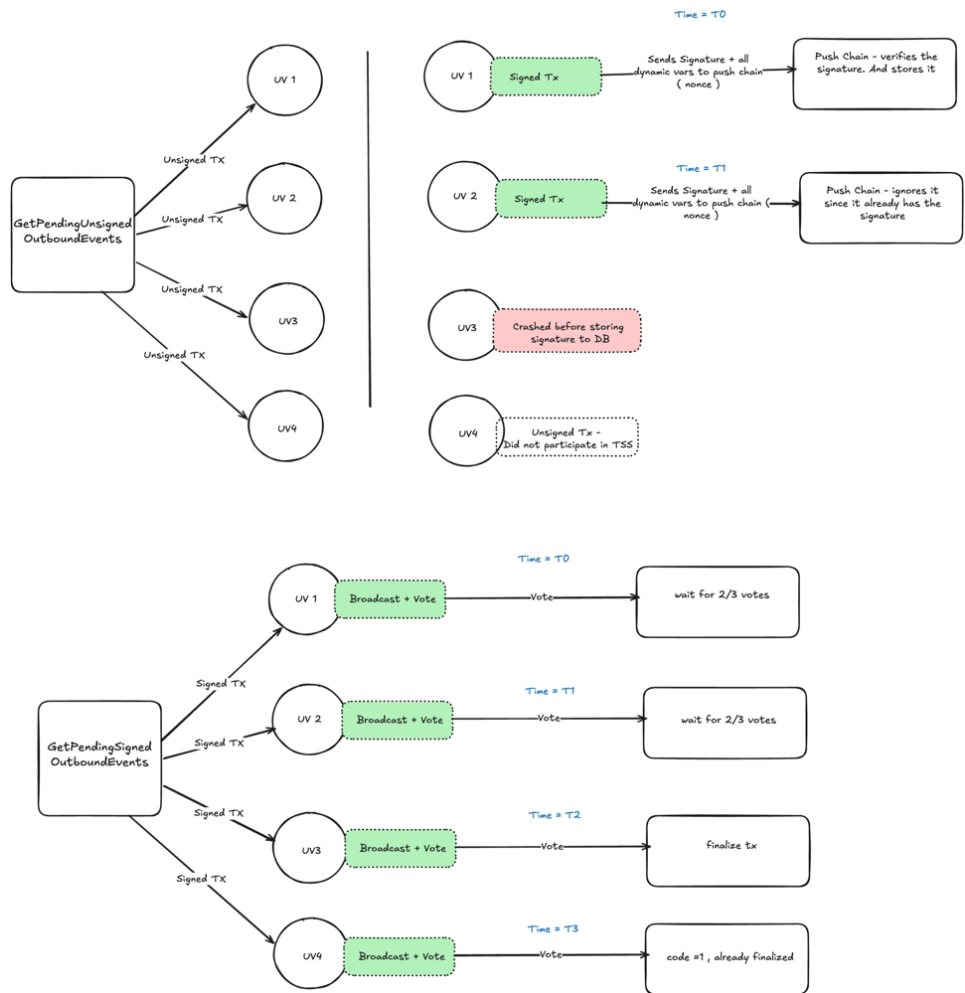
Severity: Low

Recommendations

Remediation:

Draft approach (TBD)

- All the signing participants should report signature which should be verified by push chain
- Later all the UVs can try to broadcast and process these tx and vote accordingly.



Resolution:

This issue has been addressed and resolved in commits [83a05289](#) and [1fa5a610](#).

83a05289 — After TSS signing, signatures are broadcast to all UVs via `signature_broadcast`; each node verifies and persists locally so non-signers can also broadcast and resolve reverts. Coordinator setup/ACK paths accept existing signatures for crash recovery; signing deadlines and EVM/SVM revert/broadcaster handling were tightened.

1fa5a610 — Hardens crash recovery: `RecoverInProgressEvents()` promotes IN_PROGRESS rows with persisted `signing_data` to SIGNED on startup, session expiry restores SIGNED when a signature exists, and Solana tx resolution is fixed so all UVs can settle in-progress outbounds.

[F-2026-16967](#) - [PUSHCHAIN-REPORTED] Issue 8 -

MsgExecutePayload rejects when precomputed UEA is pre-funded but not deployed - Low

Description:

Background -current deployment model

UEA addresses on Push Chain are deterministic ([Ref](#)) (CREATE2 salt = keccak256(chainNamespace, chainId, owner)), so they can be precomputed before deployment. A UEA can only be deployed through the inbound flow:

1. User on external chain calls UniversalGateway.sendUniversalTx(...) (any of the entry methods at UniversalGateway.sol:306,314,341).
2. UVs observe the UniversalTx event, vote `MsgVoteInbound`.
3. After 2/3+ quorum, x/uexecutor finalizes the inbound and deploys the UEA as part of inbound execution if the UEA isn't deployed yet.
4. From that point on, the user can submit gasless MsgExecutePayload with an EIP-712-signed UniversalPayload. Anyone (relayer, paymaster) can submit the message; the UEA contract verifies the signature against universalAccountId.Owner.

There is no user-callable MsgDeployUEA. The inbound is the only bootstrap path, and MsgExecutePayload rejected with "UEA is not deployed" if the UEA didn't exist yet.

Description

The proposed change lets a user deploy their UEA and use the gasless MsgExecutePayload route even if someone tries to sabotage them by pre-sending funds to the deterministic UEA address. Without it, a third party can dust-transfer to a user's precomputed UEA address, and a balance-based SDK that decides between "deploy via inbound" vs "execute via gasless MsgExecutePayload" by reading the on-chain balance routes the user into the gasless path — where the handler rejects with "UEA is not deployed". The user can still use their UEA, but only by performing a fresh inbound (which costs source-chain gas and bridging fees) — asymmetric griefing on the user's first interaction.

The auto-deploy is gated on the precomputed UEA address holding a non-zero native balance. This gate is also what prevents the gasless handler from being a DDoS vector: an unfunded UEA cannot successfully execute a payload anyway (gas-fee deduction at the end of MsgExecutePayload would fail and the tx would roll back), so deploying-on-demand for unfunded addresses would just be wasted work. By requiring pre-funding,

we filter out the class of requests that are doomed to fail and only auto-deploy for addresses that can actually pay for their own execution.

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

Allow MsgExecutePayload to auto-deploy the UEA inline when the precomputed UEA address holds a non-zero native balance. Concretely, in msg_execute_payload.go:

```
if !isDeployed {
    // only auto-deploy if the precomputed UEA address has been funded
    ueaAccAddr := sdk.AccAddress(ueaAddr.Bytes())
    balance := k.bankKeeper.GetBalance(sdkCtx, ueaAccAddr, pchaintypes.
        BaseDenom)
    if balance.Amount.Sign() == 0 {
        return fmt.Errorf("UEA is not deployed")
    }
    if _, err := k.DeployUEAV2(ctx, evmFrom, universalAccountId); err !=
        nil {
        return errors.Wrapf(err, "failed to auto-deploy pre-funded UEA")
    }
}
```

Resolution:

This issue has been addressed and resolved in commit [1876435](#).

F-2026-16991 - Stale active universal validators inflate ballot quorum and can deadlock finalization - Low

Description:

Eligible voters for cross-chain ballots are taken from the universal validator set by lifecycle status only entries in `UV_STATUS_ACTIVE` or `UV_STATUS_PENDING_JOIN` are counted. The enumerator does not require that the corresponding Cosmos staking validator still be bonded or that slashing has not tombstoned the operator. The module is not registered as a hook on `StakingKeeper`, so jailing, unbonding, slashing, and tombstone events on the base chain do not automatically clear or downgrade universal-validator rows. Administrators must remove or update stranded entries explicitly.

Ballot creation in the executor derives the two-thirds threshold from the length of that eligible list and passes the threshold into `uvalidator` where it is stored on the ballot. Vote admission in the executor correctly rejects signers who are not bonded or who are tombstoned, but the frozen denominator still counts those addresses. When enough universal validators remain `ACTIVE` on paper while no longer being able to vote in practice, quorum becomes impossible. Ballots use a very large default expiry horizon, so stuck ballots do not age out on a human timescale, and thresholds on already-created ballots are not recomputed when the set is repaired later.

File: `push-chain-node/x/uvalidator/keeper/validator.go` .

```
// GetEligibleVoters returns all validators that are eligible to vote on external transactions.
// Eligibility: validators with status ACTIVE or PENDING_JOIN.
func (k Keeper) GetEligibleVoters(ctx context.Context) ([]types.UniversalValidator, error) {
    var voters []types.UniversalValidator

    err := k.UniversalValidatorSet.Walk(ctx, nil, func(addr sdk.ValAddress, val types.UniversalValidator) (stop bool, err error) {
        switch val.LifecycleInfo.CurrentStatus {
        case types.UVStatus_UV_STATUS_ACTIVE, types.UVStatus_UV_STATUS_PENDING_JOIN:
            voters = append(voters, val)
        }
        return false, nil
    })
}
```

File: `push-chain-node/x/uexecutor/keeper/voting.go`

```
universalValidatorSet, err := k.uvalidatorKeeper.GetEligibleVoters(
    ctx)
if err != nil {
    return false, false, err
}

totalValidators := len(universalValidatorSet)

votesNeeded := (types.VotesThresholdNumerator*totalValidators)/types.VotesThresholdDenominator + 1
// ...
```

```
_, isFinalized, isNew, err = k.uvalidatorKeeper.VoteOnBallot(
    ctx,
    ballotKey,
    uvalidatorkeeper.BallotObservationType_BALLOT_OBSERVATION_TYPE_INBOUND_TX,
    universalValidator.String(),
    uvalidatorkeeper.VoteResult_VOTE_RESULT_SUCCESS,
    universalValidator.SetStrs,
    int64(votesNeeded),
    int64(types.DefaultExpiryAfterBlocks),
)
```

File: `push-chain-node/x/uvalidator/types/ballot.go` ·

```
func NewBallot(
    id string,
    ballotType BallotObservationType,
    voters []string,
    votesNeeded int64,
    createdHeight int64,
    expiryAfterBlocks int64,
) Ballot {
    b := Ballot{
        Id: id,
        BallotType: ballotType,
        EligibleVoters: voters,
        VotingThreshold: votesNeeded,
        Status: BallotStatus_BALLOT_STATUS_PENDING,
        BlockHeightCreated: createdHeight,
        BlockHeightExpiry: createdHeight + expiryAfterBlocks,
    }
    b.InitEmptyVotes()
    return b
}
```

File: `push-chain-node/x/uexecutor/keeper/msg_server.go` ·

```
isBonded, err := ms.k.uvalidatorKeeper.IsBondedUniversalValidator(
    ctx, msg.Signer)
// ...
if !isBonded {
    return nil, fmt.Errorf("universal validator for signer %s is not bonded", msg.Signer)
}

isTombstoned, err := ms.k.uvalidatorKeeper.IsTombstonedUniversalValidator(
    ctx, msg.Signer)
// ...
if isTombstoned {
    return nil, fmt.Errorf("universal validator for signer %s is tombstoned", msg.Signer)
}
}
```

File: `push-chain-node/app/app.go` ·

```
// register the staking hooks
app.StakingKeeper.SetHooks(
    stakingtypes.NewMultiStakingHooks(
        app.DistrKeeper.Hooks(),
        app.SlashingKeeper.Hooks(),
    ),
)
```

No `x/uvalidator` staking hook is registered here, so base-chain lifecycle events do not drive universal-validator reconciliation.

File: `push-chain-node/x/uexecutor/types/constants.go` ·

```
// Default number of blocks after which ballot expires.  
// Set to 100M (~19 years at 6s blocks) to effectively disable expiry.  
DefaultExpiryAfterBlocks = 100_000_000
```

Let N denote the count returned by `GetEligibleVoters` at ballot creation and S the count of those entries that can no longer cast valid votes because they are unbonded, jailed, or tombstoned on staking. The ballot stores a threshold based on N while live votes are capped by $N - S$. When S is large enough, passing the stored threshold is impossible, so inbound and outbound observations stop finalizing. Because expiry is configured for an extremely long horizon and the threshold is fixed at creation, cleaning the universal validator set afterward does not retroactively fix ballots that were already created. Users experience stuck deposits or settlements operators lose a clear on-chain escape hatch unless governance or code adds one.

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

- Subscribe to staking and slashing hooks (or an equivalent periodic reconciler) so universal-validator lifecycle moves out of `ACTIVE` when the base validator unbonds, is jailed, or is tombstoned, and coordinate with executor and TSS hooks as needed.
- Narrow `GetEligibleVoters` so the denominator matches who may vote, for example by requiring bonded and non-tombstoned staking state before including a row.
- Provide a governed or operational path to recompute `EligibleVoters` and `VotingThreshold` on existing pending ballots when the set was wrong, or to cancel and recreate ballots with correct parameters.

Resolution: This issue has been addressed and resolved in commit [38fbf73](#).

[F-2026-16992](#) - Removing an active universal validator bypasses the ongoing-TSS guard used elsewhere - Low

Description:

When an administrator updates universal-validator status through the dedicated status message, the keeper refuses the change if the TSS module reports an ongoing process. When an administrator removes a validator that is currently **ACTIVE**, the removal path transitions the validator to **pending leave** without calling that same guard. The pending-join branch does consult current TSS participants before allowing removal, so protection is asymmetric: one admin path respects in-flight key material while another can still mutate the eligible set during keygen or resharing.

Removing an active validator triggers hooks that lead **x/utss** to start a new key process. That routine force-expires any still-active TSS process by setting its expiry height into the past, then begins a new round. A mistaken or malicious removal during TSS therefore tears down the in-flight round and can strand or confuse off-chain participants until operators recover manually.

File: `push-chain-node/x/validator/keeper/msg_update_universal_validator_status.go`

```
isOngoingTSS, err := k.UtssKeeper.HasOngoingTss(ctx)
if err != nil {
    return fmt.Errorf("failed to check TSS state: %w", err)
}
if isOngoingTSS {
    k.Logger().Warn("cannot update validator status: TSS process is ongoing",
        "validator", coreValidatorAddr,
        "requested_status", newStatus.String(),
    )
    return fmt.Errorf("cannot update validator status: TSS process is ongoing")
}
```

File: `push-chain-node/x/validator/keeper/msg_remove_universal_validator.go`

```
switch val.LifecycleInfo.CurrentStatus {
case types.UVStatus_UV_STATUS_ACTIVE:
    // Active -> Pending Leave
    if err := k.UpdateValidatorStatus(ctx, valAddr, types.UVStatus_UV_STATUS_PENDING_LEAVE); err != nil {
        return fmt.Errorf("failed to mark validator %s as pending leave: %w", universalValidatorAddr, err)
    }

    newStatus = types.UVStatus_UV_STATUS_PENDING_LEAVE

case types.UVStatus_UV_STATUS_PENDING_JOIN:
    // Check if validator is part of the current TSS process
    currentParticipants, err := k.UtssKeeper.GetCurrentTssParticipants(ctx)
    // ...
    if isParticipant {
        return fmt.Errorf("validator %s is part of the current TSS process and cannot be removed", universalValidatorAddr)
    }
}
```

The `ACTIVE` branch contains no `HasOngoingTss` or participant check before `UpdateValidatorStatus`.

File: `push-chain-node/x/utss/keeper/initiate_tss_key_process.go`

```
existing, err := k.CurrentTssProcess.Get(ctx)
if err == nil {
    if currentHeight < existing.ExpiryHeight {
        k.Logger().Info("Validator set changed: force-expiring current TSS process",
            "old_process_id", existing.Id,
            "old_expiry", existing.ExpiryHeight,
            "current_height", currentHeight)

        existing.ExpiryHeight = currentHeight - 1
        // ...
    }
}
```

An admin message that moves an `ACTIVE` universal validator to `pending leave` during TSS cancels the current process in favor of a new one without the same explicit refusal that status updates receive. Operators who assume “the chain blocks UV changes during TSS” may be misled because that rule does not apply to removal. Under a compromised admin key, the gap becomes an intentional denial vector against keygen. Even in benign cases, accidental removal mid-round wastes work and requires coordinated restart of off-chain signers.

Status:

Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

- Reuse `HasOngoingTss` at the start of `RemoveUniversalValidator`, or before the `ACTIVE` branch, mirroring `UpdateUniversalValidatorStatus`.
- Alternatively, reject removal of any address returned by `GetCurrentTssParticipants` regardless of universal-validator lifecycle state, so active participants cannot be dropped during a round.
- Document until fixed that `MsgRemoveUniversalValidator` is not safe to send while TSS is in progress.
- Consider surfacing a single policy module or precondition helper shared by all admin mutators of the universal validator set.

Resolution:

This issue has been addressed and resolved in commit [3f3d73d](#).

[F-2026-16993](#) - Votes are accepted on pending ballots without checking block height expiry - Low

Description:

Ballots carry an expiry height computed at creation. Expiry is applied when new ballots are created, because `CreateBallot` runs `ExpireBallotsBeforeHeight` first. Existing ballots are returned unchanged from `GetOrCreateBallot`, and the vote handler only requires status `PENDING`. Neither path compares the current block height to `BlockHeightExpiry` before recording a vote. Therefore a ballot that has passed its nominal expiry but has not yet been marked `EXPIRED` in state (because no subsequent `CreateBallot` ran to trigger the sweep) can still accept votes and reach a passing finalization. Outcomes can depend on whether another ballot creation occurred in between, which is a poor contract for operators and for any logic that assumes expiry is a hard cutoff at the vote boundary.

File: `push-chain-node/x/uvalidator/keeper/voting.go` ·

```
ballot, isNew, err = k.GetOrCreateBallot(ctx, id, ballotType, voter
s, votesNeeded, expiryAfterBlocks)
if err != nil {
    return ballot, false, false, errors.Wrap(err, "Error while voting o
n the ballot")
}

if ballot.Status != types.BallotStatus_BALLOT_STATUS_PENDING {
    k.Logger().Warn("ballot is not in pending state, cannot vote",
"ballot_id", id,
"ballot_status", ballot.Status.String(),
"voter", voter,
)
    return ballot, false, false, fmt.Errorf("ballot %s is already %s",
id, ballot.Status.String())
}
```

There is no `ballot.IsExpired(currentHeight)` check in this flow.

File: `push-chain-node/x/uvalidator/types/ballot.go` ·

```
func (b Ballot) AddVote(address string, vote VoteResult) (Ballot, e
rror) {
    if b.Status != BallotStatus_BALLOT_STATUS_PENDING {
        return b, fmt.Errorf("cannot vote on ballot %s: not pending", b.Id)
    }
    // ...
}

func (b Ballot) IsExpired(currentHeight int64) bool {
    return currentHeight >= b.BlockHeightExpiry
}
```

File: `push-chain-node/x/uvalidator/keeper/ballot.go` ·

```
if err := k.ExpireBallotsBeforeHeight(ctx, blockHeight); err != nil
{
    return types.Ballot{}, err
}
```

```
func (k Keeper) GetOrCreateBallot(
    ctx context.Context,
    id string,
    ballotType types.BallotObservationType,
    voters []string,
    votesNeeded int64,
    expiryAfterBlocks int64,
) (types.Ballot, bool, error) {

    if ballot, err := k.Ballots.Get(ctx, id); err == nil {
        k.Logger().Debug("ballot found (existing)", "ballot_id", id)
        return ballot, false, nil
    }

    newBallot, err := k.CreateBallot(ctx, id, ballotType, voters, votes
    Needed, expiryAfterBlocks)
```

Returning an existing ballot skips `ExpireBallotsBeforeHeight`.

File: `push-chain-node/x/validator/abci.go` .

```
func BeginBlocker(ctx sdk.Context, validatorKeeper keeper.Keeper)
error {
    // ...
    if height > 1 {
        if err := AllocateTokens(ctx, previousTotalPower, ctx.VoteInfos(),
        validatorKeeper); err != nil {
```

`BeginBlocker` allocates tokens and does not call `ExpireBallotsBeforeHeight`.

Votes cast after the documented expiry height can still finalize a ballot as passed if the lazy expiry sweep has not run. Downstream modules cannot treat `BlockHeightExpiry` as a strict cutoff unless they duplicate height checks. When traffic is low, the order in which unrelated ballots are created can change whether a late vote succeeds or fails after another creation triggers expiry, which is confusing for monitoring and incident response. Today the default expiry span from the executor is extremely large, so the defect is mostly latent until expiry parameters become operationally meaningful.

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

- Reject votes in `VoteOnBallot` (or inside vote aggregation) when `ballot.IsExpired(currentHeight)`, transition the ballot to `EXPIRED`, and move identifiers between active and expired collections for consistency.

- Invoke `ExpireBallotsBeforeHeight` from `BeginBlocker` or `EndBlocker` each height so state tracks expiry independently of ballot creation rate.
- Ensure finalization logic refuses to mark a ballot passed when the current height is at or beyond `BlockHeightExpiry`, even if the yes-count threshold is met afterward.

Resolution:

This issue has been addressed and resolved in commit [3948c36](#).

F-2026-17022 - Case-sensitive token keys versus case-insensitive PRC20 lookup and EIP-55 observer addresses - Low

Description:

Token configuration entries are stored under a composite key built from the chain identifier and the token address string with whitespace trimmed but case preserved. Direct reads use that key verbatim, while `GetTokenConfigByPRC20` compares PRC20 contract addresses using lowercasing. The EVM observer populates inbound asset addresses using `go-ethereum`'s `Address.Hex()`, which yields EIP-55 mixed-case strings. An administrator who registers the same logical token using all-lowercase text from a block explorer therefore stores a different key than the one the executor looks up at runtime, so inbounds can fail even though a token row exists. Two registrations that differ only in case are treated as distinct rows, which invites duplicate configuration and asymmetric behavior between inbound and outbound resolution paths.

File: `push-chain-node/x/uregistry/types/keys.go` ·

```
func GetTokenConfigsStorageKey(chain, address string) string {
    return fmt.Sprintf("%s:%s", chain, strings.TrimSpace(address))
}
```

File: `push-chain-node/x/uregistry/keeper/keeper.go` ·

```
func (k Keeper) GetTokenConfig(ctx context.Context, chain, address
string) (types.TokenConfig, error) {
    storageKey := types.GetTokenConfigsStorageKey(chain, address)
    config, err := k.TokenConfigs.Get(ctx, storageKey)
    if err != nil {
        return types.TokenConfig{}, err
    }
    return config, nil
}
```

File: `push-chain-node/x/uregistry/keeper/keeper.go` ·

```
func (k Keeper) GetTokenConfigByPRC20(
    ctx context.Context,
    chain string,
    prc20Addr string,
) (types.TokenConfig, error) {

    prc20Addr = strings.ToLower(strings.TrimSpace(prc20Addr))

    var found *types.TokenConfig

    err := k.TokenConfigs.Walk(ctx, nil, func(
        key string,
        cfg types.TokenConfig,
    ) (bool, error) {

        if cfg.Chain != chain {
            return false, nil
        }

        if cfg.NativeRepresentation == nil {
            return false, nil
        }
    })
}
```

```

if strings.ToLower(cfg.NativeRepresentation.ContractAddress) == prc
20Addr {
    found = &cfg
    return true, nil
}

return false, nil
})

```

File: `push-chain-node/universalClient/chains/evm/event_parser.go` ·

```

payload.Token = ethcommon.BytesToAddress(log.Data[0*32+12 : 0*32+32
]).Hex()
payload.Amount = new(big.Int).SetBytes(log.Data[1*32 : 2*32]).Strin
g()

```

Downstream, `x/uexecutor` resolves inbound assets via `GetTokenConfig` using that string as the address component, so any mismatch between registered casing and observed casing produces a not-found result and revert-style handling on the inbound path.

Operators can see systematic inbound failures after a token was "registered," because the stored key does not match the checksum form emitted by the client. Outbound flows that rely on PRC20 lookup may still resolve, so the two directions disagree. Duplicate rows for the same token under different casings create ambiguous or non-deterministic behavior when walking the map, and removals keyed by the wrong case appear to succeed at the message layer while leaving stale entries in state.

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

- Canonicalize EVM addresses when forming storage keys (for example lowercase for `eip155:*` namespaces) and apply the same rule on read and remove paths.
- Enforce valid hex address form and a single canonical representation in `TokenConfig.ValidateBasic` (and related fields such as native representation contract addresses).
- Reject duplicate registrations for the same chain when addresses are equal under case-insensitive comparison.
- Add a one-time migration to rewrite existing keys to the canonical form where mixed-case keys already exist.

Resolution:

This issue has been addressed and resolved in commit [93de525](#).

[F-2026-17025](#) - isContractDeployed treats any account with a non-empty code hash as a deployed contract - Low

Description:

Before deploying each system proxy, the registry checks whether the proxy address already has a “deployed” contract. The implementation treats any account whose `CodeHash` slice is non-empty as deployed. In Ethereum-compatible state, externally owned accounts still carry the keccak256 hash of empty bytecode (a fixed 32-byte sentinel), so the length check is true for EOAs as well as for contracts. On a fresh genesis with no prior accounts at the reserved addresses, deployment proceeds as intended. On a chain where an address was touched as an EOA before deployment runs, the predicate can skip the entire deploy sequence for that slot, leaving system contracts missing while logs suggest they were already present.

File: `push-chain-node/x/uregistry/keeper/genesis.go` ·

```
func isContractDeployed(  
    ctx sdk.Context,  
    evmKeeper types.EVMKeeper,  
    addr common.Address,  
    ) bool {  
    acc := evmKeeper.GetAccount(ctx, addr)  
    return acc != nil && acc.CodeHash != nil && len(acc.CodeHash) != 0  
}
```

File: `push-chain-node/x/uregistry/keeper/genesis.go` ·

```
proxyAddr := common.HexToAddress(contract.Address)  
if isContractDeployed(sdkCtx, evmKeeper, proxyAddr) {  
    sdkCtx.Logger().Info(  
        "system contract already deployed, skipping",  
        "name", name,  
        "proxy", proxyAddr.Hex(),  
    )  
    continue  
}
```

The skip branch omits proxy admin, implementation, and proxy deployment for that name.

File: `push-chain-node/x/uexecutor/types/constants.go` ·

```
// EmptyCodeHash is the keccak256 hash of empty bytes – the code hash for EOAs (no contract code).  
var EmptyCodeHash = common.HexToHash("0xc5d2460186f7233c927e7db2dcc703c0e500b653ca82273b7bfad8045d85a470")
```

File: `push-chain-node/x/uexecutor/keeper/execute_inbound_gas_and_payload.go` ·

```
codeHash := k.evmKeeper.GetCodeHash(sdkCtx, ueaAddr)  
if codeHash != types.EmptyCodeHash && codeHash != (common.Hash{}) {
```

```
isSmartContract = true  
}
```

A future upgrade that introduces a new reserved address or re-runs deployment on live state could skip installation if anyone created an EOA or sent value to that address first. The chain would then lack the expected proxy triple for that system contract, with only an informational log as the signal. Manual state surgery or a patched deploy routine would be required to recover.

Status:**Fixed****Classification****Impact Rate:** 2/5**Likelihood Rate:** 1/5**Severity:** **Low****Recommendations****Remediation:**

- Replace the predicate with an explicit comparison against the empty-code-hash sentinel (and zero hash if applicable), using `GetCodeHash` or `BytesToHash` on `CodeHash`, consistent with `x/uexecutor`.
- Centralize `EmptyCodeHash` in a shared types package or import the existing constant so both modules agree.
- Add a test that seeds an EOA at a system proxy address and asserts that deployment still runs or that behavior matches the chosen policy.
- If reserved addresses are protocol-owned regardless of prior activity, document that policy and consider overwrite semantics for those slots instead of skip-on-any-account.

Resolution:

This issue has been addressed and resolved in commit [a426e3f](#).

[F-2026-17038](#) - Genesis export/import omits pending TSS event

index - Low

Description:

The `x/utss` module keeps a secondary index `PendingTssEvents` (`process_id` → `event_id`) for active process-initiated events. `GenesisState` in protobuf does not include this map, and `InitGenesis` never rebuilds it from the restored `tss_events` slice. After genesis export and import (or a handcrafted genesis that lists events but not the index), RPC queries that read `PendingTssEvents` return errors or empty results even when matching `TssEvent` rows exist.

Genesis schema has no pending index field — `push-chain-`

`node/proto/utss/v1/genesis.proto`

```
// GenesisState defines the module genesis state
message GenesisState {
  // Params defines all the parameters of the module.
  Params params = 1 [(gogoproto.nullable) = false];
  // ...
  // tss_events are entries from the TssEvents store.
  repeated TssEvent tss_events = 7 [(gogoproto.nullable) = false];
  // next_tss_event_id is the next auto-increment ID for TssEvents.
  uint64 next_tss_event_id = 8;
  // ...
}
```

Init restores events but not `**PendingTssEvents**` — `push-chain-`

`node/x/utss/keeper/keeper.go`

```
// Restore TssEvents
for _, event := range data.TssEvents {
  if err := k.TssEvents.Set(ctx, event.Id, event); err != nil {
    return err
  }
}

// Restore NextTssEventId
if data.NextTssEventId > 0 {
  if err := k.NextTssEventId.Set(ctx, data.NextTssEventId); err != nil {
    return err
  }
}
```

There is no loop calling `PendingTssEvents.Set` for events with `TSS_EVENT_PROCESS_INITIATED` and `TSS_EVENT_ACTIVE` (or equivalent pending semantics).

Queries depend on the index — `push-chain-`

`node/x/utss/keeper/query_server.go`

```
func (k Querier) GetPendingTssEvent(goCtx context.Context, req *types.QueryGetPendingTssEventRequest) (*types.QueryGetPendingTssEventResponse, error) {
  ctx := sdk.UnwrapSDKContext(goCtx)

  eventId, err := k.Keeper.PendingTssEvents.Get(ctx, req.ProcessId)
  if err != nil {
```

```
return nil, err
}
// ...
}
```

Operators and the `universalClient/tss` stack that rely on pending-event queries can lose visibility into the active TSS process immediately after a genesis-based restart, even though `AllTssEvents` still lists the underlying rows. That complicates recovery, monitoring, and automation that keys off the pending index.

Status:

Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

1. **Preferred:** After loading `TssEvents` in `InitGenesis`, rebuild `PendingTssEvents` deterministically (for each event matching pending semantics, `Set(process_id, event_id)`), and assert uniqueness invariants.
2. **Alternative:** Extend `GenesisState` with an explicit repeated pending-index entry and validate it against `tss_events` on import.
3. Add genesis tests that export state with an active pending process-initiated event, re-import, and assert `GetPendingTssEvent` succeeds.

Resolution:

This issue has been addressed and resolved in commit [7be2938](#).

[F-2026-17041](#) - Fund migration ballot key uses raw txHash string -

Low

Description:

Fund migration voting derives ballot keys from `migrationId`, the raw `txHash` string, and the boolean `success`. There is no canonicalization (for example lowercasing hex, stripping `0x`) in `x/utss` before hashing. The same underlying chain transaction can therefore produce multiple distinct ballots, fragmenting quorum.

Ballot key includes raw `**txHash**`

`push-chain-node/x/utss/types/keys.go`

```
func GetFundMigrationBallotKey(migrationId uint64, txHash string, success bool) string {
    canonical := fmt.Sprintf("fm:%d:%s:%t", migrationId, txHash, success)
    h := sha256.Sum256([]byte(canonical))
    return hex.EncodeToString(h[:])
}
```

Vote path passes message hash through unchanged

`push-chain-node/x/utss/keeper/voting.go`

```
func (k Keeper) VoteOnFundMigrationBallot(
    ctx context.Context,
    universalValidator sdk.ValAddress,
    migrationId uint64,
    txHash string,
    success bool,
) (isFinalized bool, isNew bool, err error) {

    ballotKey := types.GetFundMigrationBallotKey(migrationId, txHash, success)
```

`MsgVoteFundMigration` has no `ValidateBasic` in `x/utss/types` that normalizes `txHash` before the message server calls `VoteFundMigration`.

Without a single canonical representation, honest validators observing the same migration transaction can fail to aggregate votes, leaving migrations pending until operators converge on an off-chain convention or retry with a normalized hash.

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Severity:

Low

Recommendations**Remediation:**

1. Canonicalize `txHash` once in the message server (or in `VoteFundMigration`) using the same rules as other observation paths (strip `0x`, lowercase for EVM-length hex, reject malformed lengths).
2. Add regression tests with equivalent hash encodings that must map to one ballot.

Resolution:

This issue has been addressed and resolved in commit [93de525](#).

[F-2026-17043](#) - usigverifier Ed25519 precompile verifies signature over ASCII hex, not the raw message - Low

Description:

`verifyEd25519` takes a `bytes32` value from the caller but checks the signature against the **characters** `0x` followed by that value written out in hex — not against the **32 raw bytes** themselves. Most wallets and libraries sign the **raw** bytes (or a hash treated as raw bytes). So anything that assumes “on-chain check = same bytes we signed off-chain” will **fail** unless every signer uses this exact hex-string rule. Teams can also **think** they are securing a hash when they are really securing a different byte string.

[push-chain-node/precompiles/usigverifier/query.go](#)

```
msgStr := "0x" + hex.EncodeToString(msg) // Convert the message to
a hex string
msgBytes := []byte(msgStr) // Convert the message string to original
signed bytes

ok = ed25519.Verify(pubKeyBytes, msgBytes, signature)
```

The public interface still names the second argument like a normal 32-byte message, which reads as “verify this digest” to anyone building contracts or off-chain signers.

Contracts can believe they only accept a signature over a given 32-byte hash, while the built-in check actually requires a signature over the **UTF-8 bytes of** `0x` plus the hex form of that hash. Ordinary signing stacks will not produce a passing signature without a custom step, so honest users see **rejections** and flows that depend on this verifier can **stall**. Any design that joins Push with Solana or another Ed25519 chain must spell out the same signing bytes everywhere (relayer, wallet, and contract); if they do not, relayers and on-chain checks can disagree in hard-to-spot ways. In short, the harm is broken or stuck verification, wrong trust in “we verified the hash,” and cross-chain setups that drift unless everyone follows the non-obvious hex-string rule.

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

1. **Preferred:** Add another entry point (for example `verifyEd25519Message`) that checks signatures over the **raw** message bytes (or over an explicit `bytes` blob with a clear naming convention), and document whether the old hex-string behavior stays for backward compatibility.
2. **Documentation:** If the hex-string behavior is intentional, say so in the contract interface docs and integration guides so every signer uses the same bytes.
3. **Tests:** Add fixed test vectors that show “sign raw 32-byte digest” vs “sign `0x` + hex as text” and what each API expects

Resolution:

This issue has been addressed and resolved in commit [5659657](#).

[F-2026-17736](#) - Derived call can commit state even when commit=false - Low

Description:

`DerivedEVMCallWithData` accepts a `commit` flag but calls `ApplyMessageWithConfig` with a hardcoded `true` and then commits cache context on success. This means the function can persist state even when callers pass `commit=false`.

```
tmpCtx, commitState := ctx.CacheContext()

// pass true to commit the StateDB
res, err := k.ApplyMessageWithConfig(tmpCtx, msg, nil, true, cfg, txConfig)
if err != nil {
    return nil, err
}

if !res.Failed() {
    commitState()
}
```

Any integration expecting non-mutating behavior from `commit=false` on derived execution can accidentally mutate chain state.

Assets:

- <https://github.com/pushchain/push-chain-evm/>

Status:

Fixed

Classification

Impact Rate: 4/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

- Pass `commit` into `ApplyMessageWithConfig`.
- Gate `commitState()` behind `if commit && !res.Failed()`.
- Add regression tests for both `commit=true` and `commit=false`.

Resolution:

This issue has been addressed in commit [f7b229f5](#), which wires the `commit` flag through `DerivedEVMCallWithData` and adds regression tests to ensure `commit=false` does not persist state.

F-2026-17738 - Reverted derived execution still emits events and mutates block bloom - Low

Description:

When `commit=true`, the code emits `ethereum_tx` / `tx_log` events and updates block bloom inside the same branch, even if `res.Failed()` is true. Since cache commit is skipped on failed execution, emitted metadata can represent uncommitted state.

```
// Emit events and log for the transaction if it is committed
if commit {
// ...
if res.Failed() {
  attrs = append(attrs, sdk.NewAttribute(types.AttributeKeyEthereumTx
Failed, res.VmError))
}
// ...
ctx.EventManager().EmitEvents(sdk.Events{
  sdk.NewEvent(types.EventTypeEthereumTx, attrs...),
  sdk.NewEvent(types.EventTypeTxLog, txLogAttrs...),
})
// ...
k.SetBlockBloomTransient(ctx, bloomReceipt.Big())
k.SetLogSizeTransient(ctx, (k.GetLogSizeTransient(ctx))+uint64(len(
logs)))
}
```

Clients can observe logs/receipts/bloom effects for executions that did not commit state, leading to inconsistency across explorers, traces, and downstream indexers.

Assets:

- <https://github.com/pushchain/push-chain-evm/>

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

- Emit derived events and mutate bloom only under `commit && !res.Failed()`.
- Add tests ensuring failed derived execution does not publish log/bloom side effects.

Resolution:

This issue has been addressed in commit [1a8d637](#), which gates block bloom and log-size updates on successful derived execution and adds regression tests ensuring reverted derived calls emit no log attributes while preserving `ethereum_tx` / `tx_log` event pairing.

F-2026-17740 - KV indexer path misses derived transactions - Low

Description:

The KV indexer persists Ethereum transaction metadata for accelerated JSON-RPC resolution. Indexing is gated on `isEthTx(tx)`, which requires an Ethereum extension option and an embedded `MsgEthereumTx`. Derived transactions—EVM executions initiated internally by Cosmos modules and recorded exclusively as Tendermint event attributes—do not satisfy this predicate and are therefore excluded from the index.

At query time, the RPC backend delegates to the indexer when one is configured. On a cache hit, the indexer response is returned without supplementary event parsing. On a miss for a derived transaction hash, no fallback path reconstructs the transaction from block events. The execution occurred on-chain and the events are present in the block result, but indexer-backed nodes cannot resolve the transaction through standard `eth_getTransactionByHash` and related APIs.

`push-chain-evm/indexer/kv_indexer.go`

```
if !isEthTx(tx) {  
    continue  
}
```

`push-chain-evm/rpc/backend/tx_info.go`

```
if b.indexer != nil {  
    txRes, err := b.indexer.GetByTxHash(hash)  
    if err != nil {  
        return nil, nil, err  
    }  
    return txRes, nil, nil  
}
```

Nodes operating with the KV indexer enabled exhibit incomplete or failed responses for derived transaction lookups, while nodes without the indexer may resolve the same hash via event reconstruction. This deployment-dependent behavior violates JSON-RPC response consistency expectations and impairs integrators, block explorers, and indexing infrastructure that assume uniform transaction availability across node configurations.

Assets:

- <https://github.com/pushchain/push-chain-evm/>

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

- Extend the KV indexer ingestion pipeline to parse and persist derived transaction records from Tendermint execution events, using the same field schema applied to standard `MsgEthereumTx` entries.
- Alternatively, implement a fallback in the RPC retrieval path: when the indexer returns a miss or lacks derived-transaction metadata, delegate to the existing event-query reconstruction logic before returning a not-found response.

Resolution:

This issue has been addressed in commit [4b0cc66e](#), which indexes derived EVM transactions in the KV indexer, rebuilds their RPC fields from events on indexer hits, and falls back to event-query reconstruction on indexer misses for backward compatibility.

F-2026-17745 - Derived transactions share a constant txIndex value

- Low

Description:

During derived transaction event emission, the keeper attaches a transaction index attribute using the compile-time constant `DerivedTxIndex` (9999) for every internal EVM execution, regardless of its position within the block's EVM message sequence. The constant functions as a placeholder indicating non-standard indexing rather than a unique per-block ordinal.

When multiple derived executions occur within a single block, each emits an identical index value. RPC endpoints that resolve transactions by block number and index (`eth_getTransactionByBlockNumberAndIndex`), as well as any ordering or deduplication logic keyed on transaction index, cannot distinguish among co-located derived transactions.

`push-chain-evm/x/vm/types/events.go`

```
const (  
    DerivedTxIndex = 9999  
    DerivedTxType = 99  
)
```

`push-chain-evm/x/vm/keeper/call_evm.go`

```
// add event for index of valid ethereum tx; NOTE: default txindex  
for derivedTx  
sdk.NewAttribute(types.AttributeKeyTxIndex, strconv.FormatUint(type  
s.DerivedTxIndex, 10)),
```

Index collision introduces ambiguity in transaction retrieval, block ordering, and explorer presentation for derived executions. In blocks containing multiple derived transactions or a mixture of standard and derived messages, index-based queries may return incorrect results or fail to resolve the intended transaction. Severity increases with block throughput and the frequency of module-initiated EVM calls.

Assets:

- <https://github.com/pushchain/push-chain-evm/>

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Severity:

Low

Recommendations**Remediation:**

- Assign a unique, monotonically increasing Ethereum transaction index to each derived execution within a block, consistent with the indexing scheme applied to standard `MsgEthereumTx` messages.
- Align event emission in `call_evm.go` with the `EthTxIndex` semantics consumed by the RPC layer and KV indexer.

Resolution:

This issue has been addressed in commit [a6e46dfb](#), which replaces the hardcoded `DerivedTxIndex` with the shared block-level transient counter so each derived execution receives a unique, monotonically increasing Ethereum transaction index consistent with standard `MsgEthereumTx` indexing.

F-2026-17752 - Derived RPC transaction uses GasUsed as gas field - Low

Description:

The JSON-RPC transaction object schema defines the `gas` field as the transaction gas limit—the maximum gas units the sender authorized for execution. Actual gas consumption is reported separately in the transaction receipt via the `gasUsed` field.

In the derived transaction reconstruction path within `rpc/types/utls.go`, the `Gas` field of the returned `RPCTransaction` is populated from `txAdditional.GasUsed` rather than the corresponding gas limit attribute. This inverts the semantic contract of the field: clients receive execution consumption data where limit data is expected. The assignment is deterministic for all derived transaction RPC responses constructed through this code path.

`push-chain-evm/rpc/types/utls.go`

```
result := &RPCTransaction{
    Type: hexutil.Uint64(txAdditional.Type),
    From: common.HexToAddress(msg.From),
    Gas: hexutil.Uint64(txAdditional.GasUsed),
    GasPrice: (*hexutil.Big)(baseFee),
    Hash: common.HexToHash(msg.Hash),
```

Downstream consumers—including wallet software, block explorers, indexing services, and gas analysis tooling—that interpret the `gas` field as a limit will derive incorrect metadata for derived transactions. Gas utilization ratios, headroom calculations, and fee estimation logic applied uniformly across transaction types will produce erroneous results for this transaction class.

Assets:

- <https://github.com/pushchain/push-chain-evm/>

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

- Populate the `Gas` field from `txAdditional.GasLimit` or the equivalent gas-limit event attribute recorded during execution.
- Restrict gas consumption reporting to receipt-level fields (`gasUsed`) in accordance with the Ethereum JSON-RPC specification.

Resolution:

This issue has been addressed in commit [7062e604](#), which sets the derived transaction RPC `gas` field from the recorded gas limit (`txGasLimit`) instead of `gasUsed` , with a fallback to `gasUsed` only for legacy derived txs that did not emit a gas limit attribute.

[F-2026-17754](#) - TraceTransaction mixes Cosmos and Ethereum index domains - Low

Description:

The `TraceTransaction` implementation reconstructs the predecessor transaction set required for accurate EVM state replay. The outer predecessor loop bounds its iteration by `transaction.TxIndex`, which represents the ordinal position of the enclosing Cosmos transaction within the Tendermint block. Within each iteration, the loop invokes `GetTxByTxIndex`, which resolves transactions by Ethereum-level transaction index—a distinct coordinate space that enumerates EVM messages across the block's execution ordering.

These index domains diverge whenever a Cosmos transaction contains zero, one, or multiple EVM messages, or when derived transactions exist only as event records rather than embedded messages. Applying a Cosmos transaction ordinal as the upper bound for Ethereum-indexed lookups produces an incorrect predecessor set, potentially omitting relevant state-modifying executions or including unrelated transactions.

```
**push-chain-evm/rpc/backend/tracing.go**
```

```
var predecessors []*evmtypes.MsgEthereumTx
for i := 0; i < int(transaction.TxIndex); i++ {
    predecessorTx, txAdditional, err := b.GetTxByTxIndex(blk.Block.Height, uint(i))
```

Trace output may reflect an execution context that diverges from the actual on-chain state at the point of target transaction execution. Debugging workflows, incident forensics, and automated trace analysis that depend on `debug_traceTransaction` may produce misleading call stacks, incorrect storage values, and erroneous gas accounting. The defect manifests in blocks with heterogeneous transaction compositions containing both standard and derived EVM executions.

Assets:

- <https://github.com/pushchain/push-chain-evm/>

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

- Construct the predecessor set from a single canonical ordering source —the parsed per-block Ethereum message list derived from both embedded messages and event-reconstructed transactions.
- Eliminate cross-domain index translation between Cosmos transaction ordinals and Ethereum transaction index APIs within the trace reconstruction path.

Resolution:

This issue has been addressed in commit [438559e6](#), which rebuilds `TraceTransaction` predecessor assembly using `EthTxIndex` (Ethereum execution order) instead of Cosmos `TxIndex`, decodes predecessor Cosmos txs via the resolved slot index, and adds derived-tx-specific handling so mixed standard/derived blocks trace without incorrect predecessors or panics.

[F-2026-17758](#) - Missing upstream security patch train after fork point - Low

Description:

The Push Chain EVM fork diverged from an earlier snapshot of the upstream [cosmos/evm](#) repository and has not incorporated the subsequent security and correctness patch train released after the fork point. The resulting version gap encompasses a substantial set of upstream commits addressing audit findings, consensus safety, IBC callback integrity, fee market hardening, StateDB correctness, ante handler validation, precompile compatibility, and JSON-RPC/tracing defects.

The commit register below enumerates upstream fixes not yet integrated, organized by functional domain with direct references to the source commits.

Deferred integration

- [43b70ee6](#) — Non-deterministic EVM pre-blocker correction.

Sherlock audit remediation

- [322cff37](#) — Post-audit remediation batch 1.
- [369defb9](#) — Post-audit remediation batch 3.
- [3e8dc586](#) — Post-audit remediation batch 2.
- [4f565128](#) — Final audit patch from evm-internal review.
- [84178554](#) — Apply missing audit remediation changes.
- [d360903c](#) — Post-audit remediation batch 5.
- [d7d62cd5](#) — Post-audit remediation batch 4.

Consensus and transaction policy

- [0517dcf0](#) — Revert "(chore): move feemarket update BaseFee to EndBlock (#190)".
- [18ad77e2](#) — Enforce EIP-2681 nonce upper bound in ante handler.
- [1d8b9e1e](#) — remove allow-unprotected-txs (non-EIP-155) from vm params.
- [76819bae](#) — enforce one EVM tx per Cosmos tx.
- [cffad658](#) — non-determinism fix.

IBC and ERC20 callbacks

- [0310aa96](#) — add missing ValidateBasic check in ICS02 precompile.
- [0e511d32](#) — correct ICS20 balance accounting.
- [1fa87d91](#) — fix zero gas config in IBC transfer keeper.
- [20c00074](#) — block nested ICS20 forwarding in source callbacks.
- [727f3b9d](#) — fix ICS02 and ICS20 gas calculation.
- [776462c6](#) — fix error acknowledgement in IBC v2 middleware.

- [810e9ef0](#) — fix IBC token decimals query revert condition.
- [83a3a6c7](#) — gas accounting fix in IBC callback.
- [91519e45](#) — fix IBC middleware sender verification.
- [b0e16435](#) — reorder ERC20 IBC callback logic.
- [ccaee527](#) — prevent arbitrary source callback execution.

Fee market and coin metadata

- [1fd8ef37](#) — fix missing `evmCoinInfo` in historical state.
- [2a9e6879](#) — avoid nil-pointer panic when BaseFee is zero.
- [491aa78f](#) — respect `NoBaseFee` in dynamic fee checker.
- [6de21f76](#) — avoid nil-pointer when RPC runs before `evmCoinInfo` init.
- [7998121a](#) — fix denom exponent checks.
- [a48de71c](#) — validate decimals before conversion in `GetBaseFee`.
- [e7835b93](#) — align units in base fee calculation.
- [f4637461](#) — add `InitEvmCoinInfo` upgrade guard.

StateDB and execution path

- [008c1711](#) — snapshot locked balance on StateDB account.
- [264aa70f](#) — harden StateDB balance and event amount handling.
- [c0f47851](#) — fix invalid message index errors in block RPCs with StateDB commit-error txs.

Ante and account handling

- [16ec2d0e](#) — fix non-EIP-155 signer panic.
- [8156e38e](#) — fix invalid pubkey address type handling.
- [89bb3819](#) — fix EOA vs contract-account identification.
- [9ddd976e](#) — include missing ante unit test suite.
- [befde4fa](#) — delete EVM instance in ante handler.

Precompiles

- [09e1895d](#) — fix gov precompile interface typo.
- [19992de1](#) — fix SDK-inconsistent distribution precompile method name.
- [31a6edf9](#) — fix evidence precompile time type.
- [4b72cd52](#) — fix SDK-incompatible slashing precompile types.
- [679305dc](#) — apply journal-based revert approach.
- [76d8d108](#) — return data on precompile revert.
- [82023069](#) — support dynamic precompiles with `eth_call` overrides.
- [8c0d0e75](#) — fix precompile structure for local node script.
- [a3a2a068](#) — fix ERC20 precompile event emission.
- [c5ca2676](#) — align precompiles map with static availability checks.
- [e2930f63](#) — avoid nil-pointer in gov precompile response conversion.
- [ebcaefa6](#) — use address codecs in precompiles.

Assets:

- <https://github.com/pushchain/push-chain-evm/>

Status:

Mitigated

Classification**Impact Rate:**

2/5

Likelihood Rate:

2/5

Severity:

Low

Recommendations**Remediation:**

- **Preferred approach:** Merge a recent stable release branch from `cosmos/evm`, then re-apply Push-specific custom modifications atop the updated baseline.
- **Interim approach:** Cherry-pick high-priority commit groups in order of risk: Sherlock audit remediations, non-determinism corrections, IBC callback safety patches, and StateDB/RPC hardening fixes.
- Maintain a patch integration ledger tracking each upstream commit with fields for integration status, test evidence, and ownership assignment.

Resolution:

This issue has been substantially addressed in commit [96231e7](#), which merges upstream `cosmos/evm` v0.5.1 and re-applies Push-specific customizations, covering the highest-priority upstream security fixes (including Sherlock audit and consensus/tx-policy patches); however, some later upstream fixes that landed after v0.5.1 (mainly IBC, StateDB/RPC, and fee-market) remain unintegrated and should be tracked in a follow-up upgrade to v0.7.0 or current upstream/main.

[F-2026-17775](#) - Inconsistent log reporting for failed derived transactions across JSON-RPC endpoints - Low

Description:

Push Chain represents module-initiated EVM executions as derived transactions recorded through ABCI events rather than embedded `MsgEthereumTx` messages. When a derived execution reverts with state persistence enabled, the keeper may still emit `ethereum_tx` and `tx_log` events even though the underlying EVM state transition is not committed. Under certain Universal Executor inbound flows, the enclosing Cosmos transaction nevertheless completes successfully, leaving those events available in block results.

The JSON-RPC layer handles failed derived transactions inconsistently across query methods. `GetTransactionReceipt` parses logs from execution events without regard to failure status. `GetTransactionLogs` returns an empty result whenever the resolved transaction is marked failed, without consulting the event stream. Consequently, `eth_getTransactionReceipt` and log-specific queries for the same transaction hash can return materially different results.

The defect manifests on inbound processing paths that treat EVM execution failure as a recoverable outcome within an otherwise successful Cosmos transaction. In these flows, execution errors from derived calls are recorded in application state but are not propagated as Cosmos message failures. Event attributes produced during the reverted EVM invocation—including transaction logs—therefore remain indexed in the block result.

On execution paths that return an error to the Cosmos message handler, the enclosing transaction fails and associated events are not retained. The inconsistency described herein primarily affects inbound gas and funds workflows that complete at the Cosmos layer despite EVM-level revert.

Receipt construction — logs parsed irrespective of failure status

`push-chain-evm/rpc/backend/tx_info.go`

```
msgIndex := int(res.MsgIndex) // #nosec G115 -- checked for int overflow already
logs, err := TxLogsFromEvents(blockRes.TxsResults[res.TxIndex].Events, msgIndex)
if err != nil {
    b.logger.Debug("failed to parse logs", "hash", hexTx, "error", err.Error())
}
```

Dedicated log query — empty response on failure

`push-chain-evm/rpc/backend/tx_info.go`

```
if res.Failed {
    // failed, return empty logs
}
```

```
return nil, nil
}
```

Inbound handler — execution error not propagated to Cosmos message result

`push-chain-node/x/uexecutor/keeper/execute_inbound_gas.go`

```
// Never return execErr, only nil
return nil
```

Block explorers, indexing infrastructure, and client applications that source log data from `eth_getTransactionReceipt` may ingest revert-associated logs that dedicated log queries omit for the identical transaction hash. This violates the expectation—observed on Ethereum mainnet—that failed executions do not yield divergent log representations across standard JSON-RPC interfaces.

Where reverted derived executions emit log events without a corresponding state commit, receipt-level log data may not reflect durable on-chain execution outcomes. Integrators operating dual ingestion paths or cross-validating receipt and log endpoints are exposed to silent data inconsistency without error indication.

Assets:

- <https://github.com/pushchain/push-chain-evm/>

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

At the execution layer, restrict derived transaction event emission—including `tx_log` attributes and bloom updates—to successful EVM invocations only. At the RPC layer, apply uniform failure semantics across `GetTransactionReceipt` and `GetTransactionLogs`, such that failed transactions return empty log sets through both interfaces. Introduce integration coverage for inbound revert scenarios in which the Cosmos transaction succeeds, verifying consistent log responses across all supported query methods.

Resolution:

This issue has been addressed in commit [4e1260cb](#).

[F-2026-18135](#) - [Dual Defense] Nested Stateful Precompile Double- Applies Balance Events (Upstream ASA-2026-002) - Low

Description:

Nested stateful precompile frames double-apply bank events via per-frame `prevEventsLen` delta replay in `BalanceHandler.AfterBalanceChange`. The vulnerable pattern processes `events[bh.prevEventsLen:]` without tracking globally processed events, so recursive precompile calls can replay `CoinSpent` and `CoinReceived` events and inflate StateDB balances relative to the bank module.

This tree uses the two-argument `AddPrecompileFn(snapshot int, events sdk.Events)` journal API without `processedEventsCount`, `MarkEventProcessed`, or `IsEventProcessed`. Active static precompiles including distribution and ICS20 are enabled at genesis. In-tree recursive-precompile tests document the same bank and StateDB desync class.

The defect matches the published cosmos/evm advisory ASA-2026-002 ([GHSA-54gx-3cgr-7mfm](#)), patched in cosmos/evm v0.6.0. This pin remains on a v0.5.1-class fork. The issue is inherited BalanceHandler logic, not Push-specific business logic. Reports that describe empty-account free mint or novel Push-module invention misstate the published advisory and the exploitation requirements on this pin.

Impact

- Nested precompile calls can double-apply balance events on this dependency pin.
- Phantom credit is proportional to moved amount per successful nesting, not unbounded mint from an empty account.
- Exploitation requires nested stateful precompile composition plus legitimate movable value such as accrued rewards or ICS20 transfers.
- Distribution pool impact is bounded by pool liquidity; StateDB and bank inflation breaks supply integrity for moved amounts.
- Active nested-capable precompiles on a live deployment require urgent backport or interim disable regardless of bounty scope classification.

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Severity:

Low

Recommendations

Remediation:

1. Immediately disable ICS20 and other nested-capable stateful precompiles via params or upgrade per ASA interim guidance.
2. Upgrade or backport cosmos/evm $\geq$ v0.6.0 balance-event processing (`processedEventsCount` / `MarkEventProcessed`) and related ICS20 StateDB transfer path.
3. Run recursive-precompile and BalanceHandler test suites; require recursive-precompile desync tests to pass.
4. Document dependency pin policy so Critical upstream advisories cannot ship unpatched.

Resolution:

The issue has been addressed in commit [b678fbc](#).

[F-2026-18139](#) - [Dual Defense] Cross-RPC Height Skew Can Underflow Confirmation Counts and Prematurely Finalize Inbounds - Low

Description:

The EVM and SVM event confirmers calculate confirmation depth using unchecked `uint64` subtraction. Latest-height and transaction queries are separate RPC calls, and the RPC pool round-robins each call independently. If the endpoint serving the transaction is ahead of the endpoint serving the latest height, subtracting the transaction height from the lower tip wraps to a value near 2^{64} , satisfying any realistic confirmation threshold.

This can cause a Universal Validator to vote on an inbound after only its inclusion block rather than after the intended reorg-safety depth. A single validator's premature confirmation does not mint; quorum concurrence, an actual source-chain reorg, and multi-RPC production skew must all align for unbacked minting.

Single-RPC deployments are unaffected by the round-robin skew mechanism. Premature confirmation of a deposit that remains canonical is early processing, not theft. The EVM zero-value fallback defect is an administrative configuration issue, not a permissionless attacker primitive.

Impact

The defects weaken the bridge's principal reorg defense:

- EVM/SVM endpoint skew can bypass configured numeric confirmation depth.
- EVM zero-valued configuration can silently disable depth because its intended fallback is unreachable.
- A quorum of prematurely confirming Universal Validators can finalize and execute an inbound before source finality.
- A subsequent source reorg can leave minted PRC20 supply without corresponding source-chain backing.

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

1. Before subtraction, require `latestHeight >= transactionHeight`; otherwise defer the event and log endpoint inconsistency.
2. Use a checked helper for confirmation arithmetic and add boundary tests for `latest < transaction`, `latest == transaction`, and `latest > transaction`.
3. Pin logically related latest-height and receipt/transaction reads to the same RPC backend, or fetch the canonical block containing the receipt and compare against a consistently tagged head.
4. Consider quorum or median height observations across RPC backends instead of round-robin mixing.
5. Change the EVM fallback checks from `>= 0` to `> 0`.
6. Reject zero `FastInbound` and `StandardInbound` values in registry validation unless zero is an explicitly supported, documented mode.
7. For SVM, specify commitment explicitly on every relevant query and base policy on Solana commitment/finality semantics.
8. Add end-to-end tests showing that a transaction height ahead of the observed tip remains `PENDING` and does not trigger a vote.

Resolution:

The issue has been addressed in commit [115951e](#).

[F-2026-18140](#) - [Dual Defense] Underpriced Ed25519 Raw-Message Precompile Gas - Low

Description:

`USigVerifier` at `0xEC00...0001` exposes `verifyEd25519RawMessage(bytes,bytes,bytes)` with `VerifyEd25519RawMessageGas = 4000` regardless of `message` length, while the implementation calls `ed25519.Verify(pub, message, sig)` over the entire byte slice. CPU work scales with message size; gas does not. A contract that repeatedly `STATICCALL`s the precompile with a large message can obtain disproportionate validator CPU per gas unit.

The precompile is wired into core precompiles at a fixed address; any EOA or contract can call it. Under unbounded block gas (e.g. testnet genesis helpers with `max_gas` unlimited), combined with large per-tx byte limits, a high `gasLimit` loop can pack many flat-priced verifies into one block.

Only `verifyEd25519RawMessage` is length-unbounded. Legacy `verifyEd25519` operates on a 32-byte digest with fixed hex encoding and is not similarly affected.

Impact

Flat gas underprices long-message Ed25519 verification and can elevate validator CPU relative to normal transactions. Under short consensus timeouts and heavy blocks, this can contribute to propose, prevote, or precommit timeouts. Consensus safety failure (forks, double-finality) has not been demonstrated. Disruption ends when attack transactions stop being included or parameters are tightened. No fund theft or mint occurs.

The attacker still pays gas and competes for block space. Large messages also incur calldata, memory expansion, and copy costs on each call path even though precompile `RequiredGas` is flat.

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

1. Charge gas linear in `len(message)` (and optionally account for signature and pubkey checks, which are already constant-size).
2. Hard-cap maximum message length for the precompile (revert above N KiB).
3. Set a finite consensus `max_gas` for production.
4. Revisit CometBFT timeouts relative to worst-case block CPU.
5. Add a benchmark and integration test: large-message loop must be gas-prohibitive before timeout budgets.

Resolution:

The issue has been addressed in commit [7473e48](#).

[F-2026-18141](#) - [Dual Defense] Uninitialized Genesis UEA Factory Allows First Caller to Claim Ownership - Low

Description:

On fresh genesis (`Exported == false`), `deployFactoryEA` writes Factory implementation, ProxyAdmin, and Factory proxy code and slots but never calls `Factory.initialize(address)`. ProxyAdmin's Ownable slot is set at genesis; the Factory proxy's initializer is not. An unprivileged EVM caller can be first to `initialize(attacker)` and then use owner-only Factory methods (`setUEAProxyImplementation`, `registerNewChain`, `registerUEA`, etc.).

`InitGenesis` calls `deployFactoryEA` only when `!data.Exported` in `keeper.go`. `deployFactoryEA` (`genesis.go`) deploys Factory implementation bytecode, deploys ProxyAdmin and sets storage slot 0 owner to `PROXY_ADMIN_OWNER_ADDRESS_HEX`, deploys Factory proxy and sets EIP-1967 admin and implementation slots, but performs no `CalEVM(..., "initialize", owner)`.

`FactoryV1ABI` includes `initialize(address initialOwner)` in `types/abi.go`. With OpenZeppelin-style initializer unset, the first caller wins.

Integration tests in `test/utls/contracts_setup.go` `setupFactoryContract` explicitly call `initialize(owner)` after genesis, indicating post-genesis initialization is an intended setup step omitted from `deployFactoryEA`.

If an attacker wins `initialize` before a trusted owner, they can register a malicious UEA implementation. Deterministic UEA addresses that hold native balance before first deploy can be drained when `deployUEA` DELEGATECALLs into that implementation.

Impact

On uninitialized fresh genesis, the first caller can become Factory owner. If left unfixed after claim, all future UEA deploys under that Factory are compromised. Already-deployed UEAs with an honest implementation are not affected unless redeployed under a malicious configuration. Pre-funded undeployed deterministic addresses are drainable under a compromised Factory. Chains with `Exported=true` or already-initialized Factory state are not affected.

This is a one-shot launch initialization gap. After trusted `initialize`, the initializer is consumed and the issue does not persist. A coordinated launch can include initialization in genesis, block 1, or a private mempool before public UEA funding.

Status:

Accepted

Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

1. Call `Factory.initialize(trustedOwner)` inside `deployFactoryEA` (or write Ownable/initializer storage the same way ProxyAdmin owner is set).
2. Fail genesis if `owner() == 0` after deploy.
3. Keep ProxyAdmin owner and Factory owner under governance-controlled keys.
4. Add a genesis integration test that asserts non-zero Factory owner without relying on `contracts_setup` post-init.

[F-2026-18142](#) - [Dual Defense] Fund Migration Signing Hash Uses Live Balance and Is Raceable by Dust Inflows - Low

Description:

Fund migration signing derives `SigningHash` from `maxTransfer = balance - gasCost - llFee`. When `FundMigrationData.Balance` is nil, the builder queries the live tip balance of the old TSS EOA. The verify path omits `Balance`, forcing each Universal Validator to re-query independently. A permissionless 1-wei native transfer to the public old TSS address between coordinator and follower queries can change `maxTransfer` and cause hash mismatch, aborting the signing attempt.

The on-chain comment assuming outbound-disabled implies a stable balance is incorrect: outbound disable stops outflows, not inflows. The old TSS address is derived from `OldTssPubkey` on the `FundMigrationInitiated` event and accepts unstoppable EVM-layer native transfers.

Broadcast already pins `TSSFundMigrationAmount` and documents not re-querying balance; signing and verify lack the same determinism. `FundMigrationData.Balance` exists in `universalClient/common/types.go` but verify never sets it.

Impact

- Permissionless native dust to the old TSS EOA can desynchronize signing hashes among UV participants during an admin-initiated migration window.
- Non-determinism is off-chain TSS client hash disagreement, not Push CometBFT consensus failure.
- Impact surface is the old TSS EOA native balance on the migrating EVM chain, not all vault ERC-20/PRC20 TVL.
- Migration requires admin-gated `MsgInitiateFundMigration`; outbound is already disabled for the rotation.
- Sustained interference can prolong a stuck PENDING migration; recovery via client hotfix, coordinated RPC query, chain upgrade, or admin fail/re-initiate remains possible.

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

1. Record pinned `transfer_amount` and/or `balance_block_height` in `FundMigrationInitiated`, or require followers to hash coordinator `TSSFundMigrationAmount` without re-deriving from live tip.
2. In verify: if live balance is checked, require `live >= pinnedAmount + gas + 11` only; do not recompute `maxTransfer` from live tip for the hash.
3. Set `FundMigrationData.Balance` or amount explicitly on both coordinator and follower paths.
4. Add admin `FailFundMigration` or force-remove PENDING so a stuck attempt can be re-initiated after a client fix.
5. Regression: inject +1 wei between coordinator build and follower verify; followers still accept pinned hash, or reject only if `live < pinned + fees`.

Resolution:

The issue has been addressed in commit [258061f](#).

[F-2026-18144](#) - [Dual Defense] Unchecked Cumulative GasWanted Can Fail FinalizeBlock Under Unbounded Block Gas - Low

Description:

With consensus `max_gas=-1`, `ante/types.BlockGasLimit` maps the block limit to `MaxUint64`, so a transaction declaring `MaxInt64` gas passes the per-transaction check. Fee-market ante accumulates declared gas in a uint64 transient counter without enforcing the later signed-int64 bound. Two included transactions whose declared gas sums to a value in `(MaxInt64, MaxUint64]` produce a valid uint64 total that fails `feemarket EndBlock`'s `gasWanted.IsInt64()` check. That error surfaces through `FinalizeBlock` after the block can already be decided, producing CometBFT consensus failure with application height stuck.

Gasless allowlisted messages (including vote and payload messages) remove fee friction and amplify exploitability of large gas declarations, but they are not a separate root cause. The arithmetic hole exists in inherited `feemarket EndBlock` overflow handling: unchecked uint64 accumulation versus signed int64 conversion. A finite production `max_gas`, cumulative `MaxInt64` clamp, or configuration that disables the base-fee accumulation path closes the two-transaction recipe.

Impact

- Local single-validator halt under documented conditions: decide height `H`, application stuck at `H-1`.
- Multi-validator permanent halt and restart or replay persistence are not demonstrated.
- No fund theft, mint, or state corruption; impact is liveness desync between blockstore and application height.
- Primary trigger is consensus-parameter choice (`max_gas=-1`) combined with the base-fee-enabled accumulation path.
- Gasless messages amplify zero-fee broadcast of `MaxInt64` gas declarations.
- Finite block gas rejects `MaxInt64` per transaction at admission; `EndBlock` clamp instead of hard error further mitigates.

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Severity:

Low

Recommendations

Remediation:

1. Never ship public networks with `max_gas=-1`; set a finite block gas limit and document the requirement.
2. In `AddTransientGasWanted` or `CheckGasWanted`, reject if cumulative exceeds `MaxInt64` or `BlockGasLimit` when the limit is not `MaxUint64`.
3. Reject `feeTx.GetGas() > MaxInt64` per transaction even when the block limit is `MaxUint64`.
4. In `EndBlock`, clamp `gasWanted` to `MaxInt64` for base-fee math instead of returning an error that fails `FinalizeBlock`.
5. Enforce `sum(gasWanted) ≤ min(blockLimit, MaxInt64)` in `PrepareProposal` and `ProcessProposal`.
6. Cap declared gas on gasless messages independently of fees.
7. Regression: two gasless transactions with `MaxInt64` plus a second positive declaration must fail `CheckTx` or `ProcessProposal`, never `FinalizeBlock`.

Resolution:

The issue has been addressed in commit [a5aead7](#) and commit [3e63a07](#).

[F-2026-18145](#) - [Dual Defense] Missing OP-Stack L1 Data-Fee Accounting Under-Reports Outbound Cost - Low

Description:

On OP-Stack destinations (e.g. configured Base Sepolia [eip155:84532](#)), the destination sender pays L2 execution gas plus an L1 data/security fee. The universal client's outbound settlement paths persist only `receipt.GasUsed * tx.GasPrice()` into `GasFeeUsed`. `x/uexecutor` `applyGasRefund` then refunds `GasFee - GasFeeUsed`, treating the omitted L1 component as unused user gas. Fund-migration code already documents `L1GasFee` for OP-stack DA charges; normal outbounds do not.

On the success path, `universalClient/chains/evm/event_confirmer.go` sets `gasFeeUsed := new(big.Int).Mul(gasUsed, gasPrice).String()` with no OP receipt `l1Fee` or `GasPriceOracle` total. On revert/resolver path, `TxBUILDER.GetGasFeeUsed` returns only `gasUsed * gasPrice` on a generic go-ethereum receipt. `applyGasRefund` computes `refundAmount = gasFee - gasFeeUsed`; if `GasFeeUsed` omits `l1Fee`, refunds are too large by approximately that amount. Outbound calldata includes user `Payload`; on OP Stack, L1 fee scales with serialized transaction size.

Impact

L2-only cost is under-reported on OP destinations. When `GasFee > gasUsed*gasPrice`, users receive over-refunds relative to true destination cost. TSS/relayer native balance absorbs the omitted L1 fee for included transactions.

No direct theft of vault bridged ERC-20 or ETH principal has been shown; impact is relayer gas accounting. Effect is OP/Base-class destination specific, not universal across all configured chains. Operators can disable outbound per chain, recapitalize TSS balance, rate-limit oversized payloads, and alert on TSS balance. Each outbound requires gateway creation, UV ballot/TSS signing, and destination inclusion.

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

1. For OP-Stack chains, set `GasFeeUsed = executionFee + l1Fee` (parse OP receipt fields or oracle).
2. Include L1 upper bound in pre-outbound quotes (`GasPriceOracle.getL1Fee` or upper bound).
3. Cap outbound payload size for OP destinations.
4. Share the same total-cost helper used by fund migration.
5. Regression: receipt with nonzero `l1Fee` must reduce refund by that amount.

Resolution:

The issue has been addressed in commit [d28d095](#).

[F-2026-18146](#) - [Dual Defense] Oversized Gateway Payload Blinds Shipped Outbound Poll via gRPC Size Limit - Low

Description:

UniversalGatewayPC admits an unbounded dynamic payload. UExecutor hex-encodes it (`0x` plus two characters per byte) into `OutboundTx.Payload` and indexes it in `PendingOutbounds`. The shipped Universal Client polls `AllPendingOutbounds` over grpc-go's default 4 MiB receive limit with no `MaxRecvMsgSize` override. A single pending object whose serialized query response exceeds 4 MiB makes every poll fail with `ResourceExhausted`, so no pending outbound reaches the signer.

Pagination is by entry count, not byte budget; one oversized object on the first page makes the entire RPC unusable. Round-robin endpoints serve the same consensus pending set. An amount-zero oversized payload avoids PRC20 principal burn but still costs substantial gas and fees. If positive-amount outbounds are already pending, PRC20 is burned on Push while signers cannot fetch or sign; shipped user remind paths remain closed while poll is blind.

Impact

- Shipped Universal Client cannot poll pending outbounds when one oversized payload exceeds 4 MiB serialized response.
- Push block production continues; this is off-chain poll failure, not chain halt.
- An oversized-only case produces outbound signing availability impact for shipped clients.
- Co-pending funded burns can strand principal until privileged recovery.
- No theft; not absolute irreversibility given operator remediation paths.
- Sweeper drops local rows and re-pulls from consensus; does not auto-clear consensus pending.

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

1. Hard-cap payload length at gateway emit and at `BuildOutboundsFromReceipt`; reject or skip oversized entries.
2. `AllPendingOutbounds` must not embed full payload; return metadata plus hash; fetch body via `GetPendingOutbound` or chunked API with byte budgets.
3. Set `grpc.MaxCallRecvMsgSize` well above expected max, or fail soft per item without aborting the whole poll.
4. Reject creating `PendingOutbounds` when encoded outbound protobuf size exceeds configured maximum.
5. Ops runbook: pause outbound on detection; delete or quarantine the oversized pending entry; manually fail or remind funded siblings.
6. Regression: insert 2,097,153-byte payload outbound; client still processes other pendings, or admission rejects the oversized payload.

Resolution:

The issue has been addressed in commit [5f0341e](#).

[F-2026-18147](#) - [Dual Defense] RecomputeBallotQuorum Does Not Finalize When Preserved Votes Already Meet Threshold - Low

Description:

`RecomputeBallotQuorum` rebuilds eligible voters and threshold, preserves still-eligible votes, writes the ballot, and always returns `BALLOT_STATUS_PENDING` (unless zero eligible voters, in which case `EXPIRED`). It never calls `IsFinalizingVote` or `CheckIfFinalizingVote`. If preserved YES votes already satisfy the new lower threshold and every remaining eligible voter has already voted, no further `AddVote` can trigger finalization, and the ballot can remain `PENDING` with no UV re-vote path.

`x/validator/keeper/ballot.go` `RecomputeBallotQuorum` persists the ballot and returns `BALLOT_STATUS_PENDING` without `CheckIfFinalizingVote`. The implementation assumes downstream UVs must re-vote, which fails when all remaining eligibles have already voted.

`VoteOnBallot` always calls `CheckIfFinalizingVote` after `AddVote`, which may `MarkBallotFinalized`. `VoteInbound` creates the UTX and runs `ExecuteInbound` only when `isFinalized` is true from that vote. `BallotHooks.AfterBallotTerminal` for `PASSED` assumes the `VoteInbound` path already produced a UTX and does not mint or execute on its own.

`RevertStuckInbound` rejects unless ballot status is `EXPIRED`. Recompute only auto-expires on zero eligible voters. `DefaultExpiryAfterBlocks = 100_000_000` makes natural expiry impractical as recovery.

Impact

Recompute can leave a finalizable ballot in `PENDING` status when remaining eligibles cannot re-vote. Only `params.Admin` can invoke `MsgRecomputeBallotQuorum`; unprivileged parties cannot force this state. The ballot is already stuck before recompute is attempted; recompute is incomplete remediation, not the root lock.

Recovery remains available: add a new eligible UV and recompute (new voter is `NOT_YET_VOTED`, enabling normal `VoteOnBallot` finalization), chain upgrade, or a follow-up admin message. Marking `PASSED` alone would not execute inbound without `VoteInbound` wiring. Funds are delayed in gateway or pending observation, not stolen.

Status:

Fixed

Classification

Impact Rate:

1/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

1. After rebuilding votes and threshold in `RecomputeBallotQuorum`, call `CheckIfFinalizingVote` (or equivalent) before returning.
2. If the ballot becomes PASSED, also run the inbound (or outbound) post-finalization pipeline that `VoteInbound` / outbound vote would have run; do not rely on hooks alone.
3. Extend `RevertStuckInbound` (or a sibling admin msg) to accept PENDING ballots proven unreachable (e.g. yes votes already $\geq$ threshold but no UTX), not only EXPIRED.
4. Integration test: 6/10 votes $\rightarrow$ shrink to 6 eligibles / threshold 5 $\rightarrow$ recompute $\rightarrow$ ballot PASSED and UTX created/executed (or explicit revert scheduled).
5. Consider shorter ballot expiry or an admin force-expire for stuck PENDING.

Resolution: The issue has been addressed in commit [9a3a536](#).

[F-2026-18177](#) - [Dual Defense] User-Selected Rescue PRC20 Can Reinterpret a Stuck Inbound as Another Vault Asset - Low

Description:

`UniversalGatewayPC.rescueFundsOnSourceChain` emits a caller-selected PRC20 address. When Push Chain processes that event, it resolves the corresponding external token but takes the amount from the referenced original inbound. The keeper never verifies that the selected PRC20 represents the original inbound asset.

The resulting rescue outbound can combine the raw amount from original inbound asset A with the external counterpart of caller-selected PRC20 B. The gateway contract only rejects the zero PRC20 address and does not bind `prc20` to the original universal transaction's asset.

Rescue eligibility checks are meaningful: for CEA inbounds, the first Push Chain transaction must have status `FAILED`; for non-CEA inbounds, an `INBOUND_REVERT` outbound must exist with status `REVERTED`. These checks do not compare the event's PRC20 with the original inbound asset. The practical attacker is most plausibly the owner or revert recipient of a genuinely rescue-eligible universal transaction, not an arbitrary third party.

Impact

An eligible user can select a different registered PRC20 and cause the rescue flow to withdraw the corresponding source-chain asset using the original inbound's raw amount. Consequences include:

- Withdrawal of more value than the user's stuck deposit when decimal semantics differ.
- Cross-asset depletion of shared Vault liquidity.
- Amplification when moving from a high-decimal, low-value asset amount to a lower-decimal valuable asset.
- Losses borne by other depositors or protocol reserves, capped by Vault liquidity for the selected asset.

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

1. Derive the rescue PRC20 and external asset exclusively from `originalUtx.InboundTx.AssetAddr`; do not trust `event.PRC20` for asset selection.
2. Resolve the original external asset to its registered token configuration and require canonical equality between `tokenCfg.Address`, `originalUtx.InboundTx.AssetAddr`, and `tokenCfg.NativeRepresentation.ContractAddress` with `event.PRC20`.
3. Reject rescue requests for original assets that have no registered configuration rather than allowing caller substitution.
4. Include the original asset and amount in the Push Chain rescue request or verify them against immutable UTX state before emitting an outbound.
5. Add source-chain deposit accounting keyed by `universalTxId`, and require Vault/Gateway rescue execution to match the recorded token and amount.
6. Add negative tests where the event selects a different registered PRC20, including 18-to-6-decimal pairs.
7. Review existing rescue outbounds and pause manual rescue processing until the invariant is enforced.

Resolution:

The issue has been addressed in commit [aa5ad0e](#).

[F-2026-18182](#) - [Dual Defense] Unbounded UEA Payload Gas Can Stall Synchronous EVM Execution - Low

Description:

`MsgExecutePayload` allows a user-controlled `UniversalPayload.GasLimit` to become the gas budget of a synchronous derived EVM call without being clamped to the default EVM gas cap, the Cosmos transaction gas limit, or the consensus block gas limit. An attacker who controls a valid, deployed UEA can authorize a payload targeting a gas-consuming contract and select an extremely large payload gas limit.

The EVM interpreter receives that budget directly and can remain inside the call for an impractically long period. Cosmos SDK gas is charged only after the EVM returns, so the SDK transaction and block gas meters do not provide an execution-time interruption. The transaction can carry no Cosmos fee because `MsgExecutePayload` is gasless, while still declaring a modest Cosmos gas limit separate from the embedded EVM gas limit.

A defensible attack uses an attacker-controlled UEA with valid authorization rather than arbitrary execution through a victim UEA. Normal broadcasting may stall in `CheckTx` before an honest proposer includes the transaction. EVM work is finite for any finite `uint64` budget.

At minimum, the flaw permits attacker-selected, disproportionately expensive EVM work through a zero-fee Cosmos message. Depending on transaction delivery and node architecture, this can:

- Stall transaction admission on targeted nodes.
- Exhaust application execution workers or block an ABCI request for a prolonged period.
- Prevent a validator from completing a block if the transaction reaches `FinalizeBlock`.
- Threaten chain liveness if sufficient voting power evaluates the same included transaction.

The computation is ultimately finite for a finite `uint64` gas limit. With a budget such as `1018`, completion can be operationally indistinguishable from a prolonged hang. No fund theft occurs; impact is availability and resource abuse.

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Severity:

Low

Recommendations**Remediation:**

1. Reject payload gas limits that do not fit in `uint64`; do not use truncating `big.Int.Uint64()` without an `IsUint64()` check.
2. Define a protocol-level maximum for UEA payload execution gas.
3. Clamp the derived gas limit to the smallest applicable bound, including the protocol cap, remaining SDK transaction gas, and remaining block gas.
4. Reserve and consume block gas before entering the EVM rather than charging the SDK meter only after return.
5. Perform an up-front UEA balance check against the maximum payable gas cost, while retaining post-execution settlement.
6. Reconsider the gasless allowlist entry for `MsgExecutePayload`, or apply strict rate and resource controls independent of fees.
7. Add regression tests asserting that oversized payload gas limits fail before `ApplyMessageWithConfig`.

Resolution:

The issue has been addressed in commit [a726345](#).

[F-2026-18184](#) - [Dual Defense] Empty SVM CEA Recipient Encodes to Invalid Address and Strands Outbound - Low

Description:

Gateway documentation defines `bytes("")` as parking funds in the caller's destination CEA. For positive-amount SVM outbounds, the gateway burns PRC20 and emits empty recipient bytes. UExecutor hex-encodes those bytes to `"0x"` and stores a PENDING outbound. The Universal Client SVM builder requires a base58 public key or 32-byte hex recipient and rejects `"0x"` before any TSS signing request is created.

Failure occurs pre-sign, so no broadcast, observation, or automatic remind follows. Local client events remain CONFIRMED; the sweeper drops local rows and re-pulls the same pending entry from consensus. Resolver failure votes target BROADCASTED paths that are never reached. Public rescue and `RevertStuckInbound` do not apply to PC-origin UTXs where `InboundTx` is nil.

This is a documented API and consumer mismatch that strands user funds on a self-directed path. A bonded UV quorum can manually fail-observe and remind; an upgrade can normalize the CEA sentinel or reject empty recipients before burn.

Empty recipient bytes decode to `"0x"` and are persisted as an SVM outbound recipient without validation.

- SVM builder rejects `"0x"` pre-sign; no TSS setup, destination transaction, or receipt is produced.
- Pending outbound remains on Push indefinitely under shipped automatic lifecycle transitions.
- PC-origin UTXs cannot use public rescue; `RevertStuckInbound` is inbound-ballot only.
- Caller burns their own PRC20 on a documented CEA-empty composition; no third-party theft.
- Privileged UV failure observation and remind remain available outside shipped user paths.

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

1. Reject empty recipient before burn at gateway or keeper, or normalize empty bytes to the caller's destination CEA pubkey per documentation.
2. Never persist `"0x"` as an SVM recipient; fail admission if recipient is not valid base58 or 32-byte hex.
3. On deterministic builder rejection, vote outbound failure so `handleFailedOutbound` can remint.
4. Add consensus-level signing deadline expiry for outbounds stuck pre-sign, not only local sweeper cleanup.
5. Regression: `bytes("")` SVM positive-amount outbound either parks correctly in CEA or remints; must not remain PENDING forever under shipped clients.

Resolution:

The issue has been addressed in commit [2b07bcc](#).

[F-2026-18188](#) - [Dual Defense] IBC Timeout Callback Invokes CalEVM on Outer Context Instead of Cached Context - Low

Description:

`IBCOnTimeoutPacketCallback` installs an infinite limited gas meter on `cachedCtx`, then executes `onPacketTimeout` via `CalEVM(ctx, ...)` on the outer context and still calls `writeFn()`. The acknowledgement path correctly uses `CalEVM(cachedCtx, ...)`. This asymmetry is a copy-paste defect; the gas consumption log still references `onPacketAcknowledgement`.

`CalEVMWithData` creates an internal cache from the passed context and calls `commitState()` on success, writing into that parent. When the outer context is passed, successful callback storage and transfers commit to the outer store. The defect is wrong gas meter and cache discipline versus the acknowledgement path, not automatic rollback of all `onPacketTimeout` state.

ICS-20 escrow refund on timeout is independent and succeeds. In-tree middleware timeout success tests expect `getCounter() == -1` after `onPacketTimeout`, indicating persisted callback side effects on this pin.

Impact

- Timeout callback executes under the wrong context relative to the installed gas meter on `cachedCtx`.
- Escrow refund to the packet sender succeeds independently of callback behavior.
- Successful `CalEVM` writes commit to the outer context; callback effects are not universally discarded.
- Permanent freeze of refunded funds is not established without a confirmed persistence failure under sufficient gas budget.
- Missing alternate withdraw in example contracts amplifies impact only when callback-side release fails to persist.

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

1. Change `CallEVM(ctx, ...)` to `CallEVM(cachedCtx, ...)` in the timeout callback.
2. Fix the gas consumption reason string to `onPacketTimeout`.
3. Add regression tests ensuring ERC-20 forward and storage in `onPacketTimeout` persist under the `cachedCtx` gas meter.

Resolution:

The issue has been addressed in commit [0076392](#).

[F-2026-18189](#) - [Dual Defense] Manual Module EVM Nonce Desync on Reverted Inbound Execution - Low

Description:

`ModuleAccountNonce` is incremented on the outer Cosmos context before `DerivedEVMCall`, while the EVM state-DB nonce advances only when `commit && !res.Failed()`. Inbound executors swallow the EVM error and return `nil`, so a reverted module-sender call can leave `ModuleAccountNonce = N+1` while the account nonce in the EVM DB remains `N`. Later module-sender calls supply the manual nonce `N+1` with `SkipNonceChecks=false` and fail.

Module-sender helpers in `x/uexecutor/keeper/evm.go` follow: `nonce := GetModuleAccountNonce(ctx)`, `IncrementModuleAccountNonce(ctx)`, then `DerivedEVMCall(..., commit=true, isModuleSender=true, &nonce, ...)`. This pattern appears in `CallPRC20Deposit`, `CallPRC20DepositAutoSwap`, `CallUniversalCoreSetChainMeta`, `CallUniversalCoreRefundUnusedGas`, and factory deploy helpers.

In `push-chain-evm DerivedEVMCallWithData`, `SkipNonceChecks: !commit` means committed module transactions are nonce-checked against the state DB. Inner work runs in `CacheContext`; `commitState()` runs only if `commit && !res.Failed()`. On revert, the function returns `(res, ErrVMExecution)` without committing inner state, discarding the state-DB nonce increment while the Go counter has already moved.

After ballot finalization, `CreateUniversalTx` / `ExecuteInbound` run on the outer committed context. `ExecuteInboundFunds` / `ExecuteInboundGas` log deposit or swap failures and still `return nil`, keeping the bumped `ModuleAccountNonce`. `SetModuleAccountNonce` exists but is only used from genesis import in the reviewed snapshot, with no runtime Msg to resync.

Impact

Once desynced, subsequent module-sender `DerivedEVMCall`s that pass a manual nonce can fail nonce checks, blocking PRC20 deposit and auto-swap inbound credit paths, module-driven UEA factory deploys on those paths, and `setChainMeta` and other module-sender maintenance calls.

This does not automatically halt CometBFT consensus, stop all EVM user transactions (`MsgExecutePayload` uses the user/UEA sender path), or make recovery impossible. An upgrade can set `ModuleAccountNonce` equal to the EVM account nonce via the existing keeper API. Funds in external gateways for future inbounds may fail to credit until fixed.

Many GAS-path failures (`GetSwapQuote`, config lookups) abort before `IncrementModuleAccountNonce`. Plain `TxType_FUNDS` uses `depositPRC20Token`

(mint/deposit), not Uniswap auto-swap. UV quorum is still required to finalize the triggering inbound.

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

1. Prefer EVM account nonce as source of truth immediately before each module `DerivedEVMCall`, or
2. Only persist `IncrementModuleAccountNonce` when the call returns committed success; roll back or never write on `ErrVMExecution` / `res.Failed()`.
3. Add an invariant or admin/upgrade reconciliation: `ModuleAccountNonce == evm.GetNonce(moduleAcc)`.
4. Regression test: force a reverted `depositPRC20WithAutoSwap` or `depositPRC20Token`, then assert a subsequent module call still succeeds or that the counter did not advance.

Resolution:

The issue has been addressed in commit [a42effa](#).

[F-2026-18190](#) - [Dual Defense] Obsolete TSS Keyshares Survive Quorum Change Without Deletion or Retirement - Low

Description:

Quorum change correctly refreshes keyshares for the new participant set and preserves the TSS public key by DKLS design. `handleKeyFinished` persists the new share under a SHA-256 event-derived `storageID` and votes that storage ID on-chain, but never deletes or cryptographically retires the predecessor key ID from disk. `keyshare.Manager` exposes only `Store`, `Get`, and `Exists` with no deletion or retirement API. Predecessor shares therefore remain usable offline for the same vault-controlling public key at the old threshold.

On a $4 \rightarrow 3$ removal, the old threshold is 3-of-4 and the new threshold is 3-of-3. An adversary who obtains the departed party's old share plus old shares from two of three continuing parties can DKLS-sign for the current vault key without holding a full current 3-of-3 set of new shares. Host count for the old-path drain equals host count for compromising the full current new quorum; the differential is which hosts (departed plus two continuing versus all three continuing), not fewer machines.

Exploitation requires extracting encrypted keyshare material from multiple universal-validator hosts. It is not permissionless remote vault drain or online sub-quorum threshold break.

Impact

- Missing key lifecycle hygiene after an otherwise honest quorum change.
- Offline multi-host keyshare compromise can authorize vault moves using archived predecessor shares.
- Departed-party wipe alone is insufficient; continuing nodes retain old key IDs on disk.
- Public key stability is DKLS-intended; indefinite retention of predecessor files is not required and contradicts lifecycle claims that the departed party was discarded.
- Distinct from malicious coordinator setup-hash forgery and malicious threshold downgrade findings.

Status:

Fixed

Classification

Impact Rate: 3/5

Likelihood Rate: 1/5

Severity:

Low

Recommendations**Remediation:**

1. Add `keyshare.Manager.Delete` / `Retire` with secure overwrite and unlink.
2. On successful quorum-change or key-refresh finalization, all continuing parties delete the predecessor `oldKeyID` after the new share is stored and voted; document departed-host wipe requirements.
3. Optionally bind on-chain "current key id" so clients refuse non-current IDs for signing (defense in depth; offline DKLS with raw files still requires file deletion).
4. Long-term: consider pubkey-rotating migration if the product requires cryptographic retirement beyond file wipe.
5. Regression: after quorum change, `Exists(oldKeyID) == false` on all participants; offline sign with only old shares from a departed-plus-subset fixture must fail in CI policy tests.

Resolution:

The issue has been addressed in commit [8af9cd29](#)

[F-2026-18784](#) - [Dual Defense] Uppercase Bech32 Fee Payer Bypasses DIRECT_AUX Fee-Payer Guard - Low

Description:

Push enables `SIGN_MODE_DIRECT_AUX` via `tx.DefaultSignModes` (plus TEXTUAL) in `app/app.go`. The active DIRECT_AUX handler comes from `cosmossdk.io/x/tx` v0.13.3 (go.mod replace)

That upstream handler rejects a fee payer who also uses DIRECT_AUX with a case-sensitive string compare:

```
// cosmossdk.io/x/tx/signing/directaux/direct_aux.go
if feePayer == signerData.Address {
    return nil, fmt.Errorf("fee payer %s cannot sign with %s: unauthorized", ...)
}
```

BIP-173 allows an all-upercase Bech32 encoding of the same account bytes. If `Fee.Payer` is the uppercase alias of the victim while `signerData.Address` is the canonical lowercase form, the guard fails open. Downstream validation still decodes both to the same `AccAddress` bytes and deduplicates signers, so no separate fee-payer DIRECT signature is required. A malicious fee sponsor who already holds a victim DIRECT_AUX signature over a fixed `TxBody` can rewrite `AuthInfo` (raise fee, set uppercase payer, drop sponsor signer) and cause unauthorized native-PC fee deduction from the victim.

This is inherited upstream signing logic. Push does not implement or fork the DIRECT_AUX fee-payer check. The older `cosmos-sdk/x/auth/tx` DIRECT_AUX path uses `strings.EqualFold` on addresses and would not be bypassed the same way—Push's protobuf tx stack uses `cosmossdk.io/x/tx`, which still uses raw `==`.

Impact

- Mechanism (case-sensitive fee-payer check vs Bech32 case alias) is plausible on the enabled DIRECT_AUX surface.
- Requires the victim to produce a still-valid DIRECT_AUX signature and hand the sponsored flow to a malicious aggregator or sponsor.
- Unauthorized fee is paid to the fee collector / distribution, not transferred to the attacker's wallet; the attacker's direct gain is mainly avoiding their own fee (griefing / burn of victim PC).
- Not Push business logic (uexecutor / TSS / gateway); same class as other upstream SDK advisories.
- Remediate by upgrading `cosmossdk.io/x/tx` (compare decoded `AccAddress` bytes or `EqualFold`) or disabling DIRECT_AUX until patched.

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

1. Upgrade `cosmosdk.io/x/tx` once upstream compares fee payer and signer by decoded bytes (or case-insensitive Bech32).
2. Interim: remove `SIGN_MODE_DIRECT_AUX` from `EnabledSignModes` if fee-sponsored AUX is unused.

Resolution:

The issue has been addressed in commit [2f161b](#).

[F-2026-18785](#) - [Dual Defense] RevertMultiStore EventManager / writeCache Desync - Low

Description:

Stateful precompiles snapshot MultiStore + `cacheCtx` events in `AddPrecompileFn`, then on journal revert call `RevertMultiStore(snapshot, events)`:

```
func (s *StateDB) RevertMultiStore(snapshot int, events sdk.Events)
{
    s.snapshotter.RevertToSnapshot(snapshot)
    s.writeCache = func() {
        s.ctx.EventManager().EmitEvents(events) // static pre-call slice
        s.cacheCtx.MultiStore().(storetypes.CacheMultiStore).Write()
    }
}
```

The initial `cache()` `writeCache` dynamically flushes

`s.cacheCtx.EventManager().Events()`. After any precompile-frame revert, that closure is replaced and never restored (`writeCache` is only assigned in `cache()` and `RevertMultiStore`). Consequences:

1. **Swallowed post-revert events:** Successful precompiles after a caught revert still mutate MultiStore and append to `cacheCtx` EventManager, but Commit emits only the frozen snapshot slice—later Cosmos events (including ICS20/`SendPacket`-class emissions) can be dropped while store writes commit.
2. **Stale-slice re-emit:** `RevertMultiStore` does not truncate or `OverrideEvents` on `cacheCtx`; a later reverting precompile can snapshot a slice that still contains prior reverted emissions and hardcode that polluted slice into `writeCache`.

Both behaviors are inherited **cosmos/evm** StateDB/precompile plumbing, not Push-module business logic. Upstream **cosmos/evm v0.6.0** removes the static overwrite: `RevertMultiStore` only rolls MultiStore;

`precompileCallChange.Revert` restores events via `EventManager().OverrideEvents(pc.prevEvents)`; `writeCache` remains the live EventManager flush.

Exploit shape is a delivering EVM tx that reverts a precompile frame (e.g. Solidity try/catch) then continues. Packet commitments and escrow live in MultiStore; missing events impair event-driven relayers, not the existence of the commitment. ICS20 timeouts remain the standard unlock path when a packet is never relayed. Forged `CoinSpent` / deposit-style events after MultiStore revert do not recreate bank balances; impact is off-chain indexer trust of events without state checks—not on-chain mint or vault drain.

Impact

- `RevertMultiStore` replaces dynamic `writeCache` with a static event slice; no restore path on affected pins.
- `cacheCtx` EventManager is not cleared on MultiStore revert → stale events can be re-captured.
- upstream v0.6.0 uses `OverrideEvents` + non-overwriting `RevertMultiStore`.
- Exploit requires self- or dApp-triggered composition; store-side packet/escrow remain; timeout refund path exists.
- Residual if forced in-scope: event-integrity / relayer-liveness hygiene for try/catch + stateful precompile txs (ICS20 active at `0x...0802`).

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

1. Backport cosmos/evm ≥ v0.6.0 StateDB precompile revert path: do not replace `writeCache` with a static event slice; restore `cacheCtx` events via `OverrideEvents` (or equivalent truncate/replace).
2. Keep `writeCache` as a live flush of `cacheCtx.EventManager().Events()` + MultiStore `Write()`.
3. Interim: disable ICS20 / high-risk stateful precompiles if upgrade lags.
4. Regression: try/catch revert of precompile A then successful ICS20/bank precompile B must emit B's Cosmos events and must not emit A's reverted events.

Resolution:

The issue has been addressed in commit [b678fbc](#).

[F-2026-18786](#) - [Dual Defense] Unknown Selector Still Runs CommitWithCacheCtx Before ABI Fail - Low

Description:

Stateful precompiles such as bank return `RequiredGas == 0` when calldata is short or `MethodById` fails, yet `Run` still enters `runNativeAction`. That wrapper always:

1. `GetCacheContext` / `MultiStoreSnapshot` / `AddPrecompileFn`
2. `CommitWithCacheCtx()` (replays dirty accounts/storage into `cacheCtx`)
3. runs `action(ctx)` → `SetupABI` fails on the bad selector
4. returns on error **before** `contract.UseGas(cost, ...)`

So the failure path does native snapshot/commit work without the success-path precompile `UseGas` charge.

This is inherited **cosmos/evm** stateful-precompile infrastructure (`precompiles/common/precompile.go` + `StateDB.CommitWithCacheCtx`), not Push UExecutor/TSS business logic.

Mitigating factors:

1. Callers still pay ordinary CALL/STATICCALL (and calldata) gas; the gap is missing precompile-side gas for wrapper work.
2. Dirty-set size is prepaid via SSTORE (and related) gas before the cheap bad-selector calls.
3. `MaxPrecompileCalls` (20) caps how many times `AddPrecompileFn` can run per tx.
4. Failed action does not keep precompile business-state writes for the unknown-selector path (ABI fails before bank/staking handlers); impact is validator CPU vs gas mismatch / temporary liveness pressure, not mint or vault drain.

Impact

- unknown selector → `RequiredGas 0` → full `runNativeAction` preamble including `CommitWithCacheCtx` → error before `UseGas`.
- Callers still pay CALL + dirtying gas; ≤20 precompile frames; no demonstrated fund theft or consensus halt.
- Residual if forced in-scope: gas/CPU underpricing grief, not custody.

Status:

Fixed

Classification

Impact Rate:

1/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

1. Upstream: resolve ABI / reject unknown selectors before `CommitWithCacheCtx`, or charge a floor cost covering snapshot+commit even on failure.
2. Always account preamble work on the error path (`UseGas` or consume against contract gas before return).
3. Avoid `RequiredGas == 0` for inputs that still enter `Run` (minimum non-zero).
4. Backport cosmos/evm gas-alignment fixes; interim: monitor pathological patterns if needed.

Resolution: The issue has been addressed in commit [2e5db83](#).

[F-2026-18793](#) - [Dual Defense] RecomputeBallotQuorum Applies 2/3+1 Even to TSS Ballots - Low

Description:

`RecomputeBallotQuorum` always sets `newThreshold = (2*newEligibleCount)/3 + 1` and does not branch on `BallotObservationType`. TSS key ballots are created in `VoteOnTssBallot` with `votesNeeded = len(existing.Participants)` (100% of process participants). An admin `MsgRecomputeBallotQuorum` can therefore overwrite a TSS ballot's 100% threshold with 2/3+1 and rebuild `EligibleVoters` from live `GetEligibleVoters()` rather than the TSS process participant list.

That type-blind formula is a escape-hatch defect:

1. Not automatic on validator-set change. `MsgRecomputeBallotQuorum` requires `params.Admin` and an explicit `ballot_id`. No status-update / join / leave path calls `RecomputeBallotQuorum`.
2. No auto-finalization. Recompute persists the ballot and returns `PENDING`; it never calls `CheckIfFinalizingVote`. Doc comment: downstream UVs must re-vote.
3. Already-voted UVs cannot re-vote (`Ballot.AddVote` → `HasVoted`). With 4 SUCCESS votes and threshold lowered to 4, finalization still needs a later `VoteOnTssBallot` from a `NOT_YET` eligible.
4. On-chain TSS ballot ≠ DKLS key material. Finalization stores an observed `(tssPubKey, keyId)` after quorum of observation votes; it does not mint vault-signing power from an incomplete off-chain DKLS run.

Impact

- Admin can lower a pending TSS ballot's observation threshold from N to $\lfloor 2N/3 \rfloor + 1$ and rewrite eligibles to the live UV set.
- Routine set change does not auto-finalize TSS key observation.
- No permissionless exploit; no demonstrated vault drain from threshold rewrite alone.

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

1. For `BALLOT_OBSERVATION_TYPE_TSS_KEY`, either refuse recompute or set `votesNeeded = len(preservedTSSParticipants)` (process participants), not $2/3+1$ of live UVs.
2. Do not replace TSS eligibles with `GetEligibleVoters()` unless the process participant set is intentionally updated.
3. Regression: admin recompute on a TSS ballot with 4/5 votes must not finalize and must not set threshold to 4.

Resolution:

The issue has been addressed in commit [a8a997e](#).

[F-2026-18795](#) - [Dual Defense] Unbounded Active-Ballot Expiry Scan on Create + Disabled 100M Expiry - Low

Description:

`CreateBallot` always calls `ExpireBallotsBeforeHeight`, which iterates the entire `ActiveBallotIDs` set and `Ballots.Get`s each entry. `DefaultExpiryAfterBlocks = 100_000_000` is explicitly documented as disabling practical expiry until pruning/escape-hatch work lands, so the scan normally expires nothing while still charging $O(|\text{Active}|)$ store work on new ballot creation. There is no `MaxActiveBallots` cap.

That combination is create-path cost debt:

1. Not every inbound vote. `ExpireBallotsBeforeHeight` runs only inside `CreateBallot`. `GetOrCreateBallot` short-circuits when the ballot already exists, so follow-on votes on the same key skip the full scan.
2. SDK gas bounds the work. `ActiveBallotIDs.Iterate` / `Ballots.Get` are gas-metered. Growing `|Active|` makes the next create more expensive; cumulative create cost is $\sim O(A^2)$ gas across the set's lifetime, so the flood is rate-limited by declared `gasWanted` / block gas. Failed creates $\neq$ `FinalizeBlock` abort.
3. Honest quorum shrinks the set. On 2/3 finalization, `MarkBallotFinalized` removes the id. Uniform UV observations of dust deposits clear; steady-state size tracks in-flight depth, not a permanent ratchet.
4. Ballot creation is UV-gated. `MsgVoteInbound` requires a bonded UV. Permanent never-quorum dead weight needs UV-signed divergent keys—not pure external spam.
5. Not chain-wide fund lock. Inbound-create friction leaves bank/EVM/other DeliverTx paths producing blocks. Recovery: upgrade prune, budgeted EndBlock expiry, drop create-time full scan.

Impact

- every new ballot creation walks all active ids; 100M expiry makes the walk a pure tax in practice.
- no active-set cap; gasless `MsgVoteInbound` means UV fee pressure does not throttle creates.
- Practical impact: growing `|ActiveBallotIDs|` raises gas for **new** ballot creates → inbound throughput degrades; minority/stuck ballots linger until upgrade/prune.

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

1. Do not full-scan active ballots on every `CreateBallot`; maintain an expiry index / priority queue, or run budgeted expiry in `EndBlock` with a per-block cap.
2. Restore a realistic `DefaultExpiryAfterBlocks` once `MsgRevertStuckInbound` / pruning covers stuck pending, or expire into a recoverable terminal state.
3. Cap `ActiveBallotIDs` (reject/create-rate-limit) and/or bound gasless vote `gasWanted`.
4. Optional: require observation attestation / rate-limit first-vote creates per UV per block.

Resolution:

The issue has been addressed in commit [64b8aff](#).

[F-2026-18796](#) - [Dual Defense] Outbound Ballot Key Hashes Client GasFeeUsed / ErrorMsg Strings - Low

Description:

`GetOutboundBallotKey` SHA-256-hashes the full `OutboundObservation`, including `GasFeeUsed` and `ErrorMsg` verbatim (after `TrimSpace` only). `TxHash` is chain-canonicalized at vote ingress; those two strings are not. Quorum is per `ballotKey` at $(2N)/3+1$. `DefaultExpiryAfterBlocks = 100_000_000` effectively disables expiry; there is `MsgRevertStuckInbound` but no outbound equivalent; ballot hooks skip outbound terminalization. `RecordOutboundVote` already records per-variant audit trails when observations diverge.

Non-canonical strings in the digest are hygiene debt:

1. Homogeneous official clients agree today. EVM success uses the same `gasUsed * tx.GasPrice()` formula; failure `ErrorMsg` values are fixed literals. Same-version UVs produce identical keys for the same dest outcome.
2. Fracture needs an uncoordinated mid-upgrade split that leaves no variant $\geq 2/3+1$. That is ops upgrade risk (runbook / dual-vote window), not a permissionless or always-on custody bug.
3. Recovery already exists: `purge-expired-outbounds` and gov/upgrade clear stuck pendings; outbound pause/remint remain ops levers.

Impact

- `GasFeeUsed` / `ErrorMsg` enter the outbound ballot digest; only whitespace trimmed.
- long default expiry; no `RevertStuckOutbound`; variants are audit-only.
- Uniform current clients agree; fracture needs mid-upgrade sub-quorum fragmentation.
- Residual Low: uncoordinated UV string/formula changes can strand specific outbounds until upgrade clear.

Status:

Accepted

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

1. Drop `ErrorMsg` from the ballot key (or map to a small on-chain enum); keep free text off consensus identity.
2. For `GasFeeUsed`: hash a canonical numeric encoding, or derive fee on-chain from agreed receipt fields (`EffectiveGasPrice` / SVM `gas_used`), or bucket/omit from the key and settle refund from finalized observation after success quorum on `(success, height, txHash)`.
3. Add `RevertStuckOutbound` / shorten expiry with a deterministic escape hatch.
4. UV upgrade runbooks: change fee/error strings only behind coordinated releases; optional dual-vote compatibility window.

[F-2026-18797](#) - [Dual Defense] Universal Client Chain Registry Lifecycle: Stale-Remove Deadlock and Unclosed DB Handles - Low

Description:

`universalClient/chains/chains.go` has two independent lifecycle defects on the same chain-registry path. Both affect only that UV's off-chain universal client process—not Push consensus or other correctly configured UVs.

Variant A — deadlock on stale remove. `fetchAndUpdate` holds `chainsMu.RLock()` while calling `removeChain`, which takes `chainsMu.Lock()`:

```
c.chainsMu.RLock()
for chainID := range c.chains {
    if chainID != c.pushChainID && !seenChains[chainID] {
        c.removeChain(chainID) // Lock while RLock held – deadlock
    }
}
c.chainsMu.RUnlock()
```

Go's `sync.RWMutex` is not reentrant. The refresh goroutine blocks forever holding the read lock; later `getClient` / confirmation paths stall. Trigger is a previously synced non-Push chain disappearing from `getAllChainConfigs` while still in the local map. Explicit `chainActionRemove` / update paths call `removeChain` outside the RLock and are safe. Process restart rebuilds an empty map from the current registry.

Variant B — unclosed per-chain SQLite handles. `getChainDB` always opens a new `db.OpenFileDB` (no cache). `addChain` opens before client construction; unsupported `VmType`, `NewClient` error, or `Start` failure returns without `Close`. `Client.Stop` closes RPC only; `removeChain` / `StopAll` drop map entries without closing the DB. Failed adds reopen once per bad chain per `ConfigRefreshIntervalSeconds` (default 60s). File pools set `MaxOpenConns(10)` / `MaxIdleConns(10)`, so each leaked `*db.DB` can retain multiple FDs; growth is linear in (bad chains × refresh ticks). Restart and config fix clear leaked handles.

Impact

- Stale registry delist under the RLock → Lock path permanently deadlocks that process's `chainsMu`.
- Failed `addChain` / remove / Stop paths leak SQLite FDs on that UV client.
- Triggers are registry delist or persistent misconfig—not a permissionless consensus halt.

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

1. Collect stale ids under `RLock`, unlock, then `removeChain` per id (or hold `Lock` for the mutation phase / `removeChainLocked`).
2. On `addChain` failure after open: `chainDB.Close()`; close on `removeChain` / `StopAll` after `client.Stop`.
3. Cache one `*db.DB` per chain id, or have `Client.Stop` own and close the DB.
4. Regression: omit a previously added chain from the next config fetch
→ refresh completes without hang; unsupported VmType / failed Start must not grow open sqlite FDs across N refresh ticks.

Resolution: The issue has been addressed in commit [f0083a3](#).

[F-2026-18798](#) - [Dual Defense] UExecutor ValidateBasic Parses Unbounded Decimals Before Ante - Low

Description:

`BaseApp.runTx` decodes the tx, runs `validateBasicTxMsgs`, then ante. `UniversalPayload.ValidateBasic` / `MigrationPayload.ValidateBasic` call `big.Int.SetString(value, 10)` on decimal string fields with only parse-success and non-negativity checks—no max digit length and no `BitLen() ≤ 256` despite the "uint256" comment. `MsgExecutePayload` / `MsgMigrateUEA` are gasless, so fee ante does not meter this work even when reached later.

That ordering + missing bound:

1. Memory is MiB-scale, not multi-GB. A 900 000-digit decimal is already ~0.86 MiB in the decoded string; `SetString` adds on the order of ~0.4 MiB of limbs (1.2 MiB combined)—within the default ~1 MiB CometBFT tx size envelope.
2. CPU is bounded and modest per tx. Local `SetString` of 900 000 nines completes in ~0.4 s on a laptop-class CPU; painful under concurrent CheckTx spam, not a single-request multi-second death spiral.
3. Impact is node-local admission pressure. Failed/unsigned txs die in CheckTx after ValidateBasic; they do not mint, drain vaults, or by themselves stop FinalizeBlock for the network.

Impact

- ValidateBasic runs before ante; decimal fields unbounded; gasless msgs skip fee for later stages.
- Practical: permissionless CheckTx/simulate CPU (and modest heap) spikes; concurrent spam can degrade mempool admission latency on targeted nodes.
- Simulation/RPC paths that decode+ValidateBasic share the same cost.

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

1. Cap decimal string length (e.g. ≤ 78 –80 chars for uint256) before `SetString`.
2. After parse: require `bi.BitLen() ≤ 256` (and `IsUint64()` where the field is semantically uint64).
3. Optionally reject oversized numeric strings in ante/TxValidator with gas, but prefer failing cheap in ValidateBasic.
4. Regression: 78-digit OK; 1000-digit / 900k-digit rejected without heavy `SetString`.

Resolution:

The issue has been addressed in commit [bbad4a6](#).

[F-2026-18799](#) - [Dual Defense] SIGN_FUND_MIGRATE Selects Current Sign Threshold, Not TssKeyHistory Shareholders - Low

Description:

`TssKeyHistory` stores each finalized key's `Participants` (documented for fund migration). Signing still loads the **old** keyshare via `OldKeyID`, and ECDSA verify uses `OldTssPubkey`. But the coordinator groups `SIGN_FUND_MIGRATE` with `SIGN_OUTBOUND`:

```
participants = getSignParticipants(allValidators) // random >2/3 of
current Active+PendingLeave
```

`createFundMigrationSignSetup` / `validateParticipants` never intersect that set with `TssKeyHistory[OldKeyID].Participants`. Newcomers lack the old keyshare; DKLS cannot succeed for a mixed set. After UV churn, if `|oldShareholders ∩ currentEligible| < CalculateThreshold(currentEligible)`, no valid subset exists—retries cannot help. Migration for that old vault becomes unsatisfiable until code/ops change.

The selection bug:

1. Admin-initiated migration lane. `InitiateFundMigration` is admin-gated and already requires outbound disabled + no pending outbounds for that chain. Stuck value is the old TSS EOA being swept—not active current-key outbound custody.
2. Needs material UV churn after the old key's shareholder set was fixed.
3. Recovery is upgrade/ops, not cryptographic burn. Fix participant selection to history shareholders (and the old key's signing threshold).

Impact

- fund-migrate setup uses `getSignParticipants(current)` while signing with old keyshare.
- no intersection with `TssKeyHistory.Participants`.
- when too few old shareholders remain among current eligibles vs current 2/3 threshold, migration cannot complete.
- Residual Low: incomplete migration tooling can leave an admin-initiated old-vault sweep unsatisfiable after churn.

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Severity:

Low

Recommendations**Remediation:**

1. For `SIGN_FUND_MIGRATE`, select participants from `TssKeyHistory[OldKeyID].Participants` (intersect live UVs that still hold the keyshare), using the old key's threshold/quorum rules—not `getSignParticipants(current)`.
2. Refuse initiate/setup when `|available old shareholders|` is below the old key's required signing threshold; surface a clear error.
3. Optionally require migration soon after KEYGEN before heavy churn, or keep old shareholders eligible until sweep completes.
4. Regression: old key with 3 shareholders; grow current set to 10; migrate must either succeed with the 3 (if crypto allows) or fail closed—not spin forever on random current-7 subsets.

Resolution:

The issue has been addressed in commit [f119cd2](#).

[F-2026-18800](#) - [Dual Defense] Short SVM send_funds Defaults TxType=0 While Claiming Funds → Mapped to GAS - Low

Description:

`decodeUniversalTxEvent` accepts a payload that ends after `revert_recipient` (no `tx_type` byte). It logs "not enough data for tx_type, defaulting to Funds" but sets `payload.TxType = 0` and returns success. Unit tests lock that in ("returns partial result when missing tx_type" → `TxType == 0`).

`EventProcessor.constructInbound` then maps Solidity/SVM-style wire values via:

```
case 0: txType = TxType_GAS // not FUNDS
case 2: txType = TxType_FUNDS
```

So a value-bearing short/legacy `send_funds` observation becomes a GAS inbound. Keeper dispatch runs `ExecuteInboundGas` (mint+autoswap into the sender UEA) instead of `ExecuteInboundFunds` (PRC20 deposit to recipient).

Current full gateway layout includes `tx_type` (+ sig / `fromCEA`); the defect is the short/legacy compatibility path: wrong default, contradicting its own log, with real misrouting if such logs are still observed.

Impact

- missing `tx_type` → `TxType=0` with "defaulting to Funds" log; processor maps `0` → `GAS` .
- GAS path credits sender UEA via swap; FUNDS path credits recipient PRC20—different economic outcome for the same amount/token fields.
- Residual Low: short/legacy layouts still accepted with a wrong default; current full-layout gateway emits are unaffected.

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

1. On missing `tx_type`, either **reject** the event (prefer) or set wire `2` (`FUNDS`) to match the log—never `0`.
2. Do not accept partial decode past `revert_recipient` without an explicit version/legacy flag.
3. Invert/fix the unit test: missing `tx_type` must fail parse or yield FUNDS (`2`), and an end-to-end `constructInbound` assert must not produce `TxType_GAS`.
4. Confirm deployed SVM gateways never emit the short layout; drop legacy acceptance if unused.

Resolution:

The issue has been addressed in commit [b5be6a7](#).

[F-2026-18801](#) - [Dual Defense] REJECTED Inbounds Land in ExpiredInbounds With No Shipped Refund Hatch - Low

Description:

When every inbound variant's ballot terminals as `EXPIRED` or `REJECTED` without a `PASSED` path, `BallotHooks.afterInboundBallotTerminal` removes `PendingInbounds` and writes `ExpiredInbounds` "for the future escape-hatch refund flow." No `UniversalTx` is created.

`MsgRevertStuckInbound` only accepts ballots with status `EXPIRED` and explicitly errors on `REJECTED`. `AttachRescueOutboundFromReceipt` requires an existing UTX—absent for these entries. So a `REJECTED`-terminal inbound has no shipped user or admin refund msg today, while `EXPIRED` still has the admin hatch (ballot remains queryable as `EXPIRED`).

`REJECTED` via `IsFinalizingVote` needs NO votes $\geq$ `VotingThreshold`. That is a strong quorum rejection of the observation—not a silent default for successful deposits. Documented incomplete hatch, not a new custody break in the happy path. Collection + query exist for a planned refund flow. `EXPIRED` is not in the same hole. Admin revert already covers `EXPIRED` + no UTX; this finding's load-bearing hole is `REJECTED`.

Impact

- all-variants `EXPIRED` / `REJECTED` → `ExpiredInbounds`, no UTX.
- admin revert refuses `REJECTED`; rescue needs UTX.
- Residual Low: incomplete "future escape-hatch" for quorum-`REJECTED` observations; `EXPIRED` already has admin revert.

Status:

Accepted

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

1. Extend `RevertStuckInbound` (or add `RefundExpiredInbound`) to accept `REJECTED` and/or operate from `ExpiredInbounds` entries when no UTX exists.

2. Optionally auto-schedule `INBOUND_REVERT` when moving to `ExpiredInbounds` if source custody is proven—prefer admin/gated hatch first.
3. Document ops runbook: query `AllExpiredInbounds`, correlate source txs, privileged refund.
4. Regression: inbound → quorum `REJECTED` → `ExpiredInbounds` → admin refund creates signable `INBOUND_REVERT`.

[F-2026-18802](#) - [Dual Defense] EVM FilterLogs Fixed 9000-Block Chunks Stall Cursor With No Adaptive Shrink - Low

Description:

`EventListener.processBlockRange` walks `[from,to]` in fixed `maxBlockRange = 9000` slices. Each slice is a single `RPCClient.FilterLogs (eth_getLogs)` over gateway+vault addresses and configured topics. On error, `processBlockChunk` returns, `processBlockRange` aborts, and `processNewBlocks` never calls `updateLastProcessedBlock` or advances `*currentBlock`. The listen loop only logs and retries the same window on the next tick—no chunk halving, topic split, or cursor skip.

That Gravity-style "over-limit getLogs → sticky cursor" gap:

1. Observation liveness for that UV process, not Push consensus halt or vault theft. Homogeneous public RPCs with the same result/size cap can stall a chain's inbound (and vault finalize) ingestion on those UVs; diverse archive/self-hosted RPC or a peer that still succeeds can still feed ballots.
2. Trigger is provider-limit + log density, not a one-tx free freeze. Flooding a 9000-block window enough to trip typical caps costs material source-chain gas/fees.
3. Ops recovery without protocol rewrite: point UVs at higher-limit RPC, temporarily narrow addresses/topics, or bump `EventStartFrom` / chain-height past the dense range.

Impact

- fixed 9000-block chunks; FilterLogs error prevents cursor advance; identical retry forever.
- Residual Low: dense log windows can stall EVM ingestion on UVs whose RPCs reject the query until ops/RPC/config relief or a client hotfix.
- UV/provider-scoped; spam- and limit-gated; recoverable without a consensus upgrade.

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

1. On FilterLogs failure (esp. result/size-limit errors), recursively halve `[from,to]` down to a floor, then advance.
2. Optionally split by address (gateway vs vault) or by topic when still over limit.
3. Persist partial progress (advance through successfully fetched sub-ranges) even if a later sibling chunk fails.
4. Metrics/alerts on repeated identical chunk failures; document RPC provider limits for UV ops.
5. Regression: mock over-limit on 9000-block query → smaller retries succeed and cursor moves past the window.

Resolution:

The issue has been addressed in commit [b93ea41](#).

[F-2026-18803](#) - [Dual Defense] Gasless MsgVoteChainMeta Can Inflate Unregistered ChainMetas Keys - Low

Description:

`MsgVoteChainMeta.ValidateBasic` only requires non-empty `observed_chain_id` (plus price/height > 0)—no uregistry / CAIP-2 gate. The msg is gasless (`app/txpolicy/gasless.go`). Admission is bonded-UV only (`IsBondedUniversalValidator`), unlike `VoteInbound`'s enabled-chain match.

`VoteChainMeta` persists a `ChainMetas` row on the cold-start path with a single fresh vote (`SetChainMeta` when `!bootstrapped && len(fresh) < 3`). Bootstrap quorum (3) gates only the first EVM oracle write—not store creation. One bonded UV can therefore insert unbounded distinct keys.

`PruneValidatorVotes` does an unpaginated `ChainMetas.Walk(ctx, nil, ...)` and is invoked from `UValidatorHooks.AfterValidatorRemoved` via the staking begin-unbonding hook—EndBlock-class, unmetered. Large N makes every UV-removal EndBlock do O(N) unmarshal/update work on all nodes.

The inflation + unmetered prune walk:

1. Bonded UV required, not a random account.
2. Per-vote DeliverTx is still gas-metered. Gasless skips fees, not KV write gas / block `max_gas`. Fill rate is paced by block gas over many txs.
3. O(N) EndBlock hit is on validator-removal / unbonding hooks, not every block. Degrades/stalls those EndBlocks at large N.

Impact

- no registry gate; single-UV cold-start persists new `ChainMetas` keys; gasless fee bypass.
- `PruneValidatorVotes` full Walk on UV removal (unmetered).
- Residual Low: malicious bonded UV can bloat `ChainMetas` and make UV-unbonding EndBlocks expensive until pruned/capped.

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

1. Require `observed_chain_id` $\in$ enabled uregistry chains (and CAIP-2 shape/length caps).
2. Do not `Set` a new `ChainMetas` key until bootstrap quorum (or reject unknown chains earlier).
3. Cap `ChainMetas` cardinality; paginate/index prune by validator $\rightarrow$ chains if needed.
4. Bound EndBlock prune work (budget, lazy GC, or delete-only index).
5. Regression: N fake chain votes from one UV must fail admission; prune of large store must not exceed EndBlock budget.

Resolution:

The issue has been addressed in commit [5b4b744](#).

[F-2026-18804](#) - [Dual Defense] EVM ABI readDynamicBytes Uint64 Overflow Panics Can Kill puniversald - Low

Description:

`readDynamicBytes` bounds-checks with wrapping `uint64` arithmetic:

```
if absOff+32 > uint64(len(data)) { return "", false }
byteLen := ...Uint64()
dataStart := absOff + 32
dataEnd := dataStart + byteLen
if dataEnd > uint64(len(data)) { return "", false }
return ..., data[dataStart:dataEnd]
```

Two wrap cases panic with `slice bounds out of range`:

1. `absOff ≈ 264` → `absOff+32` wraps small and passes the guard; `data[absOff:absOff+32]` panics.
2. Huge `byteLen` → `dataEnd` wraps below `dataStart`; length guard fails open; `data[dataStart:dataEnd]` panics.

`ParseEvent` runs in `EventListener.listen`'s background goroutine. Production `universalClient/chains/` has no `recover()`; an unrecovered panic terminates the whole `puniversald` process (all chains + TSS in that process). A sticky poison log from a bad RPC can crash-loop restarts.

The decoder:

1. Off-chain client only. `puniversald ≠ pchaind`. Impact is UV observation/signing liveness for that process—not CometBFT halt.
2. Needs malicious or MITM RPC (or equivalent forged `eth_getLogs`). Honest gateway Solidity ABI encoding does not emit near-`264` offsets/lengths.
3. Ops recovery: switch/failover RPC, quarantine the chain cursor/`EventStartFrom`, or hotfix bounds checks.

Impact

- overflow guards fail open; slice panics; no recover on listener path → process exit.
- Residual Low: compromised/MITM RPC can crash-loop a UV's universal client; fleet impact needs correlated bad RPCs.
- Off-chain client; honest gateway ABI does not emit these words;

Status:

Fixed

Classification

Impact Rate:

1/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

1. Reject offsets/lengths that overflow when adding (`if absOff > max-32` , `if byteLen > max-dataStart` , or use checked add).
2. Prefer `int` lengths with explicit `absOff > len || absOff+32 > len` before any slice; never slice when `dataEnd < dataStart` .
3. `defer recover()` around per-log parse (log + skip) so one bad log cannot kill the process.
4. Regression unit tests for both overflow vectors must not panic and must return decode failure.

Resolution: The issue has been addressed in commit [e6ba904](#) .

[F-2026-18815](#) - [Dual Defense] SVM Client Omits Relay-Paid Recipient ATA Rent From GasFeeUsed - Low

Description:

SPL withdraw / revert / rescue builds prepend

`CreateAssociatedTokenAccountIdempotent` with the relay as payer, then run the Gateway finalize ix. Comments state rent is "reimbursed via `gas_fee`," and test budgets add ~2.04M lamports for CEA ATA. Success observations set `GasFeeUsed` only from the Gateway event's `gas_used` field. The parser documents an `ata_created` flag in the same event layout but never reads it or adds client-side ATA rent.

So destination cost paid outside the Gateway instruction (recipient ATA rent) is invisible to `x/uxecutor applyGasRefund (GasFee - GasFeeUsed)`. A recipient who forces a new ATA (fresh owner), receives the SPL transfer, empties the ATA, and closes it can reclaim that rent while Push treats the omitted lamports as unused prepaid gas.

Same accounting class as OP L1 fee omitted from `GasFeeUsed` —not vault principal theft:

1. Bounded per-outbound gas subsidy (~ATA rent-exempt), not bridged inventory drain. Loss is fee-vault / prepaid-gas accounting (and/or relay float until reimbursed), on the order of ~0.002 SOL per forced new ATA.
2. Requires a full paid SPL outbound (inbound/burn path, UV ballot, TSS sign, Solana inclusion) per cycle.
3. Under-reporting `gas_used` loosens `gas_fee ≥ gas_used`, it does not make Gateway reject.

Impact

- client prepends ATA create (relay pays); `GasFeeUsed` = Gateway `gas_used` only; `ata_created` unused.
- `applyGasRefund` can over-refund by the omitted rent when prepaid `GasFee` reserved ATA headroom.
- Residual: attacker-driven fresh-recipient cycles extract ~rent-exempt SOL economics from gas accounting; repeatable but small and UV/TSS-gated.

Status:

Fixed

Classification

Impact Rate:

1/5

Likelihood Rate: 1/5

Severity: Low

Recommendations

Remediation:

1. Include newly created recipient ATA rent in reported `GasFeeUsed` (fixed rent-exempt minimum, or `gas_used + ata_rent` when the client created / when `ata_created` is true).
2. Prefer Gateway-owned ATA create inside the metered path so `gas_used` is authoritative—or stop prepending client ATA once Gateway covers it.
3. Align UniversalCore SVM quotes with total relayer lamports (sig fee + PDAs + ATA when needed).
4. Regression: fresh SPL withdraw recipient → observation `GasFeeUsed` ≥ Gateway `gas_used` + ATA rent-exempt; refund shrinks accordingly.

Resolution:

The issue has been addressed in commit [05996bb](#) and commit [5b50e76](#).

[F-2026-18816](#) - [Dual Defense] Empty authz.MsgExec Vacuously Gasless Can Stuff Declared Block Gas - Low

Description:

`txpolicy.IsGaslessTx` treats `authz.MsgExec` as gasless when every nested `TypeURL` is allowlisted. For `Msgs: []` the inner loop runs zero times, so the outer transaction is accepted as gasless by vacuous truth. `DeductFeeDecorator` and `MinGasPriceDecorator` then skip fee checks. SDK `authz` rejects empty `Msgs` only in the message server (`messages cannot be empty`), not in ante/`ValidateBasic`, so CheckTx can pass a correctly signed zero-fee empty `MsgExec`.

Push retains default BaseApp mempool/proposal behavior (no custom Prepare/ProcessProposal). Proposal selection accounts for `FeeTx.GetGas()` (declared gas). An attacker can set declared gas to a finite consensus `max_gas`, pass as gasless, and—if ordered first in the proposer's FIFO reap—fill the block gas budget so later honest txs are not selected. FinalizeBlock still succeeds: ante passes, the authz handler fails cheaply, and the failed result is a valid included transaction.

Blocks continue to be produced; impact is per-block capacity censorship under sustained mempool spam and a finite `max_gas`. No fund theft or state corruption.

Impact

- empty nested `MsgExec` $\Rightarrow$ `IsGaslessTx` true; fee and min-gas-price bypass.
- empty list rejected only at authz msg server, after ante.
- default proposal path uses declared gas for capacity.
- Attack can exclude honest txs from a proposed block when a full-`max_gas` spam tx is selected first.
- Sustained censorship requires continuous zero-fee spam into proposers' mempools (disposable keys via gasless account init amplify this).
- Blocks still finalize; chain does not halt under the finite-`max_gas` variant.

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

1. In `IsGaslessTx`, for `MsgExec` require `len(m.Msgs) > 0` and every inner `TypeURL` allowlisted; empty nest $\Rightarrow$ not gasless (or reject).
2. Optionally reject empty `MsgExec` in ante before fee bypass.
3. Cap declared gas for gasless txs well below block `max_gas`.
4. Optional: custom `PrepareProposal` that ignores failed-cheap / gasless spam or prioritizes fee-paying txs.
5. Regression: empty `MsgExec` fails `CheckTx` or pays fees; cannot monopolize block `gasWanted` at zero fee.

Resolution:

The issue has been addressed in commit [b851fde](#).

[F-2026-18817](#) - [Dual Defense] SVM Log Truncation + Resolver No Success-Vote Starves Oldest-1000 Pending Queue - Low

Description:

SVM outbound success on Push is driven by the Universal Client parsing a `UniversalTxFinalized` `Program data: log ( event_listener.go / parseOutboundObservationEvent )`. `resolveSVM` treats an on-chain `ExecutedTx` PDA as local `COMPLETED` and returns without casting a Push success vote; its only Push vote is failure when the PDA is absent past deadline. If destination CPI logging exhausts Agave's ~10 KB log budget before the gateway's post-CPI `emit!(UniversalTxFinalized)`, the tx can still succeed (PDA + gas settlement) while the terminal log is dropped. No listener success vote and no resolver success vote leave the row in `PendingOutbounds` until a quorum vote removes it.

The shipped signer polls `GetAllPendingOutbounds` with `Limit: 1000`, oldest-first (`pushCore.go`), and does not walk `Pagination.Total`. A prefix of ~1000 permanently stuck SVM rows can exclude later pending outbounds from signing. The client also skips IDs already in the local DB via `storeEvent` duplicate discard, so a sticky oldest-1000 prefix is replayed forever and never advances to newer rows.

The terminalization gap works as follows:

1. Impact is outbound liveness / signing starvation, not remint, double settlement, or vault drain. Dest already paid for poison rows; Push pending simply never terminals.
2. Each poison row must itself be TSS-signed and executed on Solana before it becomes stuck. Filling ~1000 rows requires ~1000 successful outbound cycles (gas-token burn / fees), not a single cheap spam tx.
3. Ops/client recovery exists without reminting paid fills: outbound pause, client hotfix (PDA → success vote; paginate pending), privileged success observation / pending clear, upgrade. Remint after dest success would double-settle and is the wrong recovery story.
4. Agave log truncation is environmental; the Push defect is failing to reconcile durable PDA success into a consensus vote (and fixed oldest-1000 consumption).

Impact

- `resolveSVM` PDA-present path does not vote success on Push.
- success observation depends on destination log parsing; truncated logs yield no event.
- client pending poll is a single oldest-1000 page with no offset walk.
- Stuck executed-but-unvoted rows can starve later outbounds' signing until cleared.

- Setup cost scales with ~1000 signed SVM finalizes + gas; not zero-fee.
- Costly multi-cycle setup; ops/client recovery via success-vote/hotfix; no vault drain or remint-after-pay.

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

1. On ExecutedTx PDA present, cast authenticated outbound success vote (mirror fund-migration resolver that votes both ways from chain state).
2. Prefer gateway emit before untrusted CPI, or use non-log commitment (PDA field / return data) as the sole success signal.
3. Paginate pending polls (follow Total / offset) or exclude locally COMPLETED / already-broadcast IDs from the signing prefix.
4. Bound concurrent pending per sender / zero-amount GAS_AND_PAYLOAD rate limits.

Resolution: The issue has been addressed in commit [7f9e50f](#)

[F-2026-18818](#) - [Dual Defense] CallEVMWithData Ignores gasCap — Callback Gas Sandbox Broken - Low

Description:

`CallEVMWithData` accepts `gasCap *big.Int` but hardcodes `GasLimit: config.DefaultGasCap` (25_000_000). IBC callback keepers pass `CommitGasLimit` / remaining gas into `CallEVM` / `CallEVMWithData` intending to sandbox EVM work; that parameter is unused. `DerivedEVMCallWithData` honors `gasLimit`; this helper does not.

Callback paths install `NewInfiniteGasMeterWithLimit(cbData.CommitGasLimit)` for EVM work, then bill `res.GasUsed` onto the `ProcessCallback` basic meter via `ctx.GasMeter().ConsumeGas`. `InfiniteGasMeterWithLimit.ConsumeGas` does not panic past its limit (only uint64 overflow); `IsOutOfGas` is always false. So EVM execution can finish with `GasUsed` up to `DefaultGasCap`, then the later `ConsumeGas` on ibc-go's `NewGasMeter(ExecutionGasLimit)` can panic with `ErrorOutOfGas`.

`internal.ProcessCallback` recovers non-`SendPacket` panics into `ErrCallbackPanic`. It re-panics only when `AllowRetry()` is true (`ExecutionGasLimit < CommitGasLimit`), i.e. the relayer has not yet reserved `CommitGasLimit`. With sufficient reserved gas, ack/timeout middleware still returns `nil` after the callback and continues the packet lifecycle.

`CommitGasLimit` is capped at `maxCallbackGas` (1M). After an OOG panic on ibc-go's basic `ProcessCallback` meter, defer charges the parent `GasConsumedToLimit()` ($\leq$ execution limit), not the full 15–25M EVM figure. `OnRecvPacket` can still emit an error ack on recovered callback failure (refund path).

Impact

- `gasCap` ignored; EVM message limit is `DefaultGasCap`.
- oversized `GasUsed` can OOG-panic the `ProcessCallback` basic meter after a successful EVM call.
- With relayer gas $\geq$ `CommitGasLimit`, panic is recovered; `OnAcknowledgementPacket` / `OnTimeoutPacket` do not abort the transfer lifecycle on callback failure.
- Residual Medium impact: broken sandbox allows up to ~25M EVM gas of work per malicious callback while Cosmos charges $\leq$ `CommitGasLimit` ($\leq$ 1M)—validator/relayer CPU griefing and failed callbacks, not irreversible fund lock.

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

1. In `CallEVMWithData`, set `GasLimit` from `gasCap` when non-nil (and clamp to `DefaultGasCap` / block gas); match `DerivedEVMCallWithData`.
2. Optionally enforce `CommitGasLimit` inside `InfiniteGasMeterWithLimit.ConsumeGas` or stop replacing the `ProcessCallback` meter with a non-enforcing infinite meter.
3. Cap post-call `ConsumeGas` with `min(res.GasUsed, remaining)` and return `ErrCallbackFailed` / `ErrOutOfGas` instead of relying on panic + recover.
4. Align timeout `CallEVM` to `cachedCtx`.
5. Regression: malicious callback that burns `> CommitGasLimit` EVM gas fails the callback without exceeding EVM `GasLimit`, and with reserved `CommitGasLimit` the ack/timeout tx still commits the ICS-20 lifecycle.

Resolution: The issue has been addressed in commit [0076392](#).

[F-2026-18819](#) - [Dual Defense] IBC Refund Conversion Swallows EVM Error After Escrowing Coins Into erc20 Module - Low

Description:

Error-ACK / timeout refunds run transfer first, then erc20 middleware `ConvertCoinToERC20FromPacket` for native-ERC20 pairs. That path calls `ConvertCoinNativeERC20`, which first `SendCoinsFromAccountToModule` (bank mutation on the parent ctx), then `CallEVM(... "transfer" ...)`. `CallEVMWithData` caches only the EVM execution: on VM failure it discards the EVM cache and returns an error, leaving the already-applied bank escrow intact.

`ConvertCoinToERC20FromPacket` then does:

```
if err := k.ConvertCoinNativeERC20(...); err != nil {  
    // emit FailedConvertERC20 + telemetry  
    return nil // swallow - IBC lifecycle continues  
}
```

Comments on `OnAcknowledgementPacket` / `OnTimeoutPacket` claim that on conversion failure "the user receives the corresponding bank token" and may retry manually. That assumption is false once escrow has succeeded: the refunded coins are no longer on the sender; they sit on the `erc20` module account, and the ERC20 `transfer` never credited the user. There is no shipped user msg to reclaim those module balances.

Contrast: `OnRecvPacket` still returns `NewErrorAcknowledgement(err)` on `ConvertCoinNativeERC20` failure (does not swallow). The swallow is specific to the refund reconversion path.

The non-atomic convert + swallow:

1. Failure-conditioned, not free remote drain. Needs post-refund EVM unescrow failure (paused/blacklisted token, VM revert, fee-on-transfer invariance, etc.).
2. Sender-scoped refund amount only — not third-party vault wipe.
3. Balances are not burned. Coins remain on the known `erc20` module account; gov/upgrade `SendCoinsFromModuleToAccount` recovers without cryptographic destruction.

Impact

- bank escrow precedes EVM transfer; EVM failure does not undo the bank send.
- refund path returns `nil` on convert error → commit keeps module escrow; user gets no ERC20.
- code comments promising retained bank coins are incorrect after successful escrow.

- Needs conversion-time EVM failure; sender-scoped; module-held (privileged recoverable).
- Manual `MsgConvertCoin` still errors out (full tx revert) — bug is the IBC swallow + non-atomic convert composition.

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

1. Run the entire `ConvertCoinNativeERC20` body (bank + EVM + burn) inside a `CacheContext` and only write on full success; on any error discard and return nil from the IBC wrapper.
2. Or, on EVM/invariant failure, explicitly `SendCoinsFromModuleToAccount` back to sender before returning nil.
3. Fix comments: do not claim the user retains bank coins unless rollback is implemented.
4. Regression: forced EVM transfer revert after refund must leave coins on the sender (or equivalently unlocked), never stranded on `erc20` module.

Resolution: The issue has been addressed in commit [913bc7c](#).

[F-2026-18820](#) - [Dual Defense] GAS Auto-Swap minPCOut = 95% of Live Spot Quote Is Self-Referential Slippage - Low

Description:

`TxType_GAS` / `GAS_AND_PAYLOAD` execution quotes Uniswap V3 via `GetSwapQuote` (`quoteExactInputSingle`, live pool), sets `minPCOut = quote * 95 / 100`, then immediately `CallPRC20DepositAutoSwap` with that minimum (deadline `0` → contract default). Quote and swap share one Push tx, so the 5% band only covers movement *after* the quote inside that tx—not a pool price already skewed before the quorum-finalizing vote.

Users cannot attach `minPCOut`, deadline, TWAP bound, or impact cap on the source Gateway inbound. Combined with the cross-chain observation window (source deposit public → UV confirm → finalizing vote), an attacker can try to move the pGasToken/WPC pool on Push, let UExecutor market-sell the victim mint against that spot, then unwind.

The self-referential slippage construction protocol pricing gap:

1. Holding a bad spot through confirmation is the hard part. UVs need standard inbound depth; arbs and other flow push the pool back. Inventory used to skew price sits at adverse selection risk for the whole window.
2. PoC extraction ratios imply shallow liquidity. Growing $\sim 0.1 \rightarrow \sim 5.46$ against a 10-unit victim flow requires a pool where tiny inventory dominates depth.
3. Loss class is forced AMM execution quality, not unbacked mint or vault bug. Victim still receives a successful swap at the manipulated-but-self-consistent minOut; attacker profits via ordinary pool inventory + later gateway burn/redeem.
4. Scoped to automatic GAS conversion paths that call `GetSwapQuote` + `DepositAutoSwap`—not every FUNDS mint or all bridge custody.

Impact

- `minPCOut` derived from the same mutable spot quote used to execute; no user minOut/TWAP/oracle gate.
- cross-chain delay can give a pre-vote window to move Push pool state.
- Practical Medium: users of automatic GAS → PC conversion can receive worse PC (and attackers can capture inventory/redeem edges) when pools are shallow and the skew is maintained until finalize.

Status:

Accepted

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

1. Carry user-authorized `minPCOut` / deadline from authenticated source inbound into the ballot and enforce on Push.
2. Bound execution vs TWAP / external oracle; reject or defer swap on excess deviation (revert path already exists on quote/swap failure).
3. Cap GAS auto-swap size / max price impact per pool.
4. Prefer fail-closed (revert outbound) over executing at unsafe self-quoted mins.
5. Regression: pre-skew pool → finalize GAS inbound must not accept a minOut that only mirrors the skewed spot (or must revert).

[F-2026-18821](#) - [Dual Defense] Public InboundKeys Query Runs Unbounded Quadratic Solana Base58 Decode - Low

Description:

`Querier.InboundKeys` is a public gRPC/REST helper: nil-check only, then `Inbound.Canonicalize`, `GetInboundBallotKey`, and `GetInboundUniversalTxKey`. No `ValidateBasic`, signer, state read, fee, or query-gas budget.

For `source_chain` in the `solana` namespace, `LenientCanonicalizeTxHash` → `canonicalizeSolanaTxHash` calls `base58.Decode` on any nonempty base58-alphabet string with no max length. If the decode is not exactly 64 bytes, the long string is still returned after the full decode. Pinned github.com/mr-tron/base58 `FastBase58DecodingAlphabet` is quadratic: for each of `n` characters it walks `ceil(n/4)` limbs.

The same `tx_hash` is decoded three times on this path (`Canonicalize`, then UTX key, then ballot key)

The unauthenticated quadratic query cost:

1. Query-path / process CPU grief, not consensus message validity. Impact is public API workers (and shared-process CPU contention), not an unmetered DeliverTx state transition or vault drain.
2. Ops controls exist. Reverse-proxy body/time limits, gRPC max recv size, disabling public REST/gRPC, or IP rate limits cut remote reachability.

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

1. Cap Solana tx hash / base58 inputs (e.g. ≤ 128 chars for a 64-byte sig) before `Decode`; reject oversize in `InboundKeys` (and prefer the same cap on vote canonicalize).
2. Decode once per request; reuse the canonical string for UTX/ballot key helpers.

3. Skip `Decode` unless `len` is in the plausible signature band; do not fully decode obviously-too-long strings just to return them unchanged.
4. gRPC/REST: max body size + handler timeout; avoid exposing unbounded canonicalize helpers publicly.
5. Regression: `n=1e5` base58 `tx_hash` on `InboundKeys` must fail fast (<10 ms), not scale as $\Theta(n^2)$.

Resolution:

The issue has been addressed in commit [b3a2058](#).

[F-2026-18822](#) - [Dual Defense] Native ERC20 Convert Skips Approval Log Check — Unbacked Mint on Malicious Pair - Low

Description:

`convertERC20IntoCoinsForNativeToken` (and `ConvertCoinNativeERC20`) document that conversion must reject unexpected `Approval` logs.

`validateApprovalEventDoesNotExist` exists and is unit-tested, but neither convert path calls it. After a successful `transfer` into the module, the keeper only checks module ERC20 balance increased, then mints bank coins.

`MsgRegisterERC20` is permissionless when `params.PermissionlessRegistration` is true (`DefaultParams` sets it true). An attacker can register a malicious ERC20 whose `transfer` also `approve`s a thief on the module escrow, convert, then `transferFrom` the escrow. Integration suite even labels this `"pass - delayed malicious contract"` (`contractMaliciousDelayed`)—convert is expected to succeed today.

The invariant break:

1. Per-pair escrow only. `Approval` / `transferFrom` are on the malicious ERC20 contract. Module balances of other registered pairs (WPC wrappers, honest USDC, etc.) are untouched.
2. Malicious-token / social-engineering class. Harm falls on converters/holders of the attacker-registered denom (`erc20: <malicious>`).
3. Self-deal is the clean PoC. Attacker deposits own tokens, keeps bank mint, pulls ERC20 back via allowance. Net: unbacked bank supply of their denom; third-party loss needs others to accept or convert that denom.
4. Incomplete mitigation, not absent intent. Helper + comments + fixture show the intended defense; wiring is missing.

Impact

- convert comments require Approval check; helper unused on both native convert directions.
- permissionless registration default on; malicious delayed convert tested as `expPass: true`.
- after convert + `transferFrom`, that pair's module ERC20 escrow can be zero while bank supply remains.
- Residual Medium: permissionless register + convert yields unbacked bank supply / rug for that attacker-registered denom only; third-party loss needs holders of that evil pair.

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

1. After escrow `transfer`, call `validateApprovalEventDoesNotExist(res.Logs)` and fail the tx (both convert directions).
2. Also assert module `allowance(module, *) == 0` (or no unexpected spenders)—silent approves without logs would bypass log-only checks.
3. Flip integration case to `expPass: false` for `ERC20MaliciousDelayed`; add post-condition that convert does not leave nonzero allowances on the module.
4. Consider defaulting `PermissionlessRegistration` false until convert hardening lands, or gov-gate new pairs.

Resolution: The issue has been addressed in commit [2dd5e31](#).

[F-2026-18823](#) - [Dual Defense] buildRevertOutbound Fail-Open Leaves Unsignable INBOUND_REVERT and Blocks Rescue - Low

Description:

Non-CEA inbound failures attach an `INBOUND_REVERT` via `buildRevertOutbound`. That helper always returns a `PENDING` outbound. On `GetTokenConfig` or `GetGasFeeInfoForRevertOutbound` error it only logs and returns with empty `GasToken` / `GasFee` / `GasPrice` / `GasLimit`:

```
// "proceeding without gas fields"
return outbound
```

`attachOutboundsToUtx` indexes that row in `PendingOutbounds`. EVM `UniversalClient` `GetOutboundSigningRequest` then hard-fails (`gas price is zero or missing`, `gas limit is required`)—no TSS sign, no broadcast. Non-CEA `RESCUE_FUNDS` requires an existing `INBOUND_REVERT` with status `REVERTED`; a forever-`PENDING` auto-revert never qualifies. Fixing the oracle later does not rewrite the already-stored empty fields.

Empty gas only when PRC20 lookup or `UniversalCore` `getOutboundTxGasAndFees` fails at build time—not on every failed inbound when gas metadata is healthy. Sender-scoped stranded principal, not permissionless vault drain. Recovery is upgrade/ops: fill gas fields / force `REVERTED` / alternate return path via gov upgrade.

Impact

- fail-open on gas metadata errors; empty-gas `PENDING` persisted.
- EVM builder rejects zero/missing gas before signing.
- non-CEA rescue gated on `INBOUND_REVERT` + `REVERTED`.
- Residual Medium: when gas lookup fails at revert construction, that inbound's recovery dead-ends until privileged state repair.
- Needs concurrent gas-metadata failure; healthy oracle path still signs reverts; sender-scoped.

Status:

Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity: Low

Recommendations

Remediation:

1. Fail closed: if gas metadata cannot be filled, do not attach/index a PENDING revert (surface `RevertError` / retry), or attach only after fields are complete.
2. If a partial row must exist, mark it non-signable and allow rescue / admin rebuild once gas oracle is healthy (or admin msg to patch gas fields / force `REVERTED`).
3. Make `buildRevertOutbound` return `(outbound, error)`; fix admin `nil` check accordingly.
4. Regression: force `GetOutboundTxGasAndFees` error → no empty-gas `PendingOutbounds` row; or rescue unblocked by explicit policy.

Resolution:

The issue has been addressed in commit [4f6a125](#).

[F-2026-18824](#) - [Dual Defense] Failed DerivedEVMCallWithData / CalLEVMWithData Skip Cosmos ConsumeGas - Low

Description: Both `DerivedEVMCallWithData` and `CalLEVMWithData` charge the parent Cosmos gas meter only on the success path:

```
if res.Failed() {  
    return res, errorsmod.Wrapf(...)  
}  
ctx.GasMeter().ConsumeGas(res.GasUsed, "apply evm message")
```

Failed EVM work (revert / out-of-gas inside the message) never hits `ConsumeGas`. A prior change left the parent meter untouched on revert to avoid charging `GasMeter().Limit()` (which aborts the whole Cosmos transaction). Charging `res.GasUsed` on failure was never applied, so failed derived work is free at the Cosmos meter.

Reachability via gasless `MsgExecutePayload`:

1. The msg is gasless; ante fee and min-gas-price early-return.
2. `CallUEAExecutePayload` passes caller `UniversalPayload.GasLimit` into `DerivedEVMCall` with no protocol clamp.
3. `DerivedEVMCall` discards the non-nil failed response and returns `nil, err`.
4. `DeductGasFeesFromReceipt` short-circuits on `receipt == nil`, so UEA gas fees are not deducted on failed derived calls despite the "regardless of success/failure" comment.

Impact

- Failure branches of `DerivedEVMCallWithData` and `CalLEVMWithData` omit `ConsumeGas(res.GasUsed)`.
- Gasless `MsgExecutePayload` plus nil receipt yields no UEA gas fee on failed derived calls.
- Unprivileged zero-fee CPU via failed UEA payloads (and other callers that rely on Cosmos metering for cost).
- No mint, remint, or vault movement; availability and fee hygiene only.

Status: Fixed

Classification

Impact Rate: 2/5

Likelihood Rate: 2/5

Severity:

Low

Recommendations**Remediation:**

1. On failure, `ConsumeGas(res.GasUsed)` against the parent meter—do not charge `Limit()`; do not charge nothing.
2. Propagate failed `MsgEthereumTxResponse` (or `GasUsed`) through `DerivedEVMCall` so UEA fee deduction can run on revert/OOG.
3. Clamp `UniversalPayload.GasLimit` / derived `gasCap` to `DefaultGasCap` and/or Cosmos tx/block gas.
4. Regression: gasless failed UEA call with `GasLimit=25_000_000` must consume ~25M Cosmos gas (or reject at admission); UEA fee path must see non-zero `GasUsed` on VM failure.

Resolution:

The issue has been addressed in commit [7ac130d](#) and commit [f38bf13](#).

[F-2026-16598](#) - Unsafe Status Modification Order in MarkBallotExpired() and MarkBallotFinalized() - Info

Description: Ballot `Status` is modified and persisted to storage before all collection operations complete.

The `'MarkBallotExpired'` function in `'x/validator/keeper/ballot.go'` (lines 114-133) removes a ballot ID from `'ActiveBallotIDs'` without first verifying the ID is currently present in the set. Similarly, `'MarkBallotFinalized'` (lines 137-160) has the same issue. If either function is invoked multiple times for the same ballot (e.g., due to network retries or transaction replays), the second call will fail because it attempts to remove an already-removed key.

The effects is that the Ballot is marked `EXPIRED` in storage, but not removed from `ActiveBallotIDs` and not added to `ExpiredBallotIDs`. The ballot becomes "orphaned" in an inconsistent state.

Status: Fixed

Classification

Severity: Info

Recommendations

Remediation: Only set the Ballot after all checks are done:

```
func (k Keeper) MarkBallotExpired(ctx context.Context, id string) error {
    // ... get ballot ...

    // Only modify and persist atomically with all other operations
    if err := k.ActiveBallotIDs.Remove(ctx, id); err != nil {
        return err
    }
    if err := k.ExpiredBallotIDs.Set(ctx, id); err != nil {
        return err
    }

    ballot.Status = types.BallotStatus_BALLOT_STATUS_EXPIRED
    return k.Ballots.Set(ctx, id, ballot)
}
```

Resolution: This issue has been addressed and resolved in commit [4d9c8ee](#).

[F-2026-16614](#) - Centralized registry and Universal Validator roster - Info

Description:

Chain and token configuration under `x/uregistry` is governed by administrative parameters and governance. The Universal Validator roster is maintained under `x/uvalidator`. Inbound voting and user execution paths consult registry flags such as whether inbound transfers are enabled for a given chain. Supported routes may therefore be disabled or reconfigured without individual user consent, which affects new activity and may exclude specific assets or chains according to policy.

Users may find that new bridging or execution stops abruptly when governance or administrators change registry settings. Operators must align incident response and communications with governance outcomes and on-chain configuration snapshots. From a compliance and product perspective, centralized route control can be desirable or sensitive depending on the regulatory environment. It should be disclosed plainly rather than implied.

Status:

Accepted

Classification

Severity:

Info

Recommendations

Remediation:

Document for end users and integrators that route availability is governance- and admin-controlled. Use time-locks, multisig thresholds, and transparent changelogs for registry updates where appropriate. Describe exit or recovery paths when a chain is disabled, and add monitoring that surfaces inbound-enablement changes for operators.

[F-2026-16616](#) - Chain Metadata Oracle Using First Vote Instead of Median Aggregation - Info

Description: When no `ChainMeta` record exists for `observedChainId`, the first successful `VoteChainMeta` path invokes `CallUniversalCoreSetChainMeta` immediately with that validator's price and block height, then persists a single-voter state. Median aggregation, staleness filtering, and stricter height rules apply only after initialization. The first writer therefore temporarily defines on-chain oracle values without multi-validator aggregation.

Contracts or applications that consume chain metadata may observe briefly skewed gas prices or heights during oracle bootstrap. DeFi components that price or time operations from this feed inherit that cold-start sensitivity. The first validator to succeed on a new `observedChainId` exerts disproportionate influence unless governance seeds values or multiple votes are required first.

Status: Fixed

Classification

Severity: Info

Recommendations

Remediation: Implement multiple agreeing votes or a governance-supplied seed before the first contract write, or defer the EVM update until a median can be computed from an initial batch. Document cold-start semantics for integrators. Add property or integration tests for the transition from empty metadata to multi-voter median behavior.

Resolution: This issue has been addressed and resolved in commit [8848c0a](#).

[F-2026-16619](#) - ExecutePayload and binding of UniversalAccountId.owner to the signer - Info

Description:

The `ExecutePayload` implementation does not require that `UniversalAccountId.Owner` equal the EVM address derived from `MsgExecutePayload.Signer`. The factory resolves the target UEA from the entire `UniversalAccountId` embedded in the transaction, while `CallUEAExecutePayload` sends the call with `evmFrom` computed solely from `Signer`. Neither the keeper nor the gRPC handler introduces an equality predicate linking those two identities.

Accordingly, who signs the Cosmos transaction and which owner the message names may diverge without rejection at this layer. Whether that divergence is benign (e.g., sponsored execution with off-chain proof) or hazardous is determined only by the UEA's `executeUniversalTx` implementation and the semantics of `verificationData`.

Status:

Accepted

Classification

Severity:

Info

Recommendations

Remediation:

1. For paths where `Owner` must equal `Signer`, enforce equality in `ValidateBasic`, the handler, or the keeper (normalize addresses before comparison).
2. If relay remains supported, distinguish message variants or flags so unchecked divergence is explicit at the API surface.
3. Exercise UEA `executeUniversalTx` under `Owner ≠ evmFrom` with crafted `verificationData` in unit and integration tests.
4. Publish the authoritative rule: module-enforced binding vs contract-only binding.

[F-2026-16648](#) - Default registry administrator in DefaultParams - Info

Description:

`x/uregistry` `DefaultParams` returns a fixed bech32 administrator address alongside a `TODO` marker. `Validate` rejects only an empty administrator string. If genesis or migrations omit an override, that default address controls registry-gated messages until changed through authorized flows, which is undesirable for production networks.

`x/uregistry/types/params.go` — `DefaultParams`,

```
// DefaultParams returns default module parameters.
func DefaultParams() Params {
// TODO:
return Params{
Admin: "pushlnegskcfqu09j5zvpk7nhvacnwyy2mafffy7r6a",
}
}
```

`x/uregistry/types/params.go` — `Validate`,

```
func (p Params) Validate() error {
if strings.TrimSpace(p.Admin) == "" {
return fmt.Errorf("admin address cannot be empty")
}
return nil
}
```

Improperly configured genesis files for testnets or mainnet deployments can unintentionally grant elevated privileges to unauthorized keys. Furthermore, reliance on hardcoded default addresses introduces ambiguity between development and production environments, increasing the risk of misconfiguration during security reviews.

Status:

Fixed

Classification

Severity:

Info

Recommendations

Remediation:

Always set the registry administrator explicitly in production genesis. Prefer defaults that cannot be mistaken for production keys, or fail genesis validation unless an administrator is supplied. Add continuous integration checks on generated genesis artifacts for expected multisig administrators.

Resolution:

This issue has been addressed and resolved in commit [2003414](#)

[F-2026-16655](#) - Fund migration votes pass txHash without normalization - Info

Description:

`MsgVoteOutbound` strips an optional `0x` prefix from transaction identifiers before voting. `MsgVoteFundMigration` forwards `txHash` unchanged into the ballot key helper. Validators that report the same logical transaction with and without the `0x` prefix therefore hash different ballot identifiers, splitting quorum in the same structural class of issue as serialization-sensitive keys elsewhere.

Outbound normalization occurs in the executor message server.

File: `push-chain-node/x/uexecutor/keeper/msg_server.go` ·

```
// Normalize IDs: strip 0x prefix
msg.TxId = strings.TrimPrefix(msg.TxId, "0x")
msg.UtxId = strings.TrimPrefix(msg.UtxId, "0x")
```

Fund migration passes the raw message field through to the keeper without altering `TxHash`.

File: `push-chain-node/x/utss/keeper/msg_server.go` ·

```
err = ms.k.VoteFundMigration(ctx, signerValAddr, msg.MigrationId, msg.TxHash, msg.Success)
if err != nil {
    return nil, err
}
```

The ballot key concatenates the migration identifier, raw hash string, and success flag before hashing.

File: `push-chain-node/x/utss/types/keys.go` ·

```
func GetFundMigrationBallotKey(migrationId uint64, txHash string, success bool) string {
    canonical := fmt.Sprintf("fm:%d:%s:%t", migrationId, txHash, success)
    h := sha256.Sum256([]byte(canonical))
```

Fund migration finalization can stall despite off-chain agreement because on-chain ballots never merge. Operators waste time reconciling formatting rather than chain facts. Users experience delayed or stuck migration completion.

Status:

Accepted

Classification

Severity:

Info

Recommendations

Remediation:

Canonicalize `txHash` in the fund migration message server before voting—strip optional `0x`, lowercase hexadecimal for EVM hashes, and reject malformed lengths—reusing shared helpers with outbound handling. Add parameterized tests that enumerate common alternate representations of the same logical hash. Document the canonical format for Universal Validator operators.

[F-2026-16867](#) - Logging and standard output disclose sensitive material - Info

Description:

The Universal Client exposes sensitive operational data through two channels that are commonly aggregated, retained, and forwarded to centralized monitoring. First, the `puniversald start` command serializes the loaded `Config` structure to JSON and writes it to standard output before initialization completes. That output includes `keyring_password`, `tss_password`, and `tss_p2p_private_key_hex`, which therefore appear in process output captured by init systems, container runtimes, and log shipping pipelines on every restart. Whatever values are present in the loaded configuration file—whether initial defaults or operator-supplied replacements are serialized in full on each launch. Second, upon completion of TSS signing and key-handling sessions, the session manager records hex-encoded signatures, public keys, and (for key sessions) a hash of the keyshare, placing per-event cryptographic material into the same structured log streams. Either channel increases the consequence of insufficient access controls on log storage, export policies, or diagnostic bundles.

The start path loads configuration from disk and prints the full marshalled document without redaction.

```
/cmd/puniversald/commands.go
```

```
loadedCfg, err := uvconfig.Load(home)
if err != nil {
    return fmt.Errorf("failed to load config: %w", err)
}

configJSON, err := json.MarshalIndent(loadedCfg, "", " ")
if err != nil {
    return fmt.Errorf("failed to marshal config: %w", err)
}
fmt.Printf("\n=== Loaded Configuration ===\n%s\n=====\n", string(configJSON))
```

The `Config` type carries credentials and key material with ordinary JSON field names, so they serialize verbatim.

```
/universalClient/config/types.go
```

```
// Keyring
KeyringBackend KeyringBackend `json:"keyring_backend"`
KeyringPassword string `json:"keyring_password"`
// ...
// TSS
TSSP2PPrivateKeyHex string `json:"tss_p2p_private_key_hex"`
TSSP2PListen string `json:"tss_p2p_listen"`
TSSPassword string `json:"tss_password"`
```

Disclosure of the keyring password and TSS password expands the attack surface for offline attempts against the keyring and against keyshare encryption, subject to the configured backend and filesystem protections.

Disclosure of the TSS libp2p private key in hexadecimal form permits impersonation of the validator's peer identity on the TSS network for any party that obtains log or stdout access without direct host compromise.

After outbound signing completes, the session manager logs the signature and public.

```
/universalClient/tss/sessionmanager/sessionmanager.go
```

```
func (sm *SessionManager) handleSignFinished(ctx context.Context, eventID string, result *dkls.Result, signingReq *common.UnsignedSigningReq) error {
    sm.logger.Info().
        Str("event_id", eventID).
        Str("signature", hex.EncodeToString(result.Signature)).
        Str("public_key", hex.EncodeToString(result.PublicKey)).
        Msg("signature generated from sign session")
}
```

Key protocol completion records the public key and a SHA-256 hash of the keyshare (the hash does not reveal the keyshare itself but still constitutes operational metadata at elevated sensitivity).

```
/universalClient/tss/sessionmanager/sessionmanager.go
```

```
keyshareHash := sha256.Sum256(result.Keyshare)
sm.logger.Info().
    Str("event_id", eventID).
    Str("protocol", protocolType).
    Str("storage_id", storageID).
    Str("public_key", hex.EncodeToString(result.PublicKey)).
    Str("keyshare_hash", hex.EncodeToString(keyshareHash[:])).
    Msg("saved keyshare")
```

A produced signature does not, by itself, authorize arbitrary new transactions without the corresponding signing key material however, persisting full signature values alongside identifiers at default severity elevates the confidentiality requirements on log repositories and may conflict with policies that restrict distribution of transaction-correlated cryptographic artifacts. Exposure facilitates correlation and forensic analysis by any principal with log read access beyond the minimum required for operations.

Status:

Fixed

Classification

Severity:

Info

Recommendations

Remediation:

Production builds should omit unconditional printing of the full configuration, or should emit a redacted summary that masks password fields and truncates or omits hex-encoded secrets. Credential delivery via environment variables or a secrets manager, with explicit policy against

echoing those values, aligns with common operational controls. Any diagnostic mode that reproduces full configuration should be disabled by default (for example behind an explicit `--print-config` flag), documented as unsuitable for production, and restricted to controlled environments. Logging should prefer identifiers such as `event_id` and, once available, the transaction hash, while relegating full signature octets to Debug level or omitting them from automated pipelines. Where retention of signature material is required for incident response, organizations may scope retention to restricted indices, role-based access, or complementary controls on fields that carry cryptographic content.

Resolution:

This issue has been addressed and resolved in commit [a00e7d6](#)

[F-2026-16877](#) - Chains.Start returns successfully when the Push native chain client is not attached - Info

Description:

During startup, `ensurePushChain` is invoked on a best-effort basis. If it fails, the implementation logs a warning and `Start` still returns a nil error. A background goroutine continues to call `fetchAndUpdate`, which retries attachment of the Push native client; until that succeeds, the process may run with external chain clients active while the Push native chain client is absent. Monitoring that infers full observation capability from process or generic health status alone may not detect that Push-originated ingress and egress events are not yet processed unless dedicated readiness checks exist.

`/universalClient/chains/chains.go`

```
// Always create push chain client first
if err := c.ensurePushChain(ctx); err != nil {
    c.logger.Warn().Err(err).Msg("failed to create push chain client; continuing")
}

go c.run(ctx)
return nil
```

Votes or processing for work scoped to the Push native client may be omitted or delayed until a later synchronization cycle succeeds.

Status:

Fixed

Classification

Severity:

Info

Recommendations

Remediation:

Provide a strict startup mode that exits with a non-zero status when the Push chain client is mandatory expose an explicit readiness signal so orchestration can wait until native observation is active.

Resolution:

This issue has been addressed and resolved in commit [634e234](#)

[F-2026-16994](#) - Nil Network on MsgUpdateUniversalValidator panics in validation and in the handler - Info

Description:

`MsgUpdateUniversalValidator` carries network metadata as an optional protobuf pointer. `ValidateBasic` forwards directly to `msg.Network.ValidateBasic()` without checking for nil, and the message server dereferences `msg.Network` when calling the keeper. A nil network therefore panics in the usual ante path or, if validation were bypassed, on the first keeper call. The add-validator message in the same module already returns `ErrInvalidRequest` when `Network` is nil, so the update path is inconsistent and likely an oversight rather than intentional.

File: `push-chain-node/x/uvalidator/types/msg_update_universal_validator.go` ·

```
func (msg *MsgUpdateUniversalValidator) ValidateBasic() error {
    if _, err := sdk.AccAddressFromBech32(msg.Signer); err != nil {
        return errors.Wrap(err, "invalid signer address")
    }

    return msg.Network.ValidateBasic()
}
```

File: `push-chain-node/x/uvalidator/keeper/msg_server.go` ·

```
func (ms msgServer) UpdateUniversalValidator(ctx context.Context, msg *types.MsgUpdateUniversalValidator) (*types.MsgUpdateUniversalValidatorResponse, error) {
    // ...
    validator, err := ms.k.StakingKeeper.GetValidator(ctx, valAddr)
    if err != nil {
        return nil, errors.Wrap(err, "signer is not a validator operator")
    }

    err = ms.k.UpdateUniversalValidator(ctx, validator.OperatorAddress, *msg.Network)
}
```

File: `push-chain-node/x/uvalidator/types/msg_add_universal_validator.go` ·

```
func (msg *MsgAddUniversalValidator) ValidateBasic() error {
    // ...
    if msg.Network == nil {
        return errors.Wrap(sdkerrors.ErrInvalidRequest, "network info is required")
    }

    return msg.Network.ValidateBasic()
}
```

Clients that omit `network` receive a generic panic-recovery failure instead of a clear invalid-request error. Internal or test callers that skip `ValidateBasic` hit the dereference in the handler. The mismatch with the add message increases support burden and hides the actual validation rule.

Status:

Fixed

Classification

Severity:

Info

Recommendations

Remediation:

- Add the same nil guard and error message to `MsgUpdateUniversalValidator.ValidateBasic` as on the add message.
- Guard `msg.Network` in `UpdateUniversalValidator` before logging or dereferencing, and return `ErrInvalidRequest` if absent.
- Optionally change the keeper API to accept `*types.NetworkInfo` so nullability is explicit in the type system.

Resolution:

This issue has been addressed and resolved in commit [a4ae809](#)

[F-2026-16996](#) - Unwired v2 migration overwrites every universal validator network identity with placeholders - Info

Description:

A version-two migration function rewrites the universal validator map from a legacy key set into the current `UniversalValidator` structure. For every migrated row it assigns the same hardcoded libp2p peer identifier and a single loopback multiaddr. The routine is not registered on the module configurator, and consensus version remains one, so it does not run in production builds today. The hazard is forward-looking: wiring this migration without replacing the placeholders would collapse every validator's advertised network identity to localhost and a shared peer id, which would break TSS and libp2p routing across the set. The file also invites copy-paste into a future upgrade handler.

File: `push-chain-node/x/uvalidator/migrations/v2/migrate.go` ·

```
func MigrateUniversalValidatorSet(ctx sdk.Context, k *keeper.Keeper
, cdc codec.BinaryCodec) error {
// ...
for ; iter.Valid(); iter.Next() {
valAddr, err := iter.Key()
// ...

newVal := types.UniversalValidator{
IdentifyInfo: &types.IdentityInfo{
CoreValidatorAddress: valAddr.String(),
},
NetworkInfo: &types.NetworkInfo{
PeerId: "12D3KooWFNC8BxiPoHyTJtiNlulctw3nSexuJHUBv4mMMmqEtQgg",
MultiAddrs: []string{"/ip4/127.0.0.1/tcp/39001/p2p/12D3KooWFNC8BxiPoHyTJtiNlulctw3nSexuJHUBv4mMMmqEtQgg"},
},
LifecycleInfo: &types.LifecycleInfo{
CurrentStatus: types.UVStatus_UV_STATUS_PENDING_JOIN,
History: []*types.LifecycleEvent{
{
Status: types.UVStatus_UV_STATUS_PENDING_JOIN,
BlockHeight: ctx.BlockHeight(),
},
},
},
}

if err := k.UniversalValidatorSet.Set(ctx, valAddr, newVal); err !=
nil {
return err
}
}
```

File: `push-chain-node/x/uvalidator/module.go` ·

```
const (
ConsensusVersion = 1
)
```

```
func (a AppModule) RegisterServices(cfg module.Configurator) {
types.RegisterMsgServer(cfg.MsgServer(), keeper.NewMsgServerImpl(a.
keeper))
types.RegisterQueryServer(cfg.QueryServer(), keeper.NewQuerier(a.ke
```

```
eper))  
}
```

No `RegisterMigration` call registers `MigrateUniversalValidatorSet`.

If the migration were enabled as written, every validator would share one peer id and one loopback address, so handshakes and routing would fail until operators issued updates for real network parameters. Recovery would be operationally heavy and concurrent with broken TSS connectivity. While the code is dead, retaining it without clear warnings risks accidental activation during a consensus-version bump.

Status:**Fixed**

Classification**Severity:****Info**

Recommendations**Remediation:**

- Replace placeholder `PeerId` and `MultiAddrs` with empty values or migration-supplied parameters read from genesis or governance, and block promotion to `ACTIVE` until real network info is present.
- Remove the migration file if it is obsolete, relying on version control for history.
- Add an integration or unit test that fails if a migration run would overwrite distinct network identities with identical literals.

Resolution:

This issue has been addressed and resolved in commit [82fd7fe](#)

[F-2026-17024](#) - Nil nested configs on add/update messages cause panics in validation and logging - Info

Description:

The add and update messages for chain and token configuration carry nested protobuf messages as pointers. `ValidateBasic` on each message calls `ValidateBasic` on the nested config without checking for a nil pointer first. The nested validators use value receivers and read fields immediately, so a nil pointer dereference panics during ante validation in typical flows. The message server handlers also log fields from the nested config before any nil check, which creates a second panic surface if validation were ever skipped or reordered. The practical outcome is failed transactions with panic-recovery messaging rather than a clear validation error, which harms integrators and obscures the root cause.

File: `push-chain-node/x/uregistry/types/msg_add_chain_config.go` ·

```
func (msg *MsgAddChainConfig) ValidateBasic() error {
    if _, err := sdk.AccAddressFromBech32(msg.Signer); err != nil {
        return errors.Wrap(err, "invalid signer address")
    }

    return msg.ChainConfig.ValidateBasic()
}
```

The same pattern appears in `msg_update_chain_config.go`, `msg_add_token_config.go`, and `msg_update_token_config.go` for `ChainConfig` and `TokenConfig` respectively.

File: `push-chain-node/x/uregistry/keeper/msg_server.go` ·

```
func (ms msgServer) AddChainConfig(ctx context.Context, msg *types.
MsgAddChainConfig) (*types.MsgAddChainConfigResponse, error) {
    ms.k.Logger().Info("msg add chain config received", "signer", msg.S
igner, "chain", msg.ChainConfig.Chain)
```

Analogous logging appears in `UpdateChainConfig`, `AddTokenConfig`, and `UpdateTokenConfig` before any nil guard on `msg.ChainConfig` or `msg.TokenConfig`.

Malformed or adversarial clients that omit the nested message cause panics recovered by the SDK, so the transaction fails with a generic recovered-panic error instead of `ErrInvalidRequest` with an explicit “config required” message. Tests and simulations that construct messages without the nested field see panics where they expect typed errors. Any future change that bypasses `ValidateBasic` in the ante chain would hit the logging path first and panic there as well.

Status:

Fixed

Classification

Severity:

Info

Recommendations

Remediation:

- In each affected `ValidateBasic`, return a wrapped `ErrInvalidRequest` when `ChainConfig` or `TokenConfig` is nil before delegating to nested validation.
- At the start of each handler, validate presence of the nested message before logging its fields, and return a typed error if it is missing.
- Optionally use pointer receivers on nested `ValidateBasic` methods so nil receives an explicit error path instead of an implicit dereference panic.
- Add unit tests that submit nil nested configs and assert a non-panic validation error for all four message types.

Resolution:

This issue has been addressed and resolved in commit [bf0c3e](#)

[F-2026-17035](#) - Unpaginated query walks and full-collection scan in GetTokenConfigByPRC20 - Info

Description:

Three query endpoints — `AllChainConfigs`, `AllTokenConfigs`, and `TokenConfigsByChain` — walk their respective collections in full and return the materialized slice in a single response. None of the corresponding request types carries a `cosmos.base.query.v1beta1.PageRequest`, so clients have no way to limit response size. `TokenConfigsByChain` additionally throws away the chain-prefixed storage key (`<chain>:<address>`) it could otherwise scan via a prefix range, filtering by `value.Chain == req.Chain` in memory after reading every entry. The same pattern repeats inside `GetTokenConfigByPRC20`, which is invoked from outbound creation and revert resolution in `x/uexecutor`: it walks the entire `TokenConfigs` map per outbound, paying $O(N_{\text{total_tokens}})$ DB reads even though only the entries for one chain are eligible matches.

File: `push-chain-node/x/uregistry/keeper/query_server.go` ·

```
func (k Querier) AllChainConfigs(goCtx context.Context, req *types.
QueryAllChainConfigsRequest) (*types.QueryAllChainConfigsResponse,
error) {
    ctx := sdk.UnwrapSDKContext(goCtx)
    var configs []*types.ChainConfig
    err := k.Keeper.ChainConfigs.Walk(ctx, nil, func(key string, value
types.ChainConfig) (stop bool, err error) {
        v := value
        configs = append(configs, &v)
        return false, nil
    })
    // ...
    return &types.QueryAllChainConfigsResponse{Configs: configs}, nil
}
```

`AllTokenConfigs` (lines 78–96) follows the identical unbounded pattern.

File: `push-chain-node/x/uregistry/keeper/query_server.go` ·

```
func (k Querier) TokenConfigsByChain(goCtx context.Context, req *ty
pes.QueryTokenConfigsByChainRequest) (*types.QueryTokenConfigsByCha
inResponse, error) {
    ctx := sdk.UnwrapSDKContext(goCtx)
    var configs []*types.TokenConfig
    err := k.Keeper.TokenConfigs.Walk(ctx, nil, func(key string, value
types.TokenConfig) (stop bool, err error) {
        if value.Chain == req.Chain {
            v := value
            configs = append(configs, &v)
        }
        return false, nil
    })
    // ...
}
```

The storage key already begins with `req.Chain + ":"`, so a prefix range would scope the walk to $O(N_{\text{chain}})$ instead of the current $O(N_{\text{total}})$.

File: `push-chain-node/x/uregistry/types/query.pb.go` ·

```

type QueryAllChainConfigsRequest struct {
}
// ...
type QueryAllTokenConfigsRequest struct {
}
// ...
type QueryTokenConfigsByChainRequest struct {
Chain string `protobuf:"bytes,1,opt,name=chain,proto3" json:"chain,
omitempty"`
}

```

No `Pagination` field on any of the three requests.

File: `push-chain-node/x/uregistry/keeper/keeper.go` .

```

func (k Keeper) GetTokenConfigByPRC20(
ctx context.Context,
chain string,
prc20Addr string,
) (types.TokenConfig, error) {
prc20Addr = strings.ToLower(strings.TrimSpace(prc20Addr))
var found *types.TokenConfig
err := k.TokenConfigs.Walk(ctx, nil, func(key string, cfg types.TokenConfig) (bool, error) {
if cfg.Chain != chain {
return false, nil
}
if cfg.NativeRepresentation == nil {
return false, nil
}
if strings.ToLower(cfg.NativeRepresentation.ContractAddress) == prc20Addr {
found = &cfg
return true, nil
}
return false, nil
})
// ...
}

```

This is called per outbound transaction from

`x/uexecutor/keeper/create_outbound.go` and `build_revert_outbound.go`, so the linear scan executes inside consensus, not only at query time.

`AllChainConfigs` and `AllTokenConfigs` are public read endpoints whose cost grows linearly with registry size; an automated crawler that hammers them produces $O(N)$ DB I/O per request, with the only mitigation being whatever RPC-layer rate-limiting the operator configures. `TokenConfigsByChain` is $O(N_{\text{total}})$ instead of $O(N_{\text{chain}})$, so a query for a small chain still pays the cost of the largest chain's token list. The most consequential consumer is `GetTokenConfigByPRC20`: it runs in the outbound and revert paths inside a transaction context, so its $O(N)$ cost is paid by every outbound on the bridge — block-time and gas-meter overhead scale linearly with the number of registered tokens across all chains. Adding pagination after launch is also a breaking proto change unless paired with a major version bump, which makes the omission a forward-compatibility trap as well.

Status:

Fixed

Classification

Severity:

Info

Recommendations**Remediation:**

- Add `cosmos.base.query.v1beta1.PageRequest` pagination to `QueryAllChainConfigsRequest`, `QueryAllTokenConfigsRequest`, and `QueryTokenConfigsByChainRequest`, and switch the handlers to `query.CollectionPaginate(...)` (or the equivalent `cosmossdk.io/collections.Paginate` helper).
- Rewrite `TokenConfigsByChain` to use a prefix range over the `<chain>:` key prefix instead of a full-collection walk with an in-memory filter.
- Introduce a secondary index `PRC20Index: collections.Map[{chain, lowercased_prc20_contract}, token_storage_key]`, populated and invalidated by `AddTokenConfig` / `UpdateTokenConfig` / `RemoveTokenConfig`. Rewrite `GetTokenConfigByPRC20` as a single index lookup followed by a single `TokenConfigs.Get`.
- Even without the reverse index, scope `GetTokenConfigByPRC20` to the chain prefix so the wrong-chain entries are skipped at the storage layer.
- Add a benchmark in the keeper test suite that seeds `K * M` tokens and asserts `GetTokenConfigByPRC20` reads remain constant under load.

Resolution:

This issue has been addressed and resolved in commit [f409cb](#)

[F-2026-17040](#) - TSS event status updates scan entire event store - Info

Description:

Updating or completing TSS-related events uses `TssEvents.Walk` with a full-store scan to locate rows by `process_id` and `event_type`, and `completePreviousActiveFinalizedEvent` walks **all** events to mark prior finalized keys completed. As the event history grows, these operations become linear in store size and are invoked from consensus-critical paths (for example TSS key finalization).

Single-row lookup by scanning all events — `push-chain-`

`node/x/utss/keeper/tss_events.go`

```
func (k Keeper) updateTssEventStatusByProcessId(ctx context.Context, processId uint64, eventType types.TssEventType, newStatus types.TssEventStatus) error {
    var foundId uint64
    var foundEvent types.TssEvent
    found := false

    err := k.TssEvents.Walk(ctx, nil, func(id uint64, event types.TssEvent) (bool, error) {
        if event.ProcessId == processId && event.EventType == eventType {
            foundId = id
            foundEvent = event
            found = true
            return true, nil // stop walking
        }
        return false, nil // continue
    })
```

Prior finalized events: collect all matches in one walk

```
err := k.TssEvents.Walk(ctx, nil, func(id uint64, event types.TssEvent) (bool, error) {
    if event.EventType == types.TssEventType_TSS_EVENT_KEY_FINALIZED && event.Status == types.TssEventStatus_TSS_EVENT_ACTIVE {
        toUpdate = append(toUpdate, struct {
            id uint64
            event types.TssEvent
        }{id: id, event: event})
    }
    return false, nil // continue walking to find all
})
```

These helpers are invoked from `VoteTssKeyProcess` finalization (for example `updateTssEventStatusByProcessId` and `completePreviousActiveFinalizedEvent` in `msg_vote_tss_key_process.go`).

Long-lived networks accumulate `TssEvents`; each finalization performs $O(n)$ work over that map. Under load, this increases transaction gas for voters and creates a mild self-DoS vector tied to history size rather than current validator count.

Status:

Accepted

Classification

Severity:

Info

Recommendations

Remediation:

1. Maintain a direct secondary index `(process_id, event_type) → event_id` for rows that must be updated cheaply, or store `last_process_initiated_event_id` on `TssKeyProcess`.
2. For “complete previous finalized” semantics, keep a single `current_active_finalized_event_id` pointer updated on each rotation instead of scanning all historical finalized events.
3. Add benchmarks or fuzz tests that measure gas with large `TssEvents` tables.

[F-2026-18133](#) - [Dual Defense] Same-Block Validator Jailing Can Snapshot an Oversized Ballot Quorum - Info

Description:

Cosmos SDK processes downtime jailing in BeginBlock while staking `IsBonded()` and `AfterValidatorBeginUnbonding` updates occur in EndBlock. A ballot whose first vote lands in that block can snapshot `N` eligible voters and threshold `(2N)/3+1` while one of those voters is already `Jailed=true`. After EndBlock, `GetEligibleVoters` self-heals for new ballots, but the existing ballot retains its frozen set until administrative `MsgRecomputeBallotQuorum` or expiry.

`CreateBallot` and `GetOrCreateBallot` snapshot `EligibleVoters` and `VotingThreshold` once at creation. `GetEligibleVoters` requires Universal Validator `ACTIVE` / `PENDING_JOIN`, staking `IsBonded()`, and not tombstoned, but does not check `Jailed`. UV demotion to `PENDING_LEAVE` is driven by `AfterValidatorBeginUnbonding` from the staking EndBlocker, not from `Jail()` itself.

This is a known operational edge case around validator set churn during Cosmos SDK block lifecycle ordering, not a fund-theft, minting, or consensus defect. Vote messages require a bonded Universal Validator and are not permissionless for arbitrary accounts.

Impact

- Temporary delay or drop of specific pending cross-chain ballots until recovery.
- No unauthorized mint, burn, or vault withdrawal.
- No consensus halt or double-spend.
- Subsequent ballots after EndBlock use the healed eligible set and finalize normally.

With four or more Universal Validators, a single same-block jail does not make the snapshotted threshold unreachable. The stranding condition applies only when `N ≤ 3` at ballot creation. Ballots remain recoverable via `MsgRecomputeBallotQuorum`, an admin-gated escape hatch for stuck pending ballots.

Status:

Fixed

Classification

Severity:

Info

Recommendations

Remediation:

1. Exclude `Jailed` validators in `GetEligibleVoters` in addition to `!IsBonded()`.
2. On UV leave hooks, optionally auto-recompute pending ballots with care for vote preservation.
3. Monitor pending ballots whose threshold exceeds current eligible count.

Resolution:

The issue has been addressed in commit [861fca0](#).

[F-2026-18136](#) - [Dual Defense] Unauthenticated Peers Can Exhaust TSS Stream Quota and Stall Outbound Signing - Info

Description:

The TSS libp2p listener installs `/push/tss/1.0.0` for every peer and admits streams from any valid libp2p identity. It never checks that the remote peer is a Universal Validator before negotiating the protocol and blocking on a framed read. An unprivileged attacker can open many slow streams from throwaway identities and occupy the inbound TSS stream quota until each read deadline expires.

`New` constructs the libp2p host with only an identity and listen addresses. There is no `ConnectionGater`, validator peer-ID allowlist, custom resource manager with reserved validator capacity, or per-IP admission control. `handleStream` sets a read deadline and immediately blocks in `readFramed` without calling `stream.Conn().RemotePeer()` against the validator set. Higher-layer authorization in the coordinator occurs only after a complete frame has been unmarshaled.

Only the current coordinator processes confirmed signing events. Participants create a local session, send an ACK, and wait for `BEGIN`. The coordinator broadcasts `BEGIN` only when every selected participant has ACKed. Denying required remote ACK deliveries prevents the session from starting.

Impact

While the attack is sustained against the current coordinator:

- No new outbound TSS signatures start.
- No new external-chain outbound finalization progresses through this path.
- Fund-migration and key-lifecycle sessions using the same protocol can stall.
- Unsigned vault exits remain stuck until ACKs can again reach the coordinator.

Already-signed transactions and Push Chain block consensus are not directly affected. The core impact is repeatable loss of cross-chain outbound processing capacity. This is an availability issue for outbound signing, not unauthorized minting or vault drain.

Status:

Fixed

Classification

Severity:

Info

Recommendations

Remediation:

1. Install a validator-aware `ConnectionGater` that rejects unknown peer IDs during secured-connection admission.
2. Maintain the allowed peer set from the authoritative Universal Validator snapshot, including active lifecycle transitions.
3. Recheck `stream.Conn().RemotePeer()` at the start of `handleStream` and immediately reset unauthorized streams.
4. Configure an explicit resource manager that reserves capacity for authenticated validator traffic.
5. Add per-IP limits on connections, streams, and incomplete handshakes; peer ID alone is not Sybil-resistant.
6. Bound concurrent stream reads with a semaphore or worker pool.
7. Add delivery acknowledgement and retry for critical `ACK`, `BEGIN`, and DKLS step messages.
8. Add regression tests proving unknown peers cannot consume TSS capacity and that legitimate validator messages remain deliverable during an unauthenticated flood.

Resolution:

The issue has been addressed in commit [4a1096d](#).

[F-2026-18137](#) - [Dual Defense] Unbounded Concurrent Debug Tracing Can Cause RPC Node Out-of-Memory - Info

Description:

`debug_traceTransaction`, `debug_traceBlockByNumber`, and `debug_traceCall` re-execute historical EVM work. Per-request knobs (`TraceConfig.Limit`, `maxTracePredecessors`, `defaultTraceTimeout`) do not cap how many traces run concurrently. Many concurrent heavy traces can grow resident memory until the host OOM-kills the node process.

Trace handlers in `rpc/namespaces/ethereum/debug` route to the backend and then to `x/vm/keeper` `TraceTx`, `TraceBlock`, and `TraceCall`, which re-execute without an in-process semaphore or worker pool for concurrent trace jobs. `maxTracePredecessors` only bounds predecessors on a single request.

Per-request timeout exists: `defaultTraceTimeout = 5 * time.Second` in `grpc_query.go` limits individual trace duration, not aggregate memory across parallel calls.

Testnet start scripts enable debug on the public JSON-RPC API list. `testnet/core/post-setup/start.sh` and `start_cosmovisor.sh` use:

```
--json-rpc.api=eth,txpool,personal,net,debug,web3
```

Nginx defines `limit_req_zone ... rate=30r/s` in `setup_nginx.sh`. That throttles request rate per IP; it does not guarantee a process-wide memory budget for overlapping traces, and distributed clients can still amplify load.

This behavior is consistent with known geth/OP-style `debug_trace*` memory characteristics rather than a Push-specific precompile defect.

Impact

Concurrent public debug traces can OOM a node process when the debug namespace is publicly enabled. The affected process dies until restart. Chain-wide consensus halt does not follow from a single node failure unless that node alone holds decisive voting power. No protocol fund theft or state corruption has been demonstrated. RPC callers incur no gas cost for trace requests.

Removing `debug` from public `--json-rpc.api` fully mitigates remote reachability. OOM of one RPC process is temporary single-node availability loss recoverable by restart.

Status:

Accepted

Classification

Severity:

Info

Recommendations**Remediation:**

1. Remove `debug` from public API lists in testnet and mainnet start scripts; bind debug to a private interface if needed.
2. Add a global semaphore or worker pool for `TraceTx`, `TraceBlock`, and `TraceCall` with reject-when-full behavior.
3. Optional per-IP concurrent-trace caps at the RPC server.

[F-2026-18148](#) - [Dual Defense] Removed Universal Validator Retains ChainMeta Vote Authority - Info

Description:

`MsgVoteChainMeta` uses lifecycle-blind `IsBondedUniversalValidator` and admits any bonded registered Universal Validator regardless of ACTIVE, PENDING_LEAVE, or INACTIVE status. Inbound ballots use `GetEligibleVoters` and reject PENDING_LEAVE at `AddVote`. Admin removal moves a validator to PENDING_LEAVE while stake may remain bonded; `AfterValidatorRemoved` prunes ChainMeta rows but does not revoke AuthZ or set membership, so a removed hotkey can reinsert votes after prune.

On a two-vote UV set, `upperMedianUint64` at `len/2` selects `MaxUint64` when one honest and one maximum vote are present. `LastAppliedChainHeight = MaxUint64` blocks later heights. Revert outbounds copy UniversalCore gas price; the EVM builder uses it, making refund transactions unfunded when the oracle price is manipulated. The same `MaxUint64` median outcome is already reachable by an ACTIVE UV casting the identical vote on the current two-UV topology; the incremental defect addressed here is incomplete revocation after removal.

`MsgRemoveUniversalValidator` is admin-gated. Self-unbond clears bonded status and fails `IsBondedUniversalValidator`; the working window is admin remove while bonded.

Impact

- PENDING_LEAVE validators can submit `MsgVoteChainMeta` after vote prune.
- AuthZ grants are not revoked on removal.
- On two-UV sets, `MaxUint64` ChainMeta price or height can stick via upper median.
- Refund transaction gas quotes may become unfunded when the builder uses a manipulated price.
- An ACTIVE UV on the same two-UV set can already achieve the identical median without the removal defect.
- ChainMeta or gas-field upgrade can reset manipulated oracle state.

Status:

Fixed

Classification

Impact Rate: 1/5

Likelihood Rate: 1/5

Severity:

Info

Recommendations**Remediation:**

1. Gate `MsgVoteChainMeta` on `GetEligibleVoters` status: ACTIVE or PENDING_JOIN only.
2. Revoke or expire voting AuthZ on validator removal.
3. Bound ChainMeta price and height inputs; reject `MaxUint64` and implausible jumps.
4. Provide an admin reset path for `LastAppliedChainHeight` and pending outbound gas fields.
5. Regression: PENDING_LEAVE cannot reinsert ChainMeta votes; evaluate majority or larger-set rules for oracle updates on small UV topologies.

Resolution:

The issue has been addressed in commit [bd3c2d8](#).

[F-2026-18787](#) - [Dual Defense] IBC v2 dest_callback Memo Without Callbacks Middleware - Info

Description:

`evmd/app.go` wires the IBC classic transfer stack as `transfer → erc20 → callbacks`, and the IBC v2 transfer stack as `transfer-v2 → erc20-v2` only. That static asymmetry is real: `ibccallbacks.NewIBCMiddleware` is applied only to the v1 stack; no `callbacks/v2` middleware is registered on `ibcRouterV2`.

Destination-callback safety (execute callback or fail the receive and open refund) is provided only when callbacks middleware is on that channel stack. Without it, `dest_callback` in the ICS-20 memo is opaque packet metadata. The transfer application credits the packet `receiver` field and does not interpret the memo as a callback opt-in.

The isolated callback receiver is not assigned by the transfer or erc20-v2 modules. Module docs and `ContractKeeper.IBCReceivePacketCallback` require the packet sender to set `receiver` equal to `GenerateIsolatedAddress(destChannel, sender)`. Callbacks middleware then validates that equality before approving ERC-20 and invoking the target contract. On the v2 route, that keeper path never runs, so funds land at the isolated address only if the sender already chose that address as `receiver`.

That is equivalent to a voluntary send to an unreachable account with an ignored memo—not a permissionless freeze of third-party or vault funds, and not a broken "callback-or-refund" invariant on a stack that never installed callbacks.

`x/erc20/v2` does not implement `api.PacketDataUnmarshaler` / `CallbacksCompatibleModuleV2`, so v2 callbacks were not productized on this tree. Treat as incomplete feature parity, not a silent break of a shipped v2 callback guarantee.

Impact

- IBC v2 transfer route omits callbacks middleware; v1 includes it.
- a v2 packet whose `receiver` is the isolated address credits that address and can ack success; callback execution does not run on that route.
- Attacker-driven permanent freeze of funds the attacker does not control is not established; `receiver` is sender-controlled packet data.
- No shipped Push precompile or send path auto-rewrites v2 receivers to the isolated address when `dest_callback` is present.

Status:

Accepted

Classification

Severity:

Info

Recommendations

Remediation:

1. Document that EVM dest-callbacks are supported only on the IBC classic transfer stack as currently wired.
2. If v2 dest-callbacks are a product goal: implement `CallbacksCompatibleModuleV2` on the v2 stack (erc20-v2 must forward `UnmarshalPacketData`), wrap `ibc-go modules/apps/callbacks/v2`, and add integration tests for execute-or-fail-ack.
3. Optionally reject or warn in client/SDK tooling when `dest_callback` is set on a v2 path, or refuse packets that combine `dest_callback` with the isolated receiver while callbacks MW is absent.

[F-2026-18788](#) - [Dual Defense] Unauthenticated TraceCall Timeout Stops Tracer Only - Info

Description:

Cosmos query `Query/TraceCall` (`grpc_query.go`) accepts client `GasCap` (`uint64`), applies it via `CallDefaults`, and runs `traceTxWithMsg`. On deadline expiry the handler calls only `tracer.Stop("execution timeout")`. That stops structured-logger collection; it does not invoke go-ethereum `EVM.Cancel()` or otherwise abort `ApplyMessageWithConfig`. Execution therefore continues until EVM gas exhaustion. `buildTraceCtx` installs an infinite Cosmos gas meter parameterized by the message gas limit, so the query gas meter does not bound wall-clock work.

That cancellation gap is inherited upstream cosmos/evm (same `tracer.Stop` -only pattern in v0.6.0 `grpc_query.go`). It is query/RPC hygiene, not Push-specific business logic.

Blast radius depends on exposing the Cosmos gRPC/REST query server on validator hosts. Industry and Cosmos practice isolate validator processes from public query APIs (sentry/RPC nodes). JSON-RPC eth/debug call paths apply `Backend.RPCGasCap()`; the unbounded client `GasCap` is specific to the raw Cosmos `QueryTraceCallRequest` surface. Concurrent CPU/memory pressure on a publicly exposed query node is single-node (or RPC-tier) availability loss—not demonstrated chain-wide finality failure without assuming quorum-wide public exposure of validator query ports.

Impact

- timeout path calls `tracer.Stop` only; no `EVM.Cancel()` in Push or matching upstream cosmos/evm.
- client `GasCap` is applied without an immutable server-side max on the Cosmos TraceCall query.
- `CallDefaults` with `gas_cap == 0` and missing `args.Gas` can set gas to `MaxUint64/2`.
- Permissionless halt of Push finality requires public exposure of query gRPC on enough consensus-critical hosts to starve CometBFT—an ops/deployment precondition.
- No fund theft, mint, or state corruption.

Status:

Accepted

Classification

Severity:

Info

Recommendations

Remediation:

1. Ops: do not expose Cosmos query gRPC/REST Trace* on validator public interfaces; prefer private debug/query listeners.
2. Server-side immutable max gas for TraceCall/TraceTx/TraceBlock (ignore or clamp client `GasCap`).
3. On deadline/cancel, call EVM `Cancel()` (or equivalent) so interpreter stops near the deadline.
4. Bound concurrent traces and struct-logger `Limit` default away from unlimited.
5. Optionally unregister Trace* from the public query router.

[F-2026-18789](#) - [Dual Defense] First Fund Migration id=0 Is Unvotable

- Info

Description:

`NextMigrationId` is a `collections.Sequence` (go.mod pins `cosmos-sdk.io/collections` v0.4.0). On an unset store, `Peek` / `Next` use `DefaultSequenceStart = 0`, so the first `InitiateFundMigration` stores `FundMigration` / `PendingMigrations` under `id=0`.

`MsgVoteFundMigration.ValidateBasic` rejects `MigrationId == 0` ("migration_id is required"), so votes never reach `VoteFundMigration` / ballot creation.

`PendingMigrations.Remove` runs only after a finalized vote; there is no Begin/EndBlock sweep or admin cancel. `InitiateFundMigration` then refuses another PENDING migration for the same chain—so that chain's migration lane stays blocked until an upgrade (or genesis re-init that seeds the sequence / clears pending).

Genesis does not fix this by default: `DefaultGenesis` leaves `NextMigrationId` at protobuf 0, and `InitGenesis` only `Set`s the sequence when `data.NextMigrationId > 0`.

The id=0 vs `ValidateBasic` collision is real tooling debt.

`MsgInitiateFundMigration` requires `params.Admin`. No permissionless trigger; admin must invoke the broken first migrate. Impact is per-chain migration lane only; does not halt current-key outbound, consensus, or unrelated PRC20. Preemptive fix is one genesis/sequence seed before any migrate; if already stuck, upgrade clears pending.

Impact

- first `NextMigrationId.Next` can be `0`; votes reject id 0; pending never clears; same-chain re-initiate blocked.
- admin-gated initiate; no EndBlock expiry of unvoted migrations.
- Admin-only self-brick on the migration ops lane; no user-facing vault/outbound halt.
- Other chains' migration lanes remain independently usable if their first id was never created as 0 (or sequence already advanced).

Status:

Fixed

Classification

Severity:

Info

Recommendations

Remediation:

1. Seed sequence at genesis to `1` (or `Set(1)` in `InitGenesis` when unset).
2. Or stop rejecting `migration_id == 0` in `ValidateBasic` if 0 is a valid id.
3. Or allocate ids with `Next()+1` / custom start so stored ids are never 0 while 0 remains "unset" in API.
4. Add admin cancel / upgrade handler to remove stuck `PendingMigrations` entries.
5. Regression: first `InitiateFundMigration` must yield a votable id and clear pending on finalized vote.

Resolution:

The issue has been addressed in commit [d5d8ee8](#).

[F-2026-18792](#) - [Dual Defense] SVM revert_universal_tx /

funds_rescued → Success: true Matches Instruction Completion - Info

Description:

SVM gateway outbound logs for `finalize_universal_tx`, `revert_universal_tx`, and `funds_rescued` share `parseOutboundObservationEvent`, `common.OutboundEvent` stores only ids + `gas_fee_used`, and `buildOutboundObservation` hardcodes `Success: true` / empty `ErrorMsg`.

`TxBUILDER.determineInstructionID` maps:

Instruction	Push TxType
<code>finalize_universal_tx</code> (id 1/2)	<code>FUNDS</code> / <code>FUNDS_AND_PAYLOAD</code> / <code>GAS_AND_PAYLOAD</code>
<code>revert_universal_tx</code> (id 3)	<code>INBOUND_REVERT</code>
<code>rescue_funds</code> (id 4)	<code>RESCUE_FUNDS</code>

`revert_universal_tx` and rescue are separate TSS-signed outbounds whose job is destination refund / rescue. Emitting those program events means that instruction completed on Solana for that outbound id — the correct Push observation is success (terminalize + gas refund via `handleSuccessfulOutbound`), not `Success: false` remind of a FUNDS burn.

Genuine FUNDS failure does not emit a successful finalize log as a "failure event." Failure is "no successful destination execution": failed/missing Solana tx, absent `ExecutedTx` PDA past deadline → resolver failure vote path (`resolveSVM`).

`handleFailedOutbound` reminds PRC20 only for funds-related TxTypes (`FUNDS` / `FUNDS_AND_PAYLOAD` / `GAS_AND_PAYLOAD`). Voting failure on a completed `INBOUND_REVERT` / `RESCUE_FUNDS` outbound would mis-terminalize a successful revert/rescue outbound.

The EVM outbound path uses the same `buildOutboundObservation` hardcode: observation events are confirmed only after a successful receipt; failed receipts are marked REVERTED and never voted as success.

Impact

- No demonstrated path where a failed FUNDS SVM outbound is finalized as success because `revert_universal_tx` / `funds_rescued` share the success builder.
- those events are completion signals for `INBOUND_REVERT` / `RESCUE_FUNDS` outbounds; `Success: true` is the intended observation.
- FUNDS remind runs only on `obs.Success == false` for funds TxTypes; that path is driven by failure votes (e.g. missing PDA), not by reinterpreting revert/rescue completion logs as FUNDS failure.

- Residual: payload omits event/discriminator; improve observability without changing success semantics for completion logs.

Status:

Accepted

Classification

Severity:

Info

Recommendations

Remediation:

1. No change to set `Success=false` for revert/rescue logs: that would mis-finalize successful revert/rescue outbounds.
2. Optional hygiene: retain gateway event name/discriminator on `OutboundEvent` for metrics and defense-in-depth assertions (e.g. expect finalize only for FUNDS TxTypes).
3. Keep FUNDS failure signaling on durable absence-of-execution (PDA / failed tx).

Observation Details



Disclaimers

Hacken Disclaimer

The blockchain protocol given for audit has been analyzed based on best industry practices at the time of the writing of this report, with cybersecurity vulnerabilities and issues in the protocol source code, the details of which are disclosed in this report (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

The report contains no statements or warranties on the identification of all vulnerabilities and security of the code. The report covers the code submitted and reviewed, so it may not be relevant after any modifications. Do not consider this report as a final and sufficient assessment regarding the utility and safety of the code, bug-free status, or any other protocol statements.

While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only — we recommend proceeding with several independent audits and a public bug bounty program to ensure the security of the blockchain protocol.

English is the original language of the report. The Consultant is not responsible for the correctness of the translated versions.

As part of Hacken's ongoing quality assurance process, we may conduct re-audits of select projects. These re-audits are performed independently from the original audit and are intended solely for internal quality control and improvement. Updated reports resulting from such re-audits will be shared privately with the respective clients and may be published on the Hacken website only with their explicit consent.

The sole authoritative source for finalized and most up-to-date versions of all reports remains the Audits section at <https://hacken.io/audits/>.

Technical Disclaimer

Blockchain protocols are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the protocol can have vulnerabilities that can lead to hacks. Thus, the Consultant cannot guarantee the explicit security of the audited blockchain protocol.

Appendix 1. Severity Definitions

Severity	Description
Critical	Vulnerabilities that can lead to a complete breakdown of the blockchain network's security, privacy, integrity, or availability fall under this category. They can disrupt the consensus mechanism, enabling a malicious entity to take control of the majority of nodes or facilitate 51% attacks. In addition, issues that could lead to widespread crashing of nodes, leading to a complete breakdown or significant halt of the network, are also considered critical along with issues that can lead to a massive theft of assets. Immediate attention and mitigation are required.
High	High severity vulnerabilities are those that do not immediately risk the complete security or integrity of the network but can cause substantial harm. These are issues that could cause the crashing of several nodes, leading to temporary disruption of the network, or could manipulate the consensus mechanism to a certain extent, but not enough to execute a 51% attack. Partial breaches of privacy, unauthorized but limited access to sensitive information, and affecting the reliable execution of smart contracts also fall under this category.
Medium	Medium severity vulnerabilities could negatively affect the blockchain protocol but are usually not capable of causing catastrophic damage. These could include vulnerabilities that allow minor breaches of user privacy, can slow down transaction processing, or can lead to relatively small financial losses. It may be possible to exploit these vulnerabilities under specific circumstances, or they may require a high level of access to exploit effectively.
Low	Low severity vulnerabilities are minor flaws in the blockchain protocol that might not have a direct impact on security but could cause minor inefficiencies in transaction processing or slight delays in block propagation. They might include vulnerabilities that allow attackers to cause nuisance-level disruptions or are only exploitable under extremely rare and specific conditions. These vulnerabilities should be corrected but do not represent an immediate threat to the system.

Appendix 2. Scope

The scope of the project includes the following components from the provided repository:

Scope Details	
Repository	https://github.com/pushchain/push-chain-node
Commit	ebaa89ddc70fdf9b7d486d89add0e298c44b8973
Repository	https://github.com/pushchain/push-chain-evm
Commit	9570d87d7b5eaff643885eec380b9275c85a4638

Appendix 3. Additional Valuables

Frameworks and Methodologies

This security assessment was conducted in alignment with recognised penetration testing standards, methodologies and guidelines, including the [NIST SP 800-115 – Technical Guide to Information Security Testing and Assessment](#), and the [Penetration Testing Execution Standard \(PTES\)](#). These assets provide a structured foundation for planning, executing, and documenting technical evaluations such as vulnerability assessments, exploitation activities, and security code reviews. Hacken's internal penetration testing methodology extends these principles to Web2 and Web3 environments to ensure consistency, repeatability, and verifiable outcomes.